

# Basic Concepts

This page introduces the **main features** of the **Character Management System**:

- [Types of Characters](#) – The two fundamental different kinds of characters.
  - [Character Type Assets](#) – The core of every character.
  - [The Character Shader](#) – How characters are rendered in-game.
  - [Character Templates](#) – Blueprints for creating characters later.
  - [Character Usage](#) – Components to load and render characters.
  - [Built-in Modular Characters](#) - Modular characters pre-setup and ready for use.
  - [Character Grouping System](#) - Groups used to save and manage characters.
  - [Character Creator](#) – Modular UI framework for building customizable characters in-game.
- 

## Types of Characters



The **Character Management System** contains two types of characters for different use cases.

### Unified Characters

A **Unified Character** uses a **single spritesheet** containing the full character with all animations and frames.

**Unified Characters** are easy to setup and use but lack runtime customization since the character is pre-created in one spritesheet.

### Layered Characters

A **Layered Character** combines **multiple spritesheets** into one at runtime to create the final character. Every spritesheet must be the same size and contain the same frame sizes and positioning to work properly.

Each spritesheet is layered on top of another in order at runtime.

For example: If we had a character with four layers: **Body, Outfit, Hairstyle and Accessory**.

When the character is used, the body layer will be rendered first, then the outfit on top of that, then the hairstyle and finally the accessory layer.

You have full control over the order the layers are rendered in.



## Character Type Assets

Every character is defined by a **Character Type asset**, which is a [Scriptable Object](#)

A Character Type:

- Defines if a character is **Unified** or **Layered**
- Stores the **Base Spritesheet**
- References a single **shared Animator Controller** for all characters to use (Optional)

## Base Spritesheet

The **Base Spritesheet** defines the layout that all other spritesheets must follow.

This spritesheet is sliced into multiple frames, these are the frames that are used for any character using the same Character Type asset.

## The Character Shader

A shader is used to visually display a character over the **Base Spritesheet**.

Sprites from the **Base Spritesheet** are used directly in a renderer component (such as a **Sprite Renderer**) or used in an **Animator Controller**.

If a **Unified Character** is used, the shader overrides the **Base Spritesheet** with the single spritesheet of the character.

If a **Layered Character** is used, the shader combines all layers into one spritesheet and overrides the **base Spritesheet** with the new spritesheet.

The renderer component is only ever using sprites from the **Base Spritesheet** meaning all other spritesheet do **NOT** need to be sliced, they should instead have a **Sprite Mode** of **Single**.

This approach saves a ton of time by avoiding the headache of slicing spritesheets and setting up Animator Controllers for every character.

**i NOTE**

If a [Character Renderer](#) component is used, the shader will be applied automatically.

## Character Templates

A **Character Template** is a [Scriptable Object](#) that acts as a **blueprint** for creating characters later at runtime. Templates are supported for both **Unified** and **Layered Characters**.

Templates allow you to create a character once and reuse it anywhere it in your project just by referencing it.

Additionally templates provide extra functionality for **Layered Characters** allowing you to **randomize specific layers** based on pre-set rules you define in the editor.

[Read More → Character Templates](#)

## Character Usage

The Character Management System contains components for loading and rendering characters, making it easy to spawn new characters or reuse existing ones.

## Character Renderer Components

**Character Renderer components** are responsible for loading and showing a character in-game.

**Renderer components handle:**

1. Creating/loading a character
2. Applying the **Character Shader**
3. Assigning the **Animator Controller** if applicable
4. Managing **Overlay Layers**

| Renderer Component                                     | Purpose                                                                              |
|--------------------------------------------------------|--------------------------------------------------------------------------------------|
| <a href="#">Layered Character Template Renderer</a>    | Create and show a <b>Layered Character</b> from a template.                          |
| <a href="#">Unified Character Template Renderer</a>    | Create and show a <b>Unified Character</b> from a template.                          |
| <a href="#">Random Layered Character Renderer</a>      | Create and show a new <b>Layered Character</b> with completely random layer options. |
| <a href="#">Layered Character Group Entry Renderer</a> | Load and render a saved <b>Layered Character</b> from a group.                       |

## Built-in Characters

The **BlazerTech Character Management System** includes fully configured modular characters which can be used in any commercial or non-commercial project.

The **BlazerTech Modular Characters** consist of four layers:

1. **Body**
2. **Outfit**
3. **Hairstyle**
4. **Accessory**

Included:

- Layered Character Type
- Unified Character Type
- A few Layered and Unified sample templates

These characters will also be purchasable separately upon the full release of the Character Management System

[Read More → Built-In Characters](#)

## Character Grouping System

Charcter groups are used to sort characters. They're great for organizing characters into meaningful collections, whether that's for a dynamic roster or a fixed group size.

[Read More → Character Grouping System](#)

Two types of groups exist.

## Flexible Group Type

A dynamic list that characters can be added to, removed from, or edited at anytime.

Example Uses:

- A roster of playable characters the player can create, edit, and delete.
- A collection of background NPCs that will later be randomly selected from.

[Read More → Flexible Group Type](#)

## Fixed Group Type

An immutable list of characters. When the list is created, all characters are created immediately. Characters can then be edited but not removed and new characters cannot be added.

Example Uses:

- A predined set of characters the player can choose to play as.
- A set of main characters the player can customize.

[Read More → Fixed Group Type](#)

---

## Character Creator

The **Character Creator** is a prefab based **Character Creation Menu Framework**.

Prefabs can be combined and customized to create whatever design you want.

It makes the process of building a **Character Creation Menu** into your game easy.

### TIP

The **Character Creator** only works with **Layered Characters**. **Unified Characters** do not support runtime customization.

## Example Use Cases

1. **Customizable Player Character** – Easily setup the menu for a single character such as the player character.
2. **Editing Character Lists** – Allow players to edit a predefined roster, or manage a dynamic list (create, edit, delete).

## Key Features

| Feature                  | Description                                                         |
|--------------------------|---------------------------------------------------------------------|
| <b>Layer Selectors</b>   | Dropdowns, carousels, tabs, etc.                                    |
| <b>Character Preview</b> | Static or animated, with options to rotate or swap animations.      |
| <b>History Tracking</b>  | Every change is logged and can be shown as text or image snapshots. |
| <b>Randomization</b>     | Randomize the entire character or specific layers.                  |
| <b>Loading Screens</b>   | Customizable loading screens which hide the menu until it's ready.  |
| <b>Character Naming</b>  | Optional name field.                                                |

[Read More → Character Creator](#)

# Installation and Requirements

This guide covers the **requirements** and **installation steps** for the **BlazerTech Character Management System** (BT-CMS).

## Required Packages

### NOTE

All required packages must be installed and correctly configured **before importing BT-CMS**.

| Package                   | Purpose                                            | Notes / Links                                                                                                                                                                                                                                        |
|---------------------------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Addressables</b>       | Dynamically load/unload character spritesheets.    | Found in the <b>Unity Registry</b> via Package Manager.                                                                                                                                                                                              |
| <b>Naughty Attributes</b> | Adds additional attributes to the Unity Inspector. | <a href="#">Installation Guide</a> <br><a href="#">Unity Asset Store Link</a>  |

### IMPORTANT

**Addressables Settings file** must be created **before** importing BT-CMS. If not created beforehand the included characters will not be functional. To create the settings file navigate to: **Window > Asset Management > Addressables > Groups > Create Addressables Settings**.

## Installation Steps

### Prerequisites

Make sure **Naughty Attributes** and **Addressables** are both installed and the **Addressables Settings file** has been created ([See above^^](#)).

### Importing

**Itch.io:**

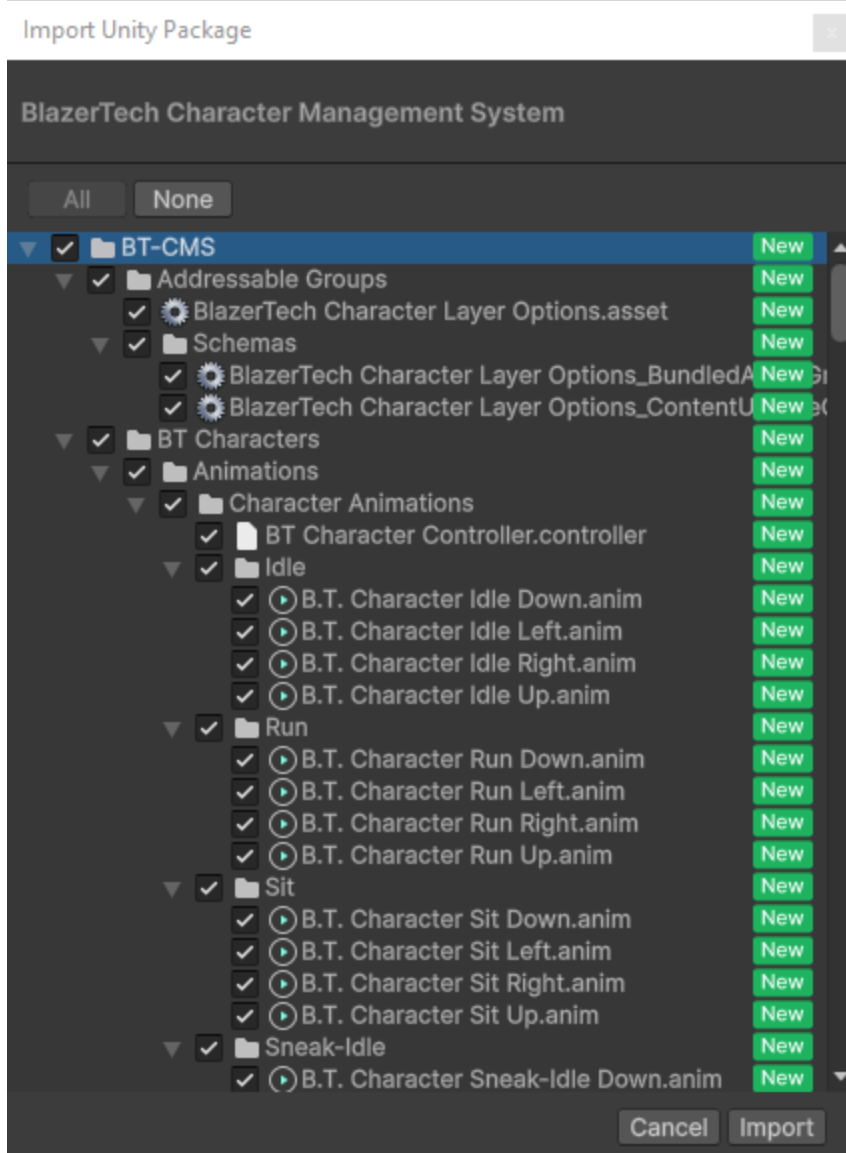
- If bought from Itch.io, download **BlazerTech Character Management System.unitypackage** and drag and drop it into your project.

**Unity Asset Store:**

- If bought from the Unity Asset Store, navigate to **Window > Package Manager > My Assets** and find **BlazerTech Character Management System** from within the list. Click **Download** if not already downloaded and then **Import**.

### Import Popup:

- Regardless of the installation method, a popup will appear prompting you to import the **BlazerTech Character Management System** into your project. Make sure all assets are selected and click the **Import** button.



## After Installtion

Here's some things to do after installing the **BlazerTech Character Management System**:

1. Check out the **Quick Start Guide** to learn how to setup your own characters or use the included ones.

[Read More → Quick Start Guide](#)

2. **Play sample scenes** - Go into the **Samples** folder and run some of the sample scenes to learn more about the Character Management System.
3. **Create your own character type** - make a new character type asset to use your own character spritesheets or third party ones.  
[Read More → Character Types](#)
4. **Implement a character creation menu** - Use the modular Character Creator System to create your own in-game character creation menu.  
[Read More → Character Creator Overview](#)

# Quick Start Guide



Welcome to the **BlazerTech Character Management System**!

This guide will get you on your feet and teach you how to setup the core for all your characters along with creating and rendering your first character.

This guide will cover the setup process for both **Unified** and **Layered Characters**.

## TIP

It's recommended to read through the [Basic Concepts](#) before reading the **Quick Start Guide**.

## Steps

1. [Create A Character Type](#)
2. [Create A Character Template](#)
3. [Use Your Character](#)

If this is your first time, start with the first step and work your way down.

## Video Guides

Prefer video guides? I've got you covered!

## BlazerTech Character Management System | Layered Character Type

Legendary Swordsman2



Watch on

### Covers:

- Layered Character Type Setup.
- Layered Character Template Setup.
- Rendering a layered character from a template.

## BlazerTech Character Management System | Unified Character Type

Legendary Swordsman2



Watch on

### Covers:

- Unified Character Type Setup.
- Unified Character Template Setup.
- Rendering a unified character from a template.

 **NOTE**

These videos will become dated fast as I continue to improve the **Character Managemnt System**, meaning the content in them may not always be 100% accurate to the current version.

Finalized versions of these videos will be created close to the full release.

# Create A Character Type

A **Character Type** is an asset which stores the **fundamental information needed for your characters**.

This guide will walk you through how to create and set one up.

[Read More → Character Type Assets](#)

## *i* NOTE

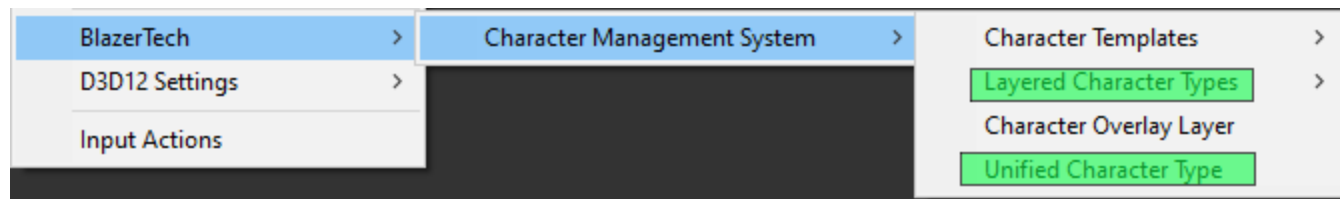
If you're using the built-in **BlazerTech Modular Characters** or other pre-configured characters, skip to [Creating a Character Template](#).

## 1 Create The Character Type Asset

To Create a Character Type asset:

1. **Right Click** the **Project window**
2. Navigate to: **Create > BlazerTech > Character Management System >**
3. Choose either **Layered** or **Unified Character Type**

this asset can be named anything you want.



## Choosing a Character Type

The **Character Management System** includes two types of characters, **Unified and Layered characters**.

A **Unified Character** is contained entirely in one spritesheet, this spritesheet has every animation and every frame for the character.

Unified Characters are simple to setup but lack customization options due to the character being pre-created.

A **Layered Character** is split into multiple spritesheets where each spritesheet is a visual layer of the character.

An example of a layered character could be as follows: **Body > Eyes > Outfit > Hairstyle > Accessory**

Where each spritesheet is layered on top of the previous one at runtime to create the final character you see in-game.

When you create a Character Type asset you must select either the Unified or Layered version. Both can be created if you wish to utilize both Unified and Layered characters.

[Read More → Choosing a Character Type](#)

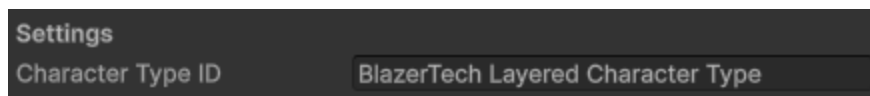
---

## 2 Setup the Character Type Asset

Two fields are required to be set for the asset to be useable.

### Field 1: Character Type ID

This is a **unique** identifier for this Character Type. Make sure it's **not** the same as any other Character Types.



### Field 2: Base Spritesheet

This spritesheet determines the layout of all character spritesheets using this same type. All other character spritesheets must be the same size and contain the same frames contained within the **Base Spritesheet**.

The **Base Spritesheet** should be your character in its most basic barebones state.



For more information about the **Base Spritesheet** [read here](#).

### Optional Field: Animator Controller

An **Animator Controller** can be assigned in your **Character Type asset** which will be used automatically when a character is rendered in-game.

This is not required if you have your own system for animating your characters.

Animations should only use sprites from the **Base Spritesheet**, otherwise they won't be rendered correctly. [Read more](#).

If you're using your own **movement** or **animator handling scripts**, you can configure the **Animator Controller** however best fits your system. However the **Character Management System** also includes **built-in movement** and **animator handler scripts** Which require the **Animator Controller** be setup with specific parameters

- [Read More → Character Animation Setup](#)

Later in this guide when you learn how to render your character, you'll see how the **Animator Controller** can be automatically applied when the character is used.

- [Read More → Animator Controller](#)

**i NOTE**

**No further setup is required for Unified Character Types.**

## 3 Layered Character Type Asset Setup

Layered Character Types require that we define what layers a character contains.

Layerd Characters are composoed of multiple layers where each layer is stacked on top of each other in order to create the final character.

### **E.G. Body > Outfit, Hairstyle > Accessory**

The body is rendered first, then the Outfit, then the Hairstyle and finally the Accessory.

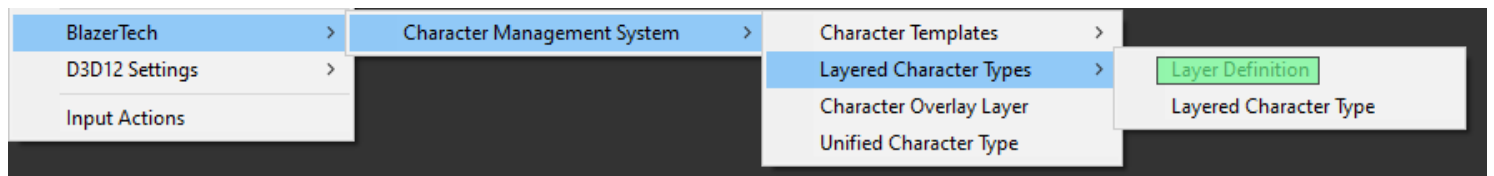
Each layer is represented as its own **Character Layer Definition asset**.

This asset contains all spritesheets (Layer Options) that can be used for that layer of the character.

## Create Layer Definitions

To create a new Layer Definition:

1. **Right click** the **Project window**
2. Navigate to: **Create > BlazerTech > Character Management System > Layered Character Type > Layer Definition**



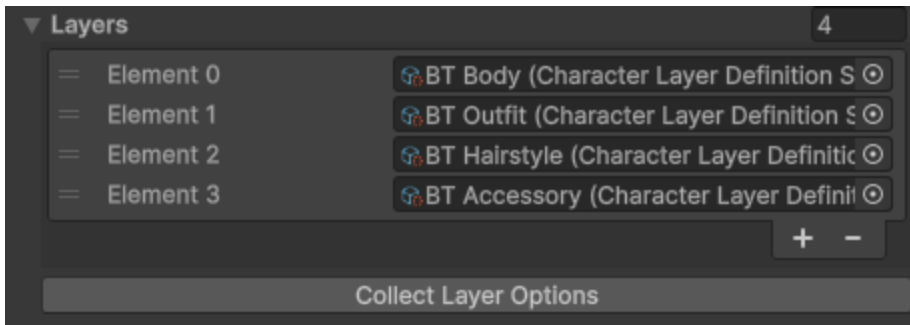
Create a new **Layer Definition asset** for each layer you want your characters to have.

Refer to [Adding Layer Options](#) to learn how to add your spritesheets as options for your layer.

## Assign Layer Definitions

Once all your **Character Layer Definition assets** have been created they need to be **assigned in your Layered Character Type**.

1. Open your **Layered Character Type asset** in the **inspector**
2. locate the **Layers** list.
3. Add all your **Layer Definitions** to that list.



The **order of the layers in this list determines the render order**.

Layers are rendered **from top to bottom in the list**, meaning:

- Top of List = Rendered First (Behind)
- Bottom of List = Rendered Last (In Front)

---

## Next Steps

Learn how to create your first Character Template which you can then create characters from during runtime.

[Read More → Create A Character Template](#)

# Create A Character Template

There are a multitude of ways to create characters in the **Character Management System**, however for this guide we're going to focus on **Character Templates**.

Think of a **Character Template** as a blueprint to create a character during runtime. The Template defines the **name** of the character and the **spritesheet(s)** used.

## Create Character Template Asset

To create a Character Template asset:

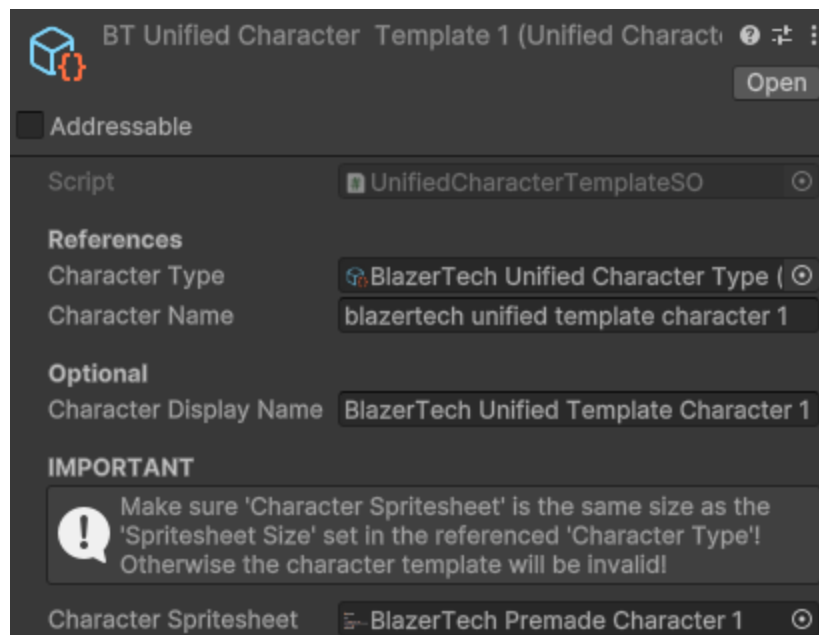
1. **Right Click** the **Project window**
2. Navigate to: **Create > BlazerTech > Character Management System > Character Templates >**
3. Select either **Layered** or **Unified Character Template**

---

## Unified Character Template Setup

A **Unified Character Template** requires:

1. A reference to a **Unified Character Type**.
2. A **name** for the character when it gets created.
3. A reference to the **spritesheet** the character will use.



## Spritesheet Requirements:

- Must be the exact same size as the **Base Spritesheet** assigned in the **Character Type** and contain the same animations.

- **Sprite Mode:** `Single` - We won't be using the spritesheet directly. It'll be passed to the [Character Shader](#).
- **Filter Mode:** `Point (No Filter)`.
- (Optional) **Compression:** `None` - Generally not needed for pixel art.

Once finished, skip to [Next Steps](#).

- [Read Also → Unified Character Templates](#)
- 

## Layered Character Template Setup

Start by setting the basic required fields:

1. A reference to a **Layered Character Type**.
2. A **Name** for the character when it gets created.
3. Optionally a **Display Name** that can be displayed along with the character.

BT Character Template 1 (Layered Character Template)    

☐ Addressable Open

**References**

Character Type  BlazerTech Layered Character Type 

Character Name blazertech template character 1

**Optional**

Character Display Name BlazerTech Template Character 1

**Body**

Search

Options (7) Body\_01\_Peach ▼

**Outfit**

Search

Options (29) Outfit\_05\_Dress\_01 ▼

**Hairstyle**

Search

Options (29) Hairstyle\_02\_Shortlock\_04 ▼

**Accessory**

Search

Options (30) Accessory\_03\_Straw-Hat\_01 ▼

Recreate List

## Assign Layer Options

Once a **Layered Character Type** has been assigned, a list of **Layers** will appear. These are the same layers you assigned in your **Layered Character Type**.

Each entry in the list has an assigned **Layer Option**, that's the **spritesheet** that'll be used for that layer of the character.

Selecting an entry will open a dropdown containing all **valid Layer Options** that can be selected. This is the same list of **Layer Options** contained in each **Character Layer Definition asset**.

The image shows a dark-themed user interface for character customization. It is divided into four horizontal sections, each with a title, a search bar, and a dropdown menu.

- Body:** Search bar is empty. The dropdown menu shows "Body\_01\_Peach" with a downward arrow.
- Outfit:** Search bar is empty. The dropdown menu shows "Outfit\_05\_Dress\_01" with a downward arrow.
- Hairstyle:** Search bar is empty. The dropdown menu shows "Hairstyle\_02\_Shortlock\_04" with a downward arrow.
- Accessory:** Search bar is empty. The dropdown menu shows "Accessory\_03\_Straw-Hat\_01" with a downward arrow.

***i* TIP**

If the layers list ever appears incorrect or invalid, click **Recreate List** at the bottom to remake the list.

(Note: This will **reset** any selected options)

Once finished, Go to [Next Steps](#).

- [Read Also → Layered Character Templates](#)

## Next Steps

Learn how to **create and render characters** from your **Character Template** during runtime.

[Read More → Use Your Character](#)

# Use Your Character

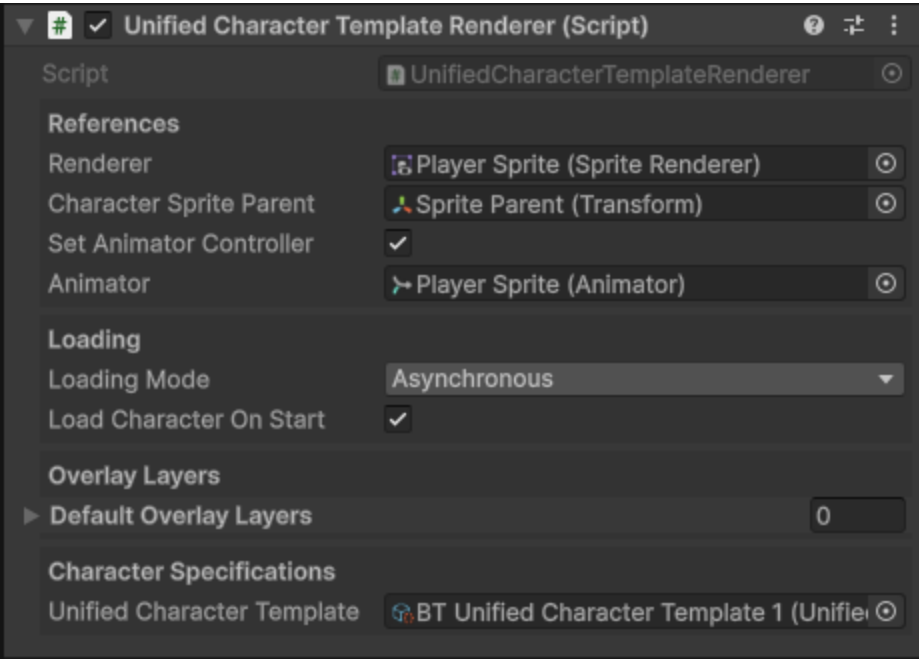
We're now going to create a **character** from the **Character Template** you've created and **render it in-game**.

**Character Renderer components** can create characters and render them for us. They handle all the setup for you meaning all you need to do is **assign your Template**.

## 1 Setup

Create a new GameObject, this will be our character.

If you're using a **Unified Character Type**, add the **Unified Character Template Renderer** component. If you're using a **Layered Character Type**, add the **Layered Character Template Renderer** component.



## Character Renderer Fields

The following fields are required and must be set for all **Character Renderers**.

### References

| Field                   | Description                                                                               |
|-------------------------|-------------------------------------------------------------------------------------------|
| Renderer                | Typically a <b>Sprite Renderer</b> . The <b>Character Shader</b> will be applied to this. |
| Set Animator Controller | Toggles whether to use the <b>Character Controller</b> from the Character Type.           |

| Field           | Description                                                      |
|-----------------|------------------------------------------------------------------|
| <b>Animator</b> | The <code>Animator</code> component to assign the controller to. |

## Loading Settings

| Field                          | Description                                                                                                                                                     |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Loading Mode</b>            | <code>Asynchronous</code> : loads in the background without freezing gameplay.<br><code>Synchronous</code> : loads immediately but may briefly freeze the game. |
| <b>Load Character On Start</b> | Automatically loads the character in <code>Start()</code> . If disabled, <code>GetAndShowCharacter()</code> must be called manually.                            |

## Template Reference

At the bottom of the component, assign your **Character Template** (Unified or Layered). This defines which character is created at runtime.

---

## 2 Test Your Character

Now enter **Play Mode**.

If `Load Character On Start` is enabled, you'll see your character in-game.

### Congrats!

You now have your first character rendered at runtime! Creating more characters is as easy as setting up a new **Character Template**.

**Happy game making!**

# Changelog

All notable updates, improvements, and fixes to the **BlazerTech Character Management System** are listed below.

This log follows the [Keep a Changelog](#) convention and uses [Semantic Versioning](#).

---

## [0.4.0] - 9-19-2026

### Added

- Added [Randomized Layered Character Template](#).
- Added [Character Overlay Layers](#).
- Added **Default Direction** setting to all Animator Handlers.
- Added **Display Name option** to [Character Preview Animation Options](#) in Layered Character Type assets.
- Added [CCMSaveCharacterButton](#) component.
- Added fourth sample scene - Continuous randomized NPC spawning demo.

### Changed

- Character Shader result is now cached instead of being re-run every frame to vastly improve performance.
- Improved many logs, warnings and errors to be more descriptive.
- Renamed **Character Piece Shaders** dictionary to **Layered Character Shaders** in CMS Reference Handler.
- **Layer Options list** in Layer Definition assets are now cleared when **Layer Asset Label** is changed.
- Extracted **Fixed** and **Randomized Layered Character Template editors** into their own classes.
- Renamed **LoadedCharacterHandler** class to **CharacterShaderState** to better convey its purpose.
- Renamed **Layered Character Group Renderer** component to **Layered Character Group Entry Renderer**.
- Renamed **Character Controller** to **Animator Controller** in Character Type asset.
- Converted all input related scripts to **New Input System**.
- The Character Previews **Preview Mode** now defaults to **Animated** instead of **Static**.
- Renamed **CCMAnimationPreviewSwitcherManager** to **CCMAnimationSwitcher**.
- Renamed **LayerOptionUIElement** to **LayerSelectorListElement**.
- Renamed **LayeredCharacterSelectionList** to **CharacterGroupEditor**.
- Renamed **Enable Menu methods** in **Character Creation Menu Manager** to better convey their meaning.

### Fixed

- Fixed Character Creator animation preview buttons staying enabled if the animation assigned to play does not exist.

## Removed

- Removed **Log When Character Material Updated** bool in **CCMCharacterPreviewHandler** component in favor of using the new **CMS logging system**.
  - Removed **Layer Option Preview Settings** section of **Character Creator Settings** in favor of using **Character Preview Sprite**.
- 

## [0.3.0] - 11-13-2025

### Added

- Added third premade Character Creation Menu prefab.
- Added [Project Settings page](#) under **Edit > Project Settings > BlazerTech/Character Management System**.
- Added [binary saving support](#) (Toggleable in project settings page).
- Added [Auto Save Triggers](#) options in project settings page.
- Added [debug log options](#) in project settings page.
- Added [Character Display Name Renderer](#) component for displaying a characters name.
- Added **ICharacter interface** for instances where any type of character can be used.
- Added **assignable Input Actions** to the [Top Down Movement Controller](#) component.
- Added **hold/toggle** options for **sprint/couch** states in the [Top Down Movement Controller](#) component.
- Added **Create Character If Null** bool to **Layered Character Group Renderer** component.
- Added **Active Character Types** list in [Project Settings page](#). (Replaces Addressables requirement for Character Types)

### Changed

- Converted [Top Down Movement Controller](#) to use the **New Input System**.
- Replaced Character Type assets **ISValid** bool with a check if the Character Type asset is contained within the **Valid Character Types** list within the **CMS Reference Handler**.
- Moved Character Shader out of Character Type asset and into a centralized dictionary with an entry for each validated Character Type.
- Migrated Character Type initialization to use pre-set list inside Project Settings page instead of loading through Addressables.

### Fixed

- Fixed major issue causing Character Types to not be functional in build versions.
- Fixed issue when loading a **Character Creation Menu** with **Animation Controls** multiple times.

---

## [0.2.0] - 10-23-2025

### Added

- Added [Top-Down Character Physics Animator Handler component](#). Uses the direction and speed of the game object to set parameters within an Animator Controller.
- Added [Random Layered Character Renderer component](#). Automatically creates a new layered character at runtime using the selected Character Type and random layer options.
- Added randomized NPCs sample scene (Sample 2).
- Added [Built-In Character documentation page](#). Explains what's included and how to use the built-in modular characters.
- Added [Character Animation Setup documentation page](#). Explains the setup process for animations and animator controllers for your characters.

### Changed

- Renamed `PlayerMovementController` to `TopDownMovementController`.
- Renamed all Character Loader components to **Character Renderers** (e.g., `Layered Character Template Renderer`).
- Renamed `CharacterAnimatorHandler` to `TopDownCharacterAnimatorHandler`.
- Exposed animator parameters in Character Animator Handler components.
- Changed Animator Controller `Speed` float to `Is Moving` bool.
- Revised Quick Start Guide.
- Revised entire site layout and structure (The site you're on right now!).

---

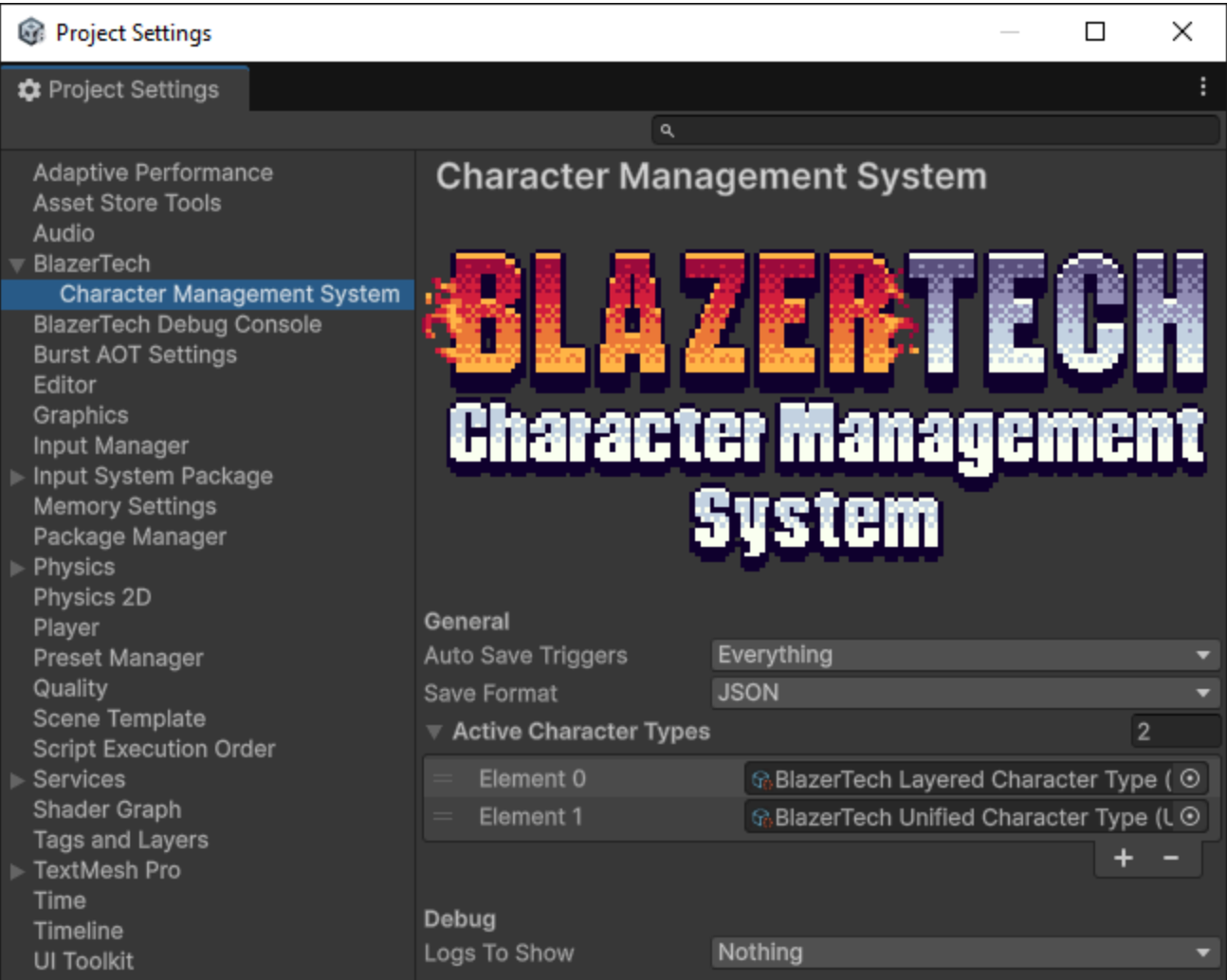
### Legend

-  **Added** — New features or systems.
-  **Changed** — Updates, improvements, or refactors.
-  **Fixed** — Bug or issue resolution.
-  **Deprecated** — Soon-to-be removed features.
-  **Removed** — Old features now removed.

# Project Settings

All global settings for the **BlazerTech Character Management System** are managed through the **Project Settings** window.

Located under **Edit > Project Settings > BlazerTech > Character Management System**.



## Saving Settings

Configure when **Character Groups** are automatically saved to disk.

### Auto Save Triggers

A multi-select dropdown defining the events that trigger automatic saves.

| Option    | Description                                            |
|-----------|--------------------------------------------------------|
| Game Exit | Saves all character groups when the application quits. |

| Option                        | Description                                                                                |
|-------------------------------|--------------------------------------------------------------------------------------------|
| <b>Character Creator Save</b> | Saves all character groups whenever a character is saved from the Character Creation Menu. |
| <b>Scene Change</b>           | Saves all character groups when the active scene changes.                                  |

## Save Format

Select the format used when **Character Groups** are saved to disk.

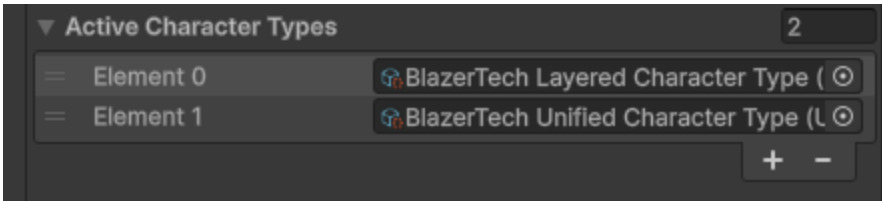
| Format                | Description                                                                 |
|-----------------------|-----------------------------------------------------------------------------|
| <b>JSON (Default)</b> | Human-readable format (.json). Easy to open and manually edit.              |
| <b>Binary</b>         | Compact binary format (.bin). Almost impossible to read or modify manually. |

### NOTE

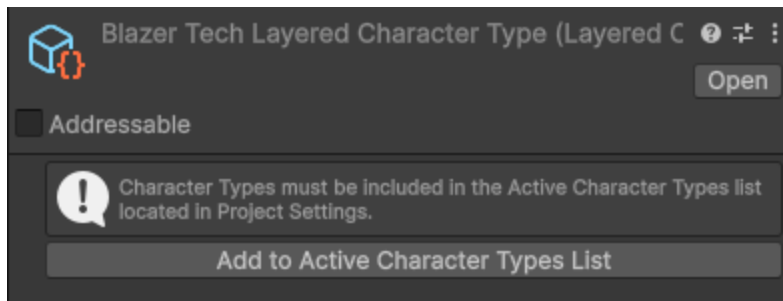
Binary saves are more secure against tampering, while JSON saves are ideal during development and debugging.

## Active Character Types

The **Active Character Types** list contains all **Character Type assets** that are enabled and ready to be used at runtime.



When a new Character Type is created, You are prompted to add it to the **Active Character Types list** before proceeding.



### ⊗ IMPORTANT

If a Character Type asset is not contained inside this list it will NOT be functional.

## Debug Options

These settings control what types of logs are shown to help debugging issues with the **BlazerTech Character Management System**.

### Logs to show

A multi-select dropdown which decides what kind of logs are shown.

| Log Type                          | Description                                                   |
|-----------------------------------|---------------------------------------------------------------|
| Character Group Save Trigger Logs | Logs when character groups are saved.                         |
| New Character Created Logs        | Logs when new characters of any type are created.             |
| Character Recreated From DTO Logs | Logs when a character is reconstructed from saved data (DTO). |
| Loaded Character Updated Logs     | Logs when an already loaded character is modified.            |
| Character Creation Menu Logs      | Logs events within the Character Creation Menu.               |

### i TIP

These logs are meant for debugging purposes and should usually be disabled in release builds.

# Character Types

The **Character Management System** supports two different character structures: **Unified Characters** and **Layered Characters**.

Once setup, both can be used the same way but differ in how the character is rendered and how it can be customized.

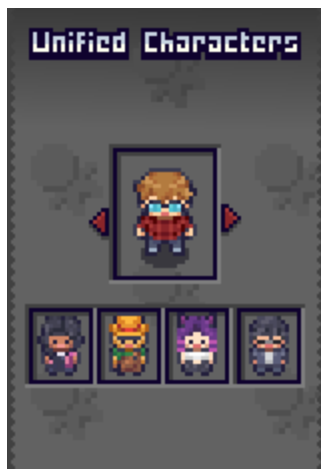
Read below to choose the correct type for your project.

## Choosing a Character Type

| Feature               | Unified Character               | Layered Character                       |
|-----------------------|---------------------------------|-----------------------------------------|
| Spritesheets          | One complete spritesheet        | Multiple layered spritesheets           |
| Runtime Customization | ✗ No                            | ✓ Yes                                   |
| Setup Complexity      | Simple                          | Moderate                                |
| Best For              | Fixed characters or simple NPCs | Customizable characters or dynamic NPCs |

## Unified Characters

A **Unified Character** uses a single spritesheet that contains the fully assembled character.



## Best Used For

- Characters with fixed appearances
- Story characters that never change
- Projects that do not need character customization

Unified Characters are the **simplest to create and manage**, but they do **not support any runtime customization** since the character is fully assembled beforehand.

[Read More → Unified Character Type](#)

---

## Layered Characters

A **Layered Character** is split into multiple spritesheets. Each spritesheet is a layer of the character.



At runtime the system stacks each layer together to create the final character.

For example a character may contain the layers:

- **Body**
- **Outfit**
- **Hairstyle**
- **Accessory**

When the character is rendered:

1. The body renders first
2. The outfit renders on top of that
3. the hairstyle renders above that
4. accessory layer renders last

The **layer order can be configured inside the editor**.

## Requirements

All layer spritesheets must:

- Be the **exact same size**
- Use the **same layout**

- Align to the **same animation frames**

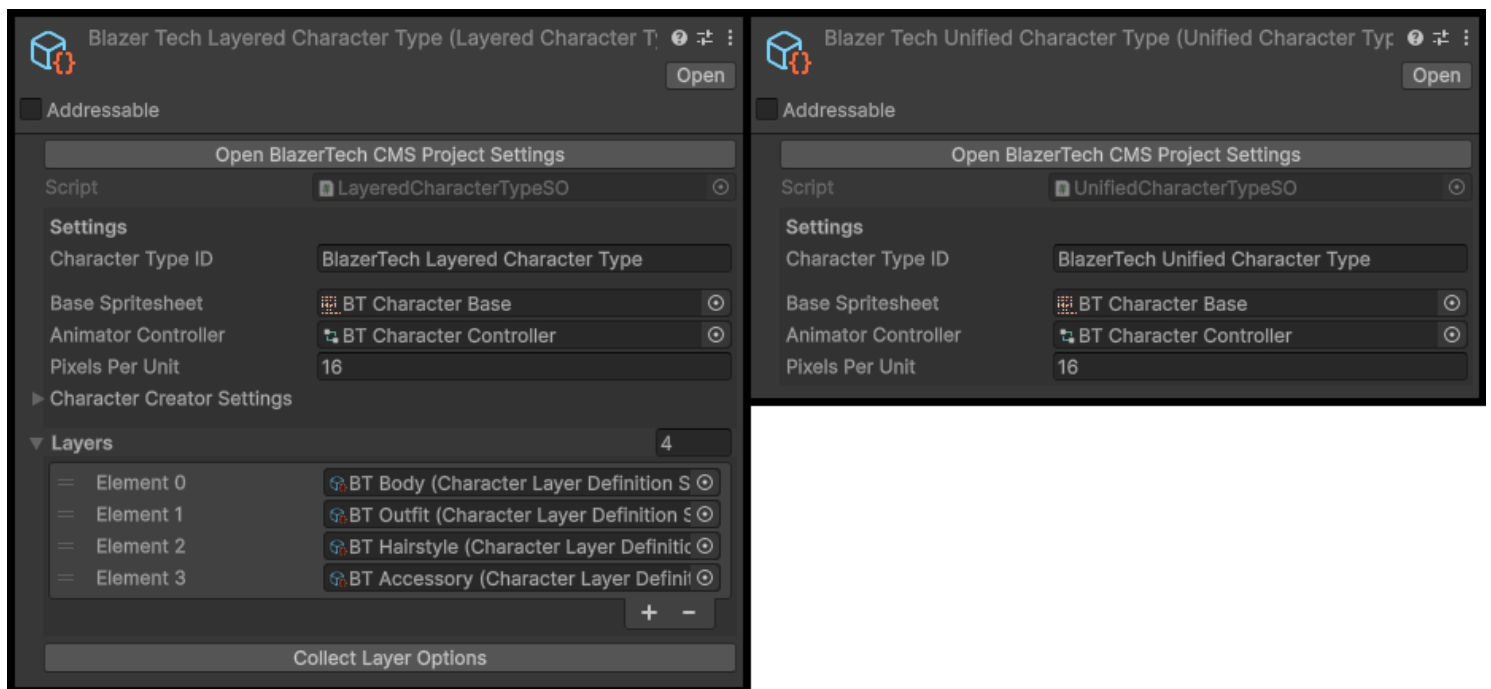
## Best Used For

- Customizable player characters
- Randomly generated NPCs

[Read More → Layered Character Type](#)

## The Character Type Asset

Character types are the heart of the **Character Management System**, they are implemented using [Scriptable Objects](#).



A Character Type asset stores the core configuration used by all characters of that type.

### This includes:

- Base Spritesheet all characters use
- Animator Controller
- Layer Definitions (For Layered Character Types)

#### **(i) NOTE**

A **Character Type asset** must exist before characters can be created.

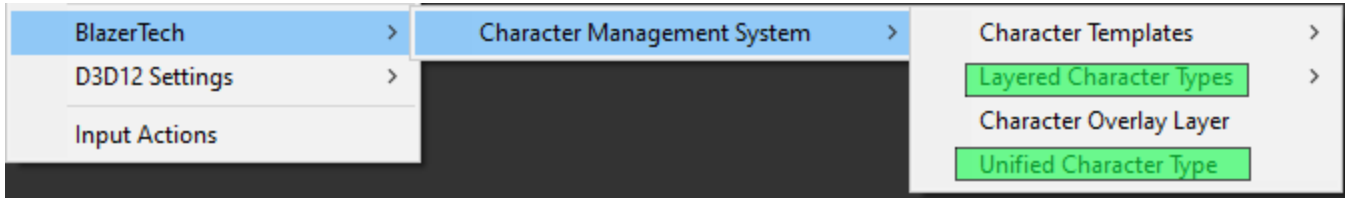
## Create a Character Type

A new Character Type asset can be created by **right clicking the Project window** and navigating to **Create > BlazerTech > Character Management System > (Choose Character Type)**

You will then choose which character type to create:

- **Unified Character Type**
- **Layered Character Type**

A **Character Type** can only create characters of the same type.



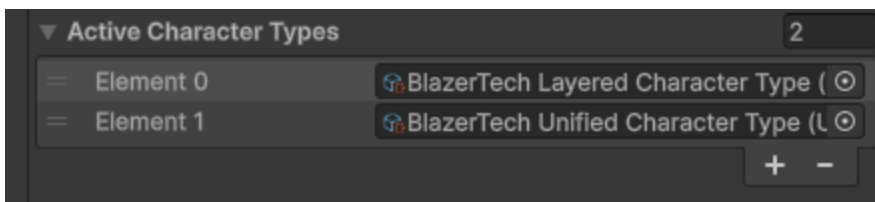
### *i* NOTE

You will need a new **Character Type** for each set of characters that have a different spritesheet layout (EG: different animations, number of frames, etc.)

## Active Character Types List

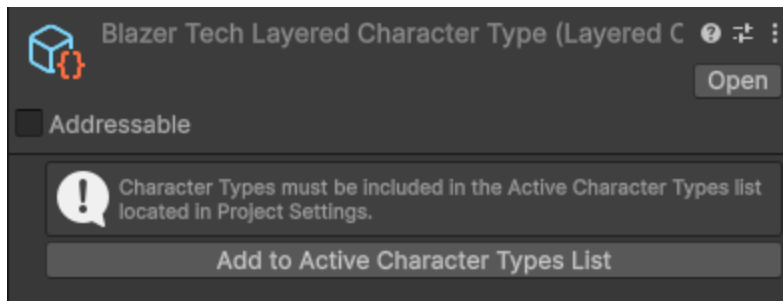
The **Active Character Types list** determines which Character Types are initialized when the game starts.

It can be found under: **Project Settings > BlazerTech > Character Management System**.



Only Character Types in this list will be useable at runtime.

When creating a new **Character Type asset** you will be prompted to add it to the **Active Character Types list** automatically.

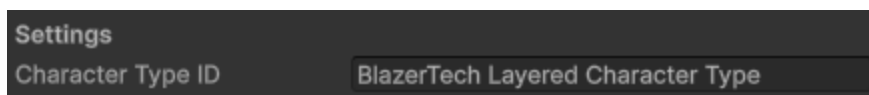


## Character Type Setup

This section explains how to configure the fields inside the **Character Type asset**.  
Setup specific to only **Layered** or **Unified Character Types** will be found in their own pages.

## Character Type ID

A **unique identifier** for the Character Type

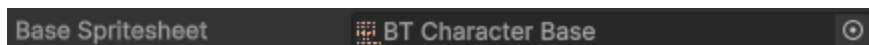


This ID **must be unique** among all Character Types listed in the **Active Character Types list**.

If two Character Types share the same ID, initialization will fail at runtime.

## Base Spritesheet

The spritesheet all characters of the same type will use.



The **Base Spritesheet** contains all animation frames that characters of this type will use.  
This spritesheet acts as a **template for frame positions, sizes and animations**.

All characters using the same **Character Type** must follow this layout.

## Why All Characters Use the Same Base Spritesheet

The [Character Shader](#) uses the animation frames from the **Base Spritesheet** to determine **which frame should be displayed**.

instead of changing the animation itself, the shader replaces the visual sprites with the character's own spritesheet.

This allows:

- One **Animator Controller** to animate every character
- New characters to be added without creating new animations

## Base Spritesheet Settings

The **Base Spritesheet** must use the correct import settings to work properly.

| Setting            | Value              | Reason                                          |
|--------------------|--------------------|-------------------------------------------------|
| <b>Sprite</b>      | Multiple           | Allows the spritesheet to be sliced into frames |
| <b>Compression</b> | None (Recommended) | Prevents artifacts in pixel art                 |
| <b>Filter Mode</b> | Point (No Filter)  | Keeps pixel art sharp                           |

## Spritesheet Layout Requirements

All character spritesheets for this Character Type must:

- Use the **same frame sizes**
- Use the **same frame positions**
- Contain the **same animation frames**

This guarantees that animations line up correctly across all characters.

### Example

If the **Base Spritesheet** contains the following animations:

- Idle (4 frames)
- Walk (6 frames)
- Run (6 frames)

Then **every character spritesheet must also contain those same frames in the same positions.**

## Animator Controller (Optional)

All character share the same spritesheet layout, meaning **one Animator Controller can animate every character of a single type.**



This removes the need to create separate **Animator Controllers** or **Animator Override Controllers** for each character.

### NOTE

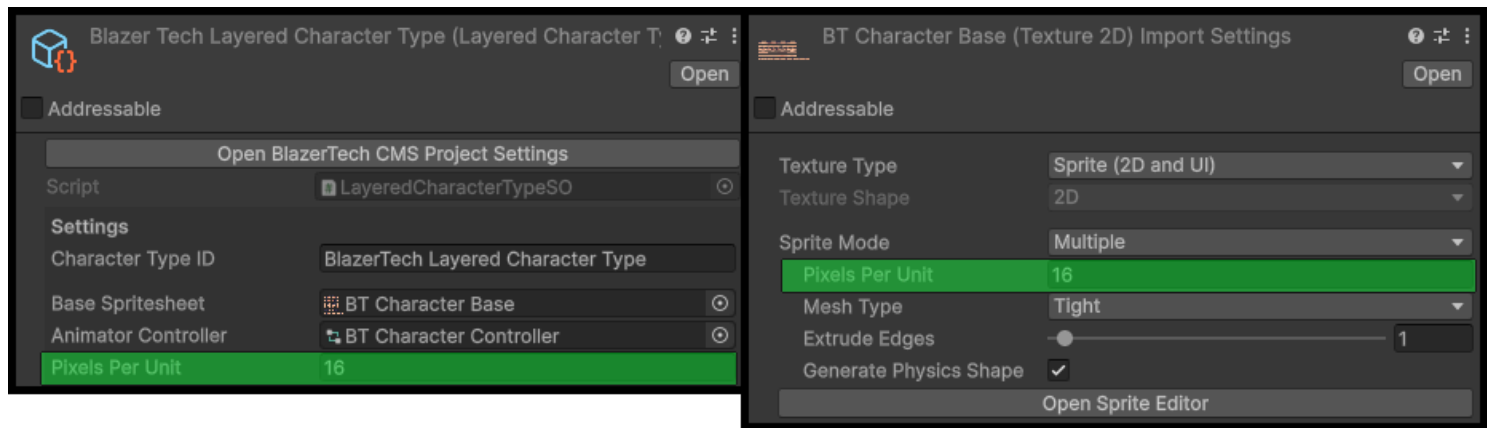
An **Animator Controller** is not required if your project uses a different system for animating your characters. The only requirement is only sprites from the **Base Spritesheet** are used.

Confused? [Read more about the Character Shader here.](#)

See [Character Animation Setup](#) for instructions on configuring your **Animator Controller**.

## Pixels Per Unit

The **render scale** of the character.



This value should match the **Pixels Per Unit** setting used in the **Base Spritesheet**.

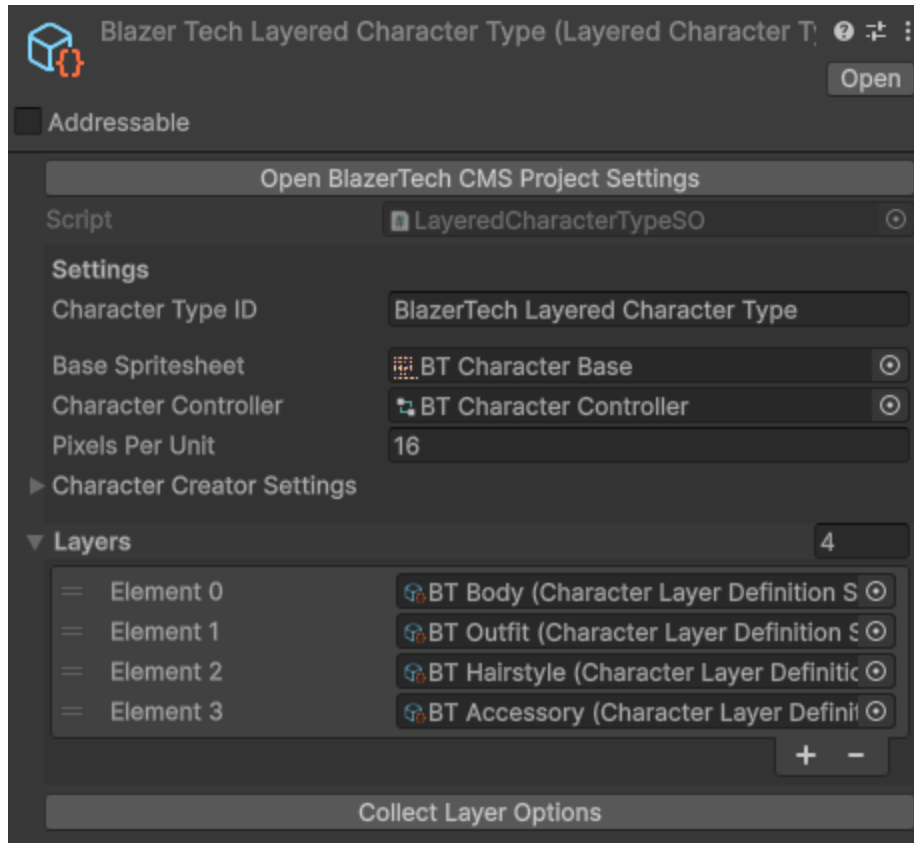
## Related

- [Layered Character Type](#)
- [Unified Character Type](#)
- [Character Templates](#)
- [Character Renderer Components](#)

# Layered Character Type

A **Layered Character Type** defines how a **Layered Character** is structured.

It contains the core data shared by all characters of that type, such as the **Base Spritesheet**, **Animator Controller**, and the list of **Character Layers** that make up the character.



## Layered Characters

A **Layered Character** is built from **multiple spritesheet layers** stacked on top of each other.

Each layer represents a different visual component of the character.

All layers must follow the **same spritesheet layout**:

- Same **spritesheet size**
- Same **frame sizes**
- Same **frame positioning**

Because every layer shares the same spritesheet layout, all layers align perfectly when stacked together.

**Example Layered Character:**

1. Body

2. Outfit
3. Hairstyle
4. Accessory

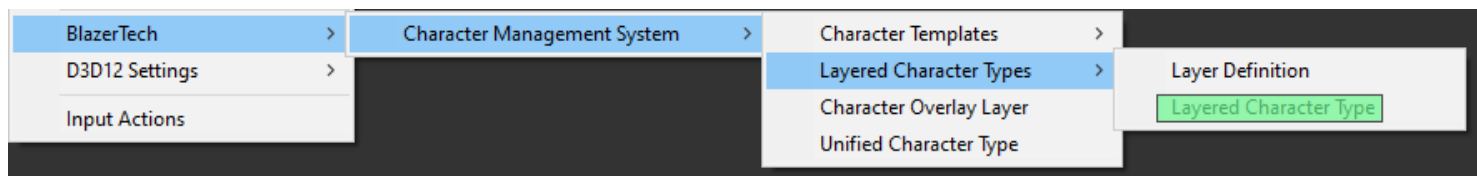
The final character is rendered by combining all layers in order.



## Create Layered Character Type Asset

To create a new Layered Character Type:

1. Right click the **Project window**
2. Navigate to: **Create > BlazerTech > Character Management System > Layered Character Type**



## Character Type Setup

A **Layered Character Type** uses the same core properties shared by all **Character Types**.

| Property                            | Type                      | Description                                             |
|-------------------------------------|---------------------------|---------------------------------------------------------|
| <a href="#">Character Type ID</a>   | String                    | A <b>unique</b> identifier                              |
| <a href="#">Base Spritesheet</a>    | Sprite                    | The <b>character spritesheet</b> used by all characters |
| <a href="#">Animator Controller</a> | RuntimeAnimatorController | Single <b>Animator Controller</b> for all characters    |
| <a href="#">Pixels Per Unit</a>     | int                       | PPU of your Base Spritesheet                            |

Read [Character Type Setup](#) for a step-by-setup guide on how to setup each field.

# Character Layers

The **Layered Character Type** also defines the layers used to build the character.

Each entry in the **Layers** list references a **Character Layer Definition** asset.

A **Character Layer Definition** describes how a single layer behaves, including:

- **Layer Name**
- Whether the **layer can be blank**
- The available **Layer Options** (Spritesheets)

Example layers: **Body, Outfit, Hairstyle, Accessory**

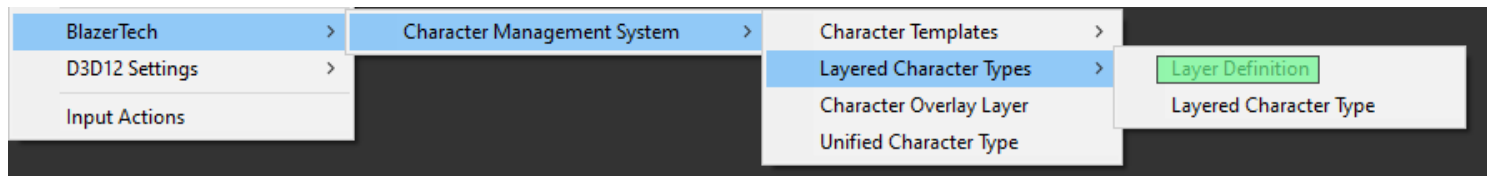
Each layer corresponds to one spritesheet that is combined during rendering.

---

## Create Character Layer Definitions

To create a Character Layer Definition:

1. **Right click** the **Project window**
2. navigate to: **Create > BlazerTech > Character Management System > Layered Character Types > Layer Definition**



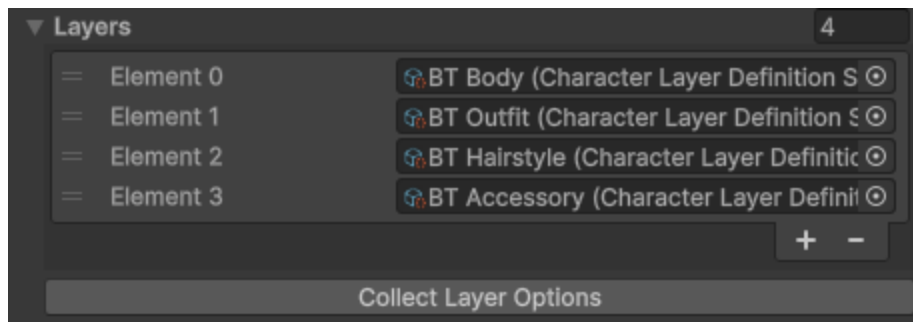
Create a **Layer Definition** for each layer you want your characters to have.

---

## Assign Layers

After creating the **Layer Definitions**:

1. Open your **Layered Character Type**
2. Add each **Layer Definition** to the **Layers** list



The **order of the layers in this list determines the render order**.

Layers are rendered **from top to bottom in the list**, meaning:

- Top of List = Rendered First (Behind)
- Bottom of List = Rendered Last (In Front)

Read [Character Layer Definitions](#) to learn how to setup each **Character Layer Definition**.

---

## Creating Layered Characters

Once your **Layered Character Type** and **Character Layers** are configured, you can begin creating characters.

**Layered Characters** can be created in several ways:

- [Layered Character Templates](#) (Simplest workflow)
  - [Random Character Generation](#)
  - [Runtime creation through C#](#)
- 

## Runtime Creation through C#

### Create Default Layered Character

This script creates a new **Layered Character** using the **default Layer Options** for each Layer.

```
using BlazerTech.CharacterManagement.Characters;
using UnityEngine;

public class CreateDefaultLayeredCharacter : MonoBehaviour
{
    [SerializeField] LayeredCharacterTypeSO characterType;

    LayeredCharacter character;
    private void Start()
    {
        character = new LayeredCharacter("layered-character", characterType,
        "Layered Character");
    }
}
```

---

### Create Randomized Layered Character

This script creates a new **Layered Character** and grabs a **completely random Layer Option** for each Layer.

```
using BlazerTech.CharacterManagement.Characters;
using UnityEngine;

public class CreateRandomizedLayeredCharacter : MonoBehaviour
{
    [SerializeField] LayeredCharacterTypeSO characterType;

    LayeredCharacter character;
    private void Start()
    {
        character = new LayeredCharacter("layered-character", characterType, "Layered
Character", LayeredCharacterCreationMode.RandomizedLayerOptions);
    }
}
```

---

## Create Layered Character With Specific Options

This script creates a new **Layered Character** by grabbing the **second layer option** for every layer if available, if not, the **first layer option** is grabbed instead.

```
using BlazerTech.CharacterManagement.Characters;
using System.Collections.Generic;
using UnityEngine;

public class CreateLayeredCharacter : MonoBehaviour
{
    [SerializeField] LayeredCharacterTypeSO characterType;

    LayeredCharacter character;
    private void Start()
    {
        List<CharacterLayerOption> layerOptions = new();

        foreach (var layer in characterType.Layers)
        {
            var layerOption = layer.GetLayerOptionFromIndex(1);
            layerOption ??= layer.GetLayerOptionFromIndex(0);

            layerOptions.Add(layerOption);
        }
    }
}
```

```
        character = new("layered-character", characterType, layerOptions,  
"Layered Character");  
    }  
}
```

---

## Related

- [Character Types](#)
- [Character Layers](#)

# Character Layers

A **Character Layer** represents a single visual component of a character such as a **body**, **outfit**, or **hairstyle**.

A **Layered Character** is created by stacking multiple layers together.

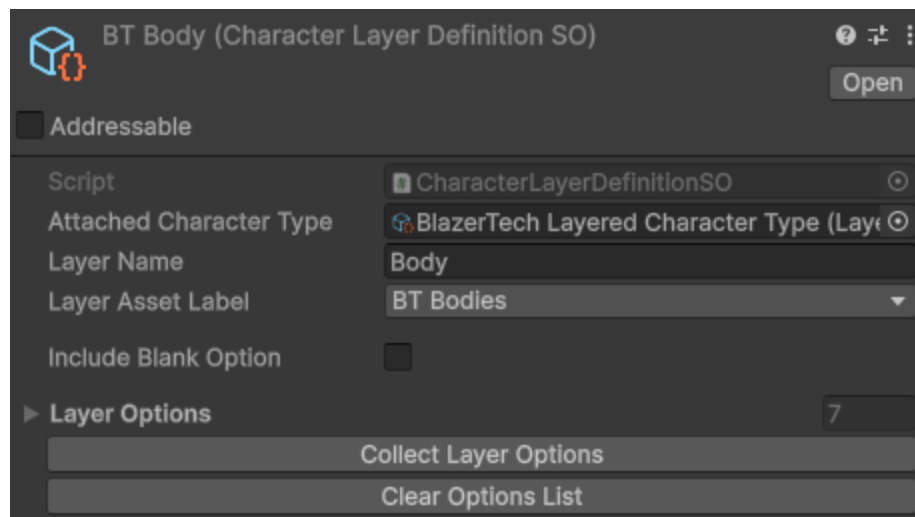
Each layer uses a **spritesheet** that contains all animation frames for that part of the character.

There are three main concepts involved when working with layers:

| Concept                           | Description                                                     |
|-----------------------------------|-----------------------------------------------------------------|
| <b>Character Layer</b>            | A visual part of a character (Body, Outfit, Hair, etc.)         |
| <b>Character Layer Definition</b> | An asset that defines all spritesheets available for that layer |
| <b>Character Layer Option</b>     | A single spritesheet that can be used for that layer            |

## Character Layer Definitions

**Character Layer Definitions** are **scriptable objects** that define all available spritesheets that can be used for a specific layer of a **Layered Character**.



Each **Layered Character** is built by combining multiple layers together.

**Example Character Layers:**

- **Body**
- **Outfit**
- **Hairstyle**
- **Accessory**

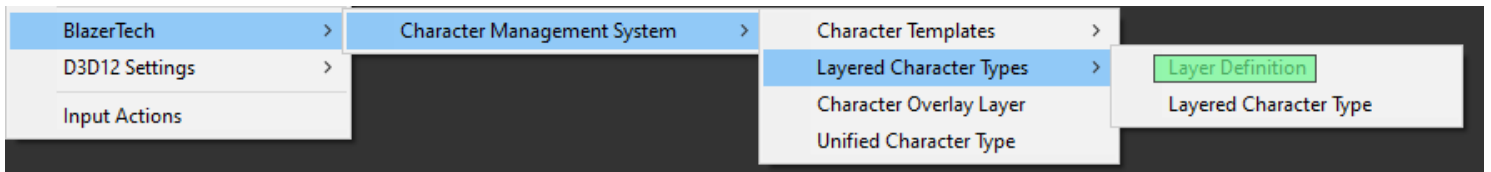
**Character Layer Definitions** are connected directly to a [Layered Character Type asset](#).

---

## Creating a Character Layer Definition

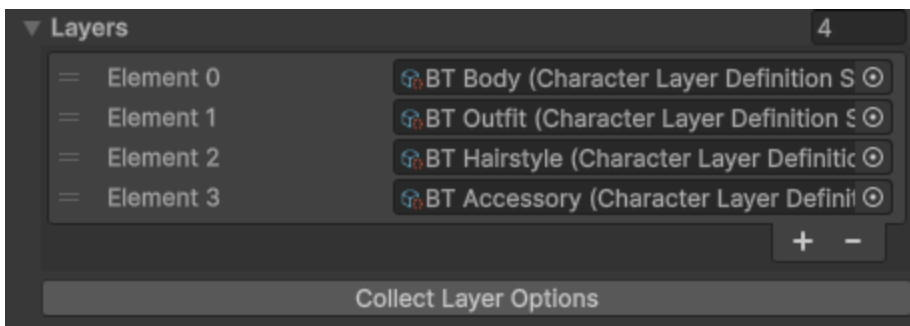
To create a new Layer Definition:

1. **Right click** the **Project window**
2. Navigate to: **Create > BlazerTech > Character Management System > Layered Character Type > Layer Definition**



Create a new **Layer Definition** for every layer you want your characters to have.

Once all **Layer Definitions** have been created, add them to the **Layers list** inside your **Layered Character Type**.



The **order of the layers in this list determines the render order**.

Layers are rendered **from top to bottom in the list**, meaning:

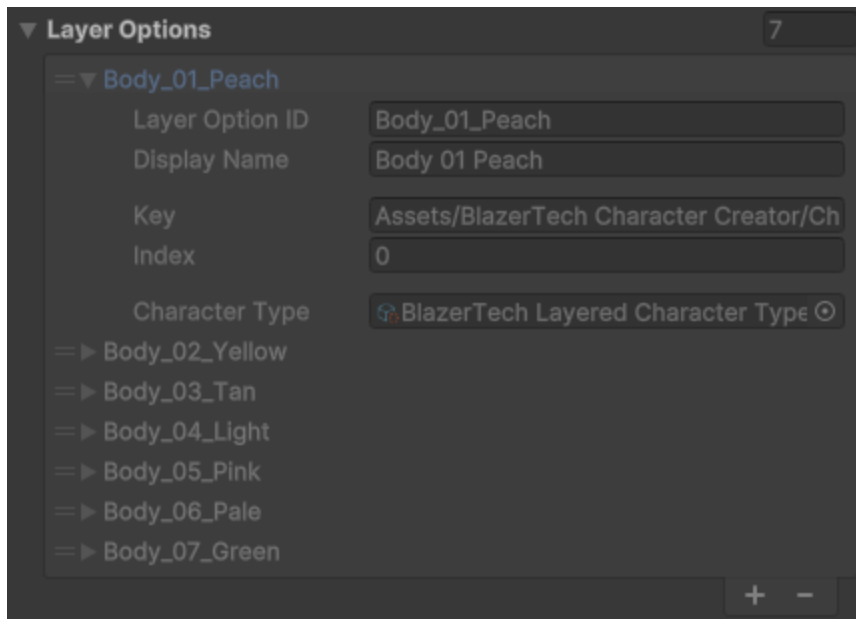
- Top of List = Rendered First (Behind)
- Bottom of List = Rendered Last (In Front)

---

## Character Layer Options

A **Character Layer Option** represents a single spritesheet that can be used for a layer.

Each entry in the **Layer Options list** corresponds to one possible appearance for that layer.



For a **Layered Character** to be created, a **Layer Option** must be chosen from every layer.

### ⊗ CAUTION

All layer spritesheets must be the exact same size as the [Base Spritesheet](#) defined in the **Layered Character Type asset**.

If the dimensions do not match, the spritesheet will be considered **invalid**.

A [Character Layer Option](#) is internally used by the **Character Management System** to reference and load the spritesheet when the character is created.

## Adding Layer Options

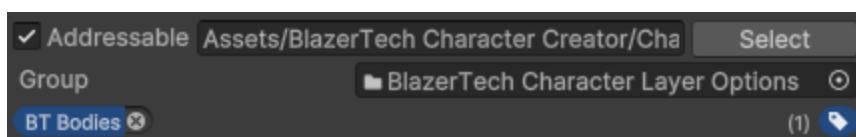
**Layer Options** are **not added manually** to the list.

Instead, they are automatically collected using Unity's Addressables system.

To add a spritesheet as a valid option:

1. Select the spritesheet in the **Project window**
2. In the **Inspector**, enable **Addressable** at the top
3. Assign the correct **Addressables Label**.

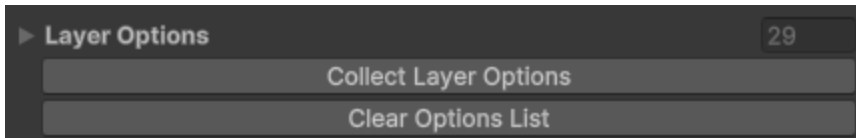
The label must match the [Layer Asset Label](#) configured in the **Character Layer Definition asset**.



### TIP

You can mark an **entire folder** as Addressable and assign a label.  
All assets inside the folder will be automatically marked as Addressable and inherit the label.

After labeling all spritesheets, return to the **Character Layer Definition asset** and press **Collect Layer Options**



### TIP

The **Layered Character Type** asset also contains a **Collect Layer Options** button which collects options for all layers at once.

## Spritesheet Requirements

### IMPORTANT

All layer spritesheets must use the **same frame layout** as the **Base Spritesheet**.

This means:

- Same **texture dimensions**
- Same **frame sizes**
- Same **frame positions**

## Required Settings

- **Sprite Mode = Single**

Layer spritesheets must use **Single** mode because the system does **not use individual sprite frames**.

Instead, the **entire spritesheet texture** is passed to the **Character Shader**, which handles rendering the character.

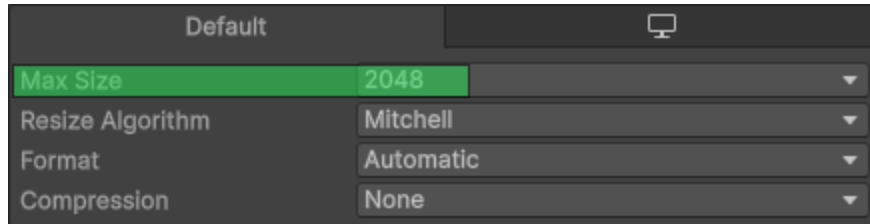
[Read More → The Character Shader](#)

## Recommended Settings

- **Filter Mode = Point (No Filter)** - Keeps image sharp
- **Compression = None** - Generally not needed for pixel art

## Spritesheet Blurry?

If your spritesheet is blurry even after setting the **Filter Mode** to **Point**, check if the **spritesheet size** is greater than the **Max Size** setting. If it is, increase the **Max Size**.

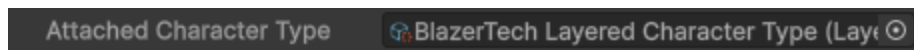


## Fields

Below is a description of every field in the **Character Layer Definition asset**.

## Attached Character Type

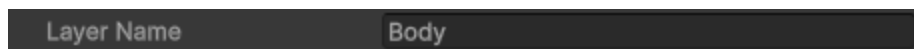
The **Layered Character Type asset** that this layer belongs to.



This is used to ensure the layer only collects spritesheets that match the Base Spritesheet size.

## Layer Name

The display name of the layer.

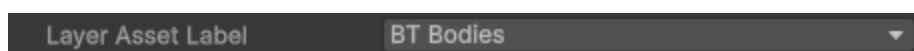


This name is used by the [Character Creator](#) when showing layer names in the UI.

The name **does not need to be unique**.

## Layer Asset Label

The Addressables Label used to collect spritesheets for this layer.



The Character Management System uses Unity Addressables to dynamically load and unload spritesheets.

All spritesheets intended for this layer must:

- Be marked as **Addressable**
- Use the **same label** configured in this field.

When the **Collect Layer Options** button is pressed, all spritesheets using this label are added to the **Layer Options list**.

Read [Adding Layer Options](#) for more info.

---

## Include Blank Option

If enabled, a **blank option** will be automatically added to the **Layer Options list**.

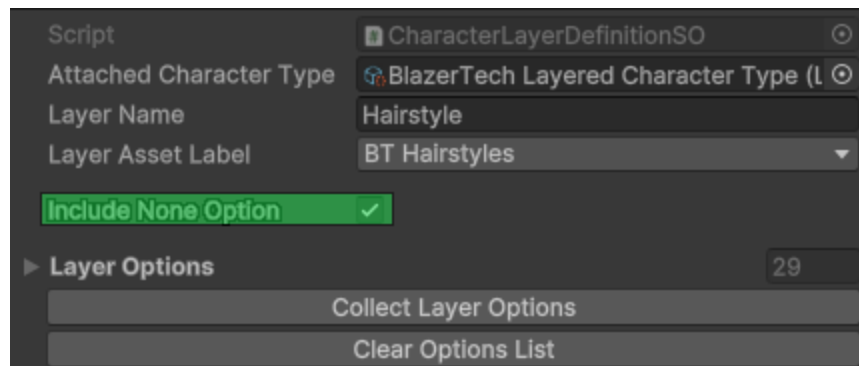


This allows characters to be created **without using that layer**.

**Example use cases:**

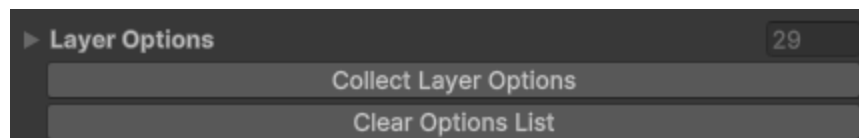
- Characters without accessories
- Characters without facial hair

If this option is selected at runtime, an **empty sprite** will be used instead of a spritesheet.



---

## Inspector Buttons



## Collect Layer Options

Searches for all spritesheets that:

- Are marked as **Addressable**
- Match the **Layer Asset Label**
- Have the same **texture dimensions** as the **Base Spritesheet**

If all conditions are met, the spritesheet is added to the Layer Options list.

---

## Clear Options List

Removes all entries from the **Layer Options list**.

### **TIP**

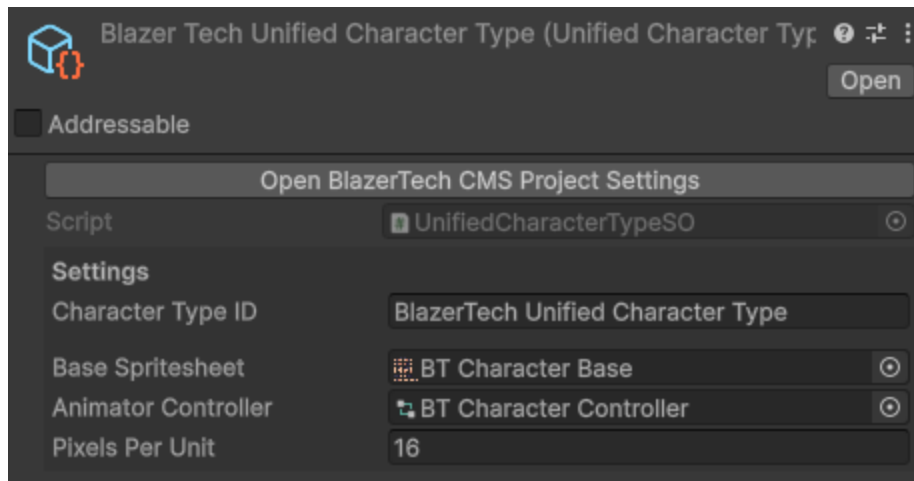
This action can be undone using **Ctrl + Z** (Windows) or **Command + Z** (Mac).

# Unified Character Type

A **Unified Character Type** defines how a **Unified Character** is structured.

It contains the core data shared across all characters of that type, such as the **Base Spritesheet** and **Animator Controller**.

A **Unified Character** uses a **single spritesheet** that contains the complete character and all its animations.



---

## Unified Characters

A **Unified Character** has the entire character contained in a single spritesheet.

This means unlike **Layered Characters**, the character is not built from multiple layers, instead, each character is self contained in its own spritesheet.

For example, if your game contains a set of characters, each character variation would have its own spritesheet. Example list:

- Knight
- Knight (Armored)
- Knight (Winter Outfit)

Each character spritesheet must follow the same layout so that all animations align correctly.

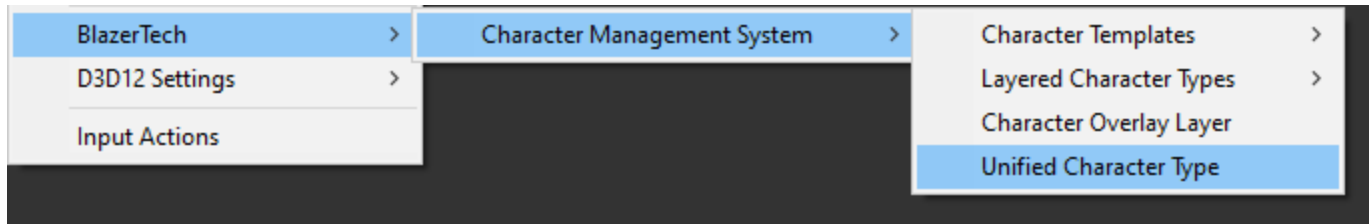
**All Unified Character Spritesheets must share:**

- The same **Spritesheet size**
- The same **frame sizes**
- The same **frame positioning**

# Create a Unified Character Type Asset

To create a new Unified Character Type:

1. Right click the **Project window**
2. Navigate to: **Create > BlazerTech > Character Management System > Unified Character Type**



## Character Type Setup

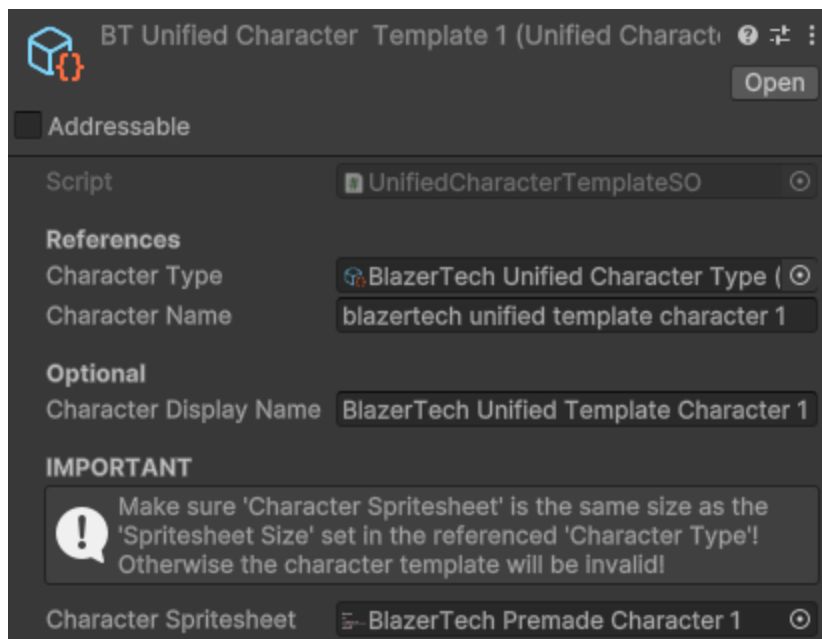
A **Unified Character Type** uses the same core properties shared by all **Character Types**.

| Property                            | Type                      | Description                                             |
|-------------------------------------|---------------------------|---------------------------------------------------------|
| <a href="#">Character Type ID</a>   | String                    | A <b>unique</b> identifier                              |
| <a href="#">Base Spritesheet</a>    | Sprite                    | The <b>character spritesheet</b> used by all characters |
| <a href="#">Animator Controller</a> | RuntimeAnimatorController | Single <b>Animator Controller</b> for all characters    |
| <a href="#">Pixels Per Unit</a>     | int                       | PPU of your Base Spritesheet                            |

Read [Character Type Setup](#) for a step-by-setup guide on how to setup each field.

## Creating Unified Characters

A **Unified Character** can be created through a **Unified Character Template**.



The template will store a reference to the spritesheet for that character.

The **Unified Character Template Renderer** component can create and render a **Unified Character** from a template.

[Read More → Unified Character Templates](#)

[Read More → Unified Character Template Renderer component](#)

---

## Related

- [Character Types](#)

# Character Animation Setup

This page goes over how to **create and properly configure an Animator Controller** to be used by a [Character Type](#) asset.

---

## Overview

Every **Character Type asset** can include an **Animator Controller**.

All animations should use sprites from the **Base Spritesheet** assigned in the **Character Type asset**.

All characters using the same **Character Type** can use a single **Animator Controller**. No need to create a separate Animator/Override Controller for every new character.

Confused how it's possible to only use one **Animator Controller** for every character? Read about [The Character Shader](#).

The setup process is the same for both **Unified** and **Layered Character Types**.

### ⊗ IMPORTANT

All animations **must only use sprites from the Base Spritesheet**.

If sprites from other sheets are used, they **will not render correctly** when used with the [Character Shader](#).

If you already have your own movement and animator handling scripts, you can setup the **Animator Controller** however you need for you project. That means you can create your own parameters, animation states, blend trees, etc.

---

## Integration with Built-In Scripts

The **BlazerTech Character Management System** includes pre-built movement and animator handling scripts. The included scripts contain support for the following movement animations:

- **Idle**
- **Walk**
- **Sprint**
- **Crouch-Idle**
- **Crouch-Moving**

Additional animations can be played at runtime using:

```
PlayAnimation("AnimationName");
```

on any **Character Animator Handler** component.

## 1 Create an Animator Controller

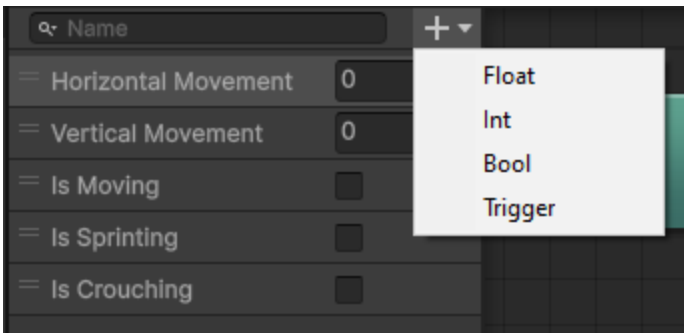
1. In the **Project window**, right-click and select  
`Create > Animation > Animator Controller`.
2. **Name asset** (Recommended to name it after the Character Type it's used for)
3. Double-click to open it in the **Animator window**.

You should now see an empty Animator Controller.

## 2 Add Parameters

The built-in **Character Animator Handler** components do **not** call animations directly. Instead, they modify **parameters** set in the **Animator Controller**, which control transitions between animation states.

The **Parameters** section is located in the top left of the **Animator window**.



## Required Parameters

| Parameter Name             | Type  | Usage                                                              |
|----------------------------|-------|--------------------------------------------------------------------|
| <b>Horiztonal Movement</b> | Float | Ranges from -1 to 1. Represents left/right movement on the X-axis. |
| <b>Vertical Movement</b>   | Float | Ranges from -1 to 1. Represents up/down movement on the Y-axis.    |
| <b>Is Moving</b>           | Bool  | True when the character is moving; otherwise false.                |

## Optional Parameters

Optional parameters will only be used if toggled in the movement and animator handler components.

| Parameter Name      | Type | Usage                                                  |
|---------------------|------|--------------------------------------------------------|
| <b>Is Sprinting</b> | Bool | True when the character is sprinting; otherwise false. |
| <b>Is Crouching</b> | Bool | True when the character is crouching; otherwise false. |

### ⊗ **IMPORTANT**

These parameter names must match **exactly** with what's set in the built-in Animator Handler components.

## 3 Add Animation States

Because the built-in animator handler components only modify parameters, you can design your animation state layout freely.

A common approach is to use Blend Trees for animations that play in four directions (Up, Down, Left, Right).

In this example we'll be setting up an **Idle** and **Walk** blend tree.

## Create Animation Clips

**Right Click** the Project window and navigate to **Create > Animation > Animation Clip**.

We'll create 4 animation clips. 1 for each direction.

### Idle

- Idle Down
- Idle Up
- Idle Left
- Idle Right

### Walk

- Walk Down
- Walk up
- Walk Left
- Walk Right

You can leave these clips empty for now, we'll add frames later.

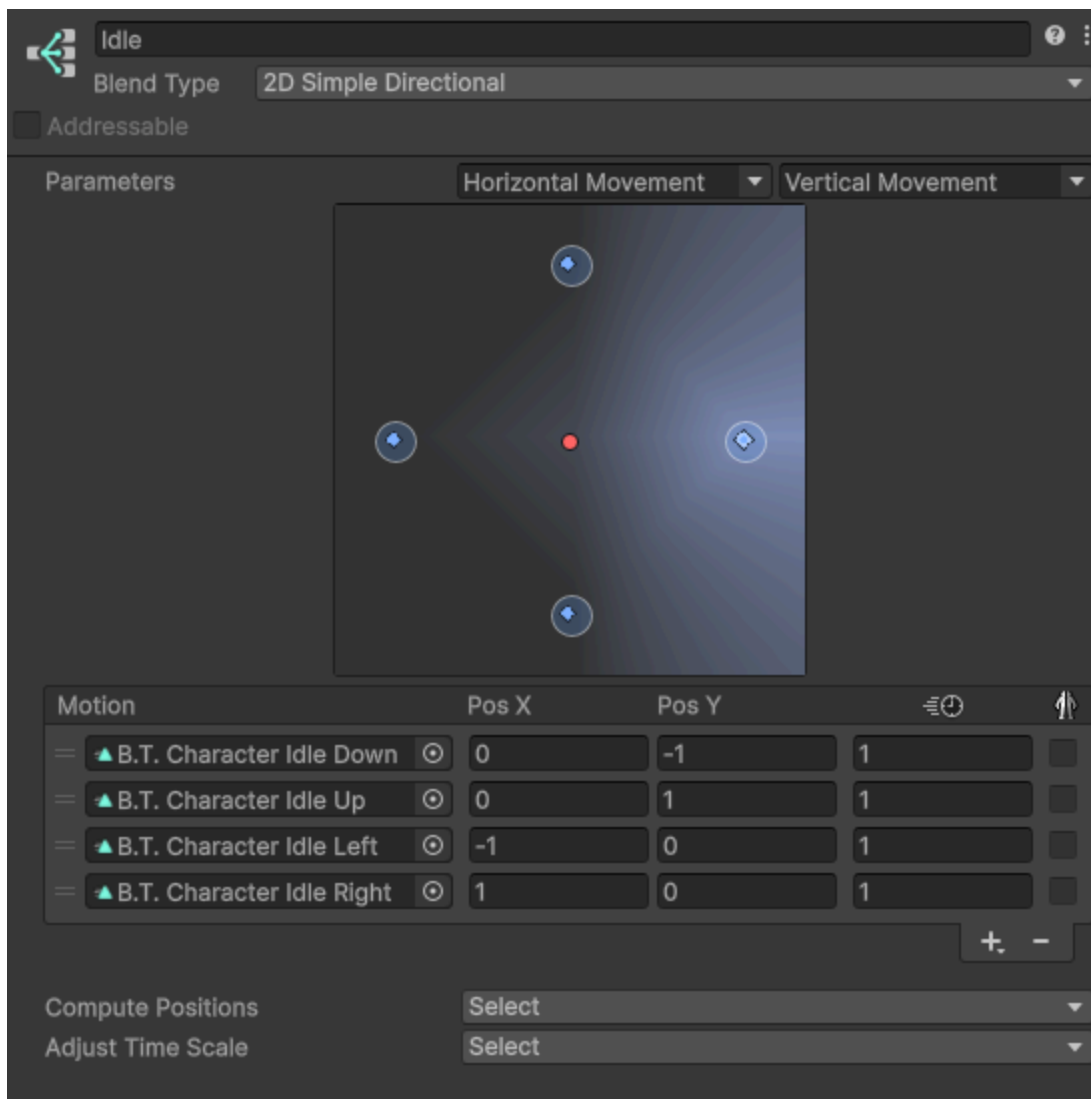
# Create and Setup Blend Trees

Repeat these steps for each 4 directional animation set you want to setup.

1. **Right-click** Animator window > **Create State** > **From New Blend Tree**.
2. Name the blend tree after the animation name (eg: Idle, Walk).
3. Double click the blend tree to open it.
4. In the **Blend Tree**, set **Blend Type** to **2D Simple Directional**.
5. Assign **Blend Parameters** to:
  - o X: **Horizontal Movement**
  - o Y: **Vertical Movement**
6. Add motions for each direction (Up, Down, Left, Right).
7. Assign animations to each motion from the animation clips we created earlier.
8. Set X and Y position to correspond to all 4 directions for each animation.

## Coordinate Cheat Sheet

| X Coordinate | Y Coordinate | Direction |
|--------------|--------------|-----------|
| 0            | -1           | Down      |
| 0            | 1            | Up        |
| -1           | 0            | Left      |
| 1            | 0            | Right     |



## 4 Setup State Transitions

Lets tell the **Animator Controller** how to transition between states.

The entry state should be the **Idle blend tree**.

Right click the Idle blend tree and select **Set As Layer Default State**.

Create a transition from **Idle** to **Walk**:

1. Right click on the **Idle Blend tree**.
2. Select **Make Transition**.
3. Hover over and click on the **Walk** blend tree.

Now lets add a condition to the transition.

1. Right click the new transition line.
2. Add a new condition.
3. Assign the **Is Moving** parameter and the value to true.
4. Disable **Has Exit Time** to avoid a delay when transitioning.

Now when the character starts moving they will automatically go from the **Idle** blend tree to the **Walk** blend tree

Now Make another transition from **Walk** to **Idle**.

Once again add a new condition, this time set **Is Moving** to false.

This way when the character stops moving they'll go back to playing The idle animations.

---

## 5 Setup Animation Clips

Now it's time to setup the **animation clips** we created earlier.

Add an **Animator** and **Sprite Renderer** component to any game object.

- Make sure the game object is selected.
- Go to the **Animation** Tab.
- Select the animation you want to edit.
- Bulk select all sprite frames.
- Drag and drop them into the **Animation timeline**.
- Set Samples. This is essentially the frame rate of your animation. (Recommended: 12)

Repeat for each animation clip.

---

## 6 Add Additional Animations (Optional)

You can expand your Animator with as many custom animations as needed.

### Single-Direction Animations

For animation that only play in one direction:

- Create a new state.
- Assign the clip directly.

### Multi-Directional Animation

For Directional animations:

- Create another **blend tree** using the same setup as before.

When played, the blend tree automatically chooses the correct direction.

If an animation is not looped, add a transition back to Idle or the Exit node.

Animations can be triggered manually via:

```
animator.Play("AnimationName");
```

or

```
animatorHandler.PlayAnimation("AnimationName");
```

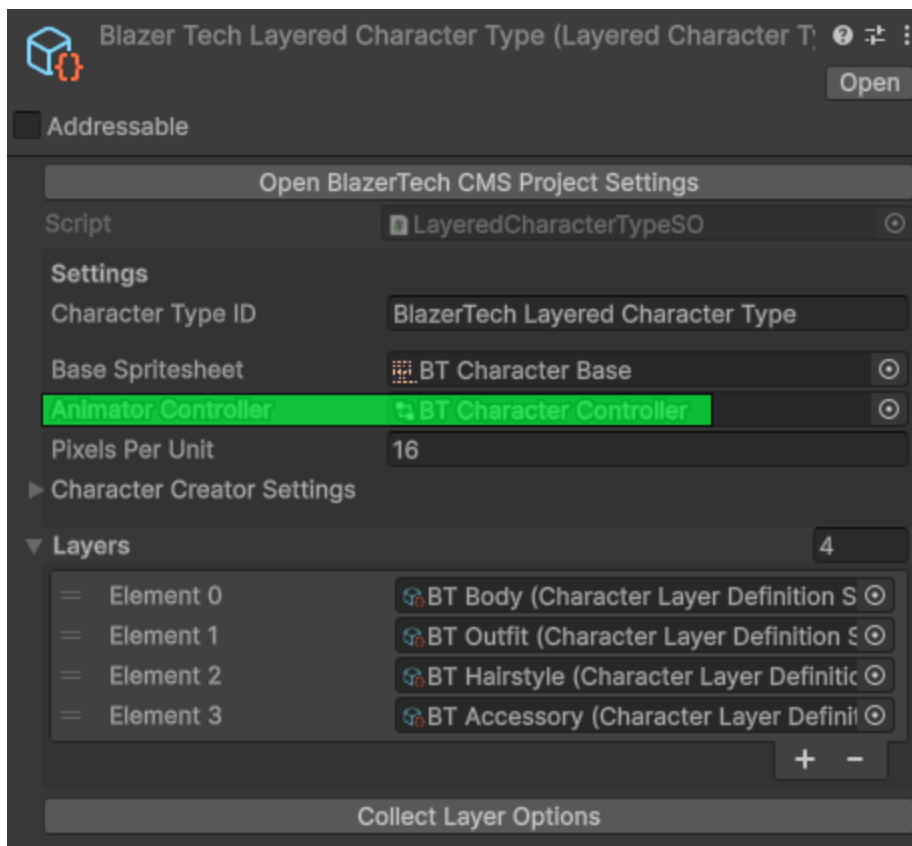
(using the included Animator Handler components).

## 7 Assign to a Character Type

Once your Animator Controller is ready:

1. Select your **Character Type** asset.
2. In the **Inspector**, find the **Animator Controller** field.
3. Drag your new **Animator Controller** into this field.

It should look like this:



When used with a [Character Renderer component](#), the assigned **Animator Controller** will automatically be applied at runtime.

# Character Renderer Components

Character Renderer components will load and render characters in-game.

**All renderer components handle:**

1. Creating/loading a character
2. Applying the **Character Shader**
3. Assigning the **Animator Controller** if applicable
4. Managing **Overlay Layers**

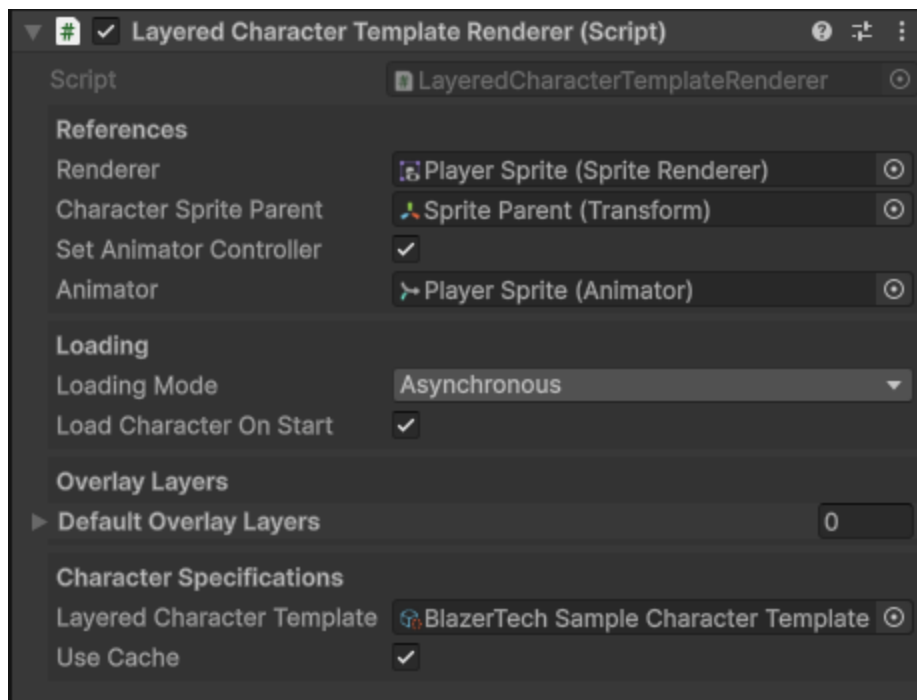
## Components

| Renderer Component                                     | Used For                                                                    |
|--------------------------------------------------------|-----------------------------------------------------------------------------|
| <a href="#">Layered Character Template Renderer</a>    | Create and render a <b>Layered Character</b> from a template.               |
| <a href="#">Unified Character Template Renderer</a>    | Create and render a <b>Unified Character</b> from a template.               |
| <a href="#">Random Layered Character Renderer</a>      | Create a new <b>Layered Character</b> with completely random layer options. |
| <a href="#">Layered Character Group Entry Renderer</a> | Load a saved <b>Layered Character</b> from a group.                         |

---

## Layered Character Template Renderer

Creates a **Layered Character** from a **Layered Character Template** and displays it in-game.



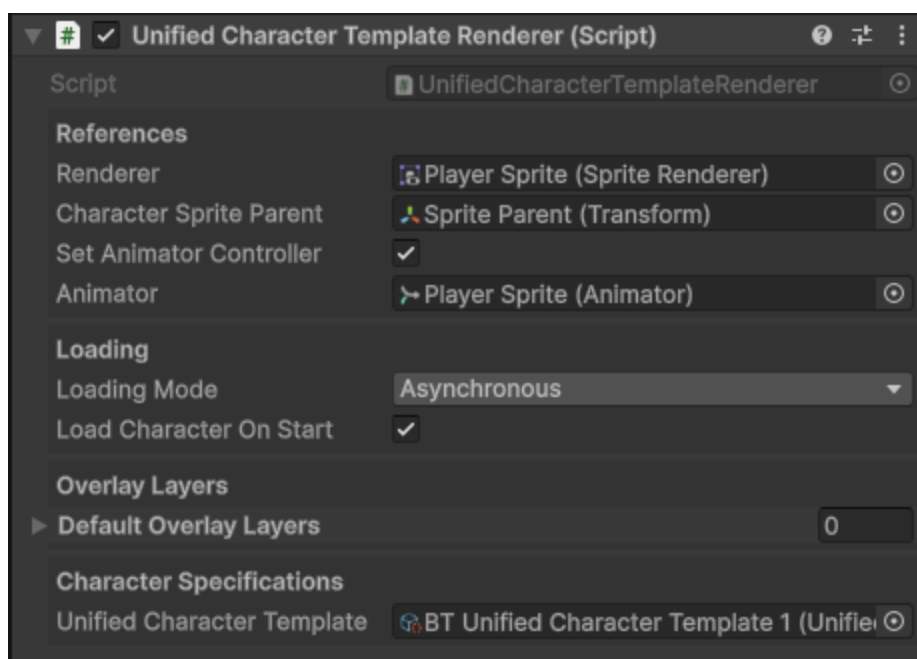
- [Read More → Layered Character Template Renderer](#)

## Requirements

- A [Layered Character Type](#)
- At least one [Layered Character Template](#)

## Unified Character Template Renderer

Creates a Unified Character from a Unified Character Template and displays it in-game.



- [Read More → Unified Character Template Renderer](#)

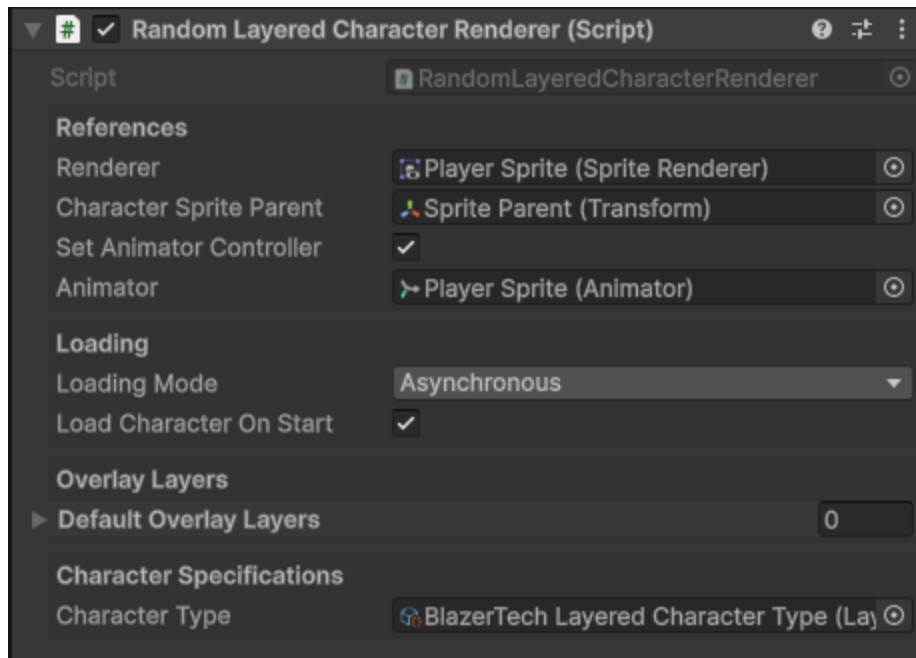
## Requirements

- A [Unified Character Type](#)
  - At least one [Unified Character Template](#)
- 

## Random Layered Character Renderer

Creates a completely randomized **Layered Character** from a **Layered Character Type**.

The **Random Layered Character Renderer** component looks at every layer in the **Character Type** and selects a random **Layer Option** for each layer.



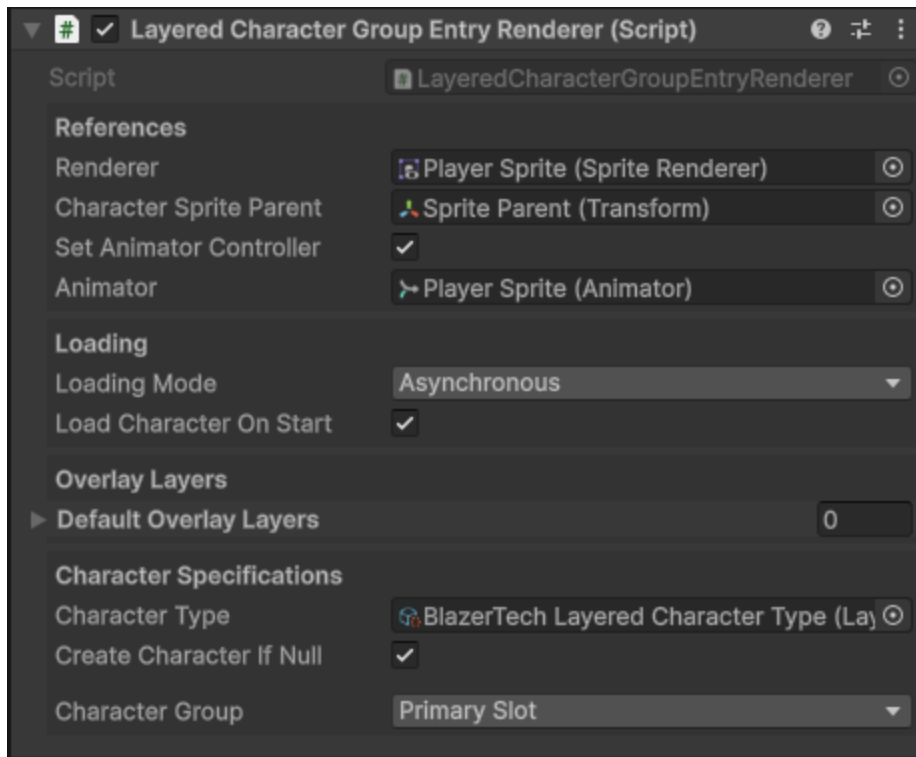
- [Read More → Random Layered Character Renderer](#)

## Requirements

- A [Layered Character Type](#)
- 

## Layered Character Group Entry Renderer

Loads a **Layered Character** of a specific **Layered Character Type** from a **Layered Character Group** and displays it in-game.



- [Read More → Layered Character Group Entry Renderer](#)

## Requirements

- A [Layered Character Type](#)
- At least one **Layered Character** saved in a group

## Related

- [Character Templates](#)
- [Character Types](#)
- [Character Groups](#)

# Layered Character Template Renderer Component

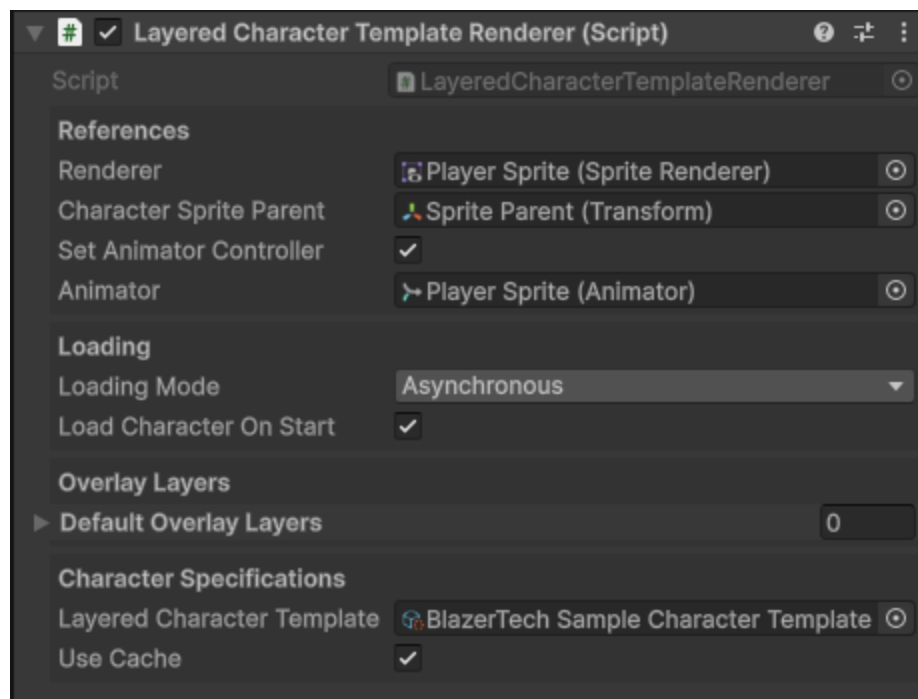
The **Layered Character Template Renderer** creates a new **Layered Character** from a **Layered Character Template** and renders it in-game.

This component is responsible for:

- Creating a **layered character**
- Applying the **Character Shader**
- Assigning the **Animator Controller** (Optional)
- Managing **Overlay Layers**

## Requirements:

- A [Layered Character Type](#)
- At least one [Layered Character Template](#)



## Quick Setup Overview

Quick overview of how to setup the **Layered Character Template Renderer** component.

1. Add the component to a **GameObject**.
2. Assign **Renderer** (Usually a Sprite Renderer).
3. optionally assign **Animator**.
4. Configure **Loading Settings**.

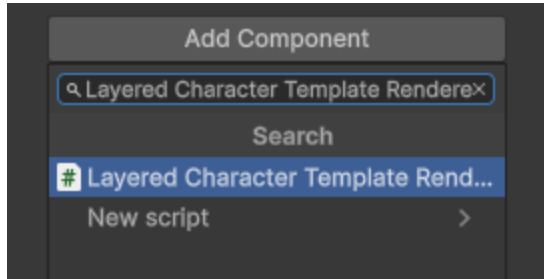
5. Assign the **Layered Character Template** you want to use.
  6. Play your game and if **Load Character On Start** is enabled, your character will be displayed.
- 

## Setup

Full step-by-step guide to setup the **Layered Character Template Renderer** component.

## Add Component

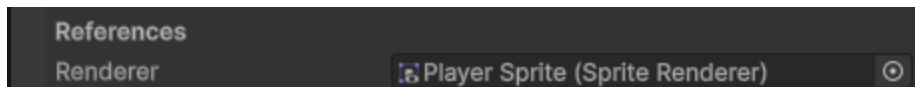
Add the component to a GameObject by clicking **Add Component** in the inspector window.



## Assign Renderer Component

Assign the component responsible for rendering the character's sprites.

In most cases, this will be a **Sprite Renderer**. The **Character Shader** will be applied to this renderer.

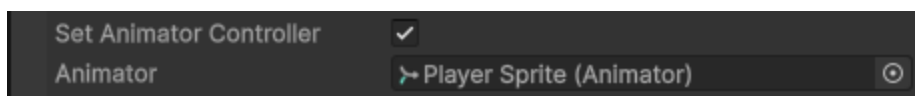


## Assign Animator Component (Optional)

If your **Character Type** asset includes an **Animator Controller**, it can be applied automatically when the character is loaded.

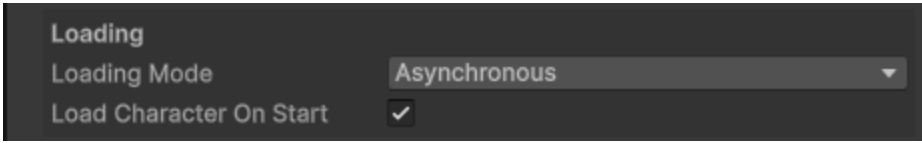
To enable this:

1. Enable **Set Animator Controller**.
2. Assign the target Animator component.



## Configure Loading Settings

The **Loading Mode** determines how the character is loaded at runtime. This can significantly impact performance.



| Loading Mode | Description                             | Advantages                                        | Cons                                                    |
|--------------|-----------------------------------------|---------------------------------------------------|---------------------------------------------------------|
| Asynchronous | Loads the character in the background.  | No frame stutters or freezes.                     | Character is temporarily invisible while loading.       |
| Synchronous  | Loads the character on the main thread. | Character is immediately rendered the same frame. | May cause frame stutters depending on spritesheet size. |

## Recommended Usage

- Use **Asynchronous** loading for background NPCs or characters that aren't time sensitive.
- Use **Synchronous** loading when the character must appear Immediately, such as a player character.

## Load Character On Start

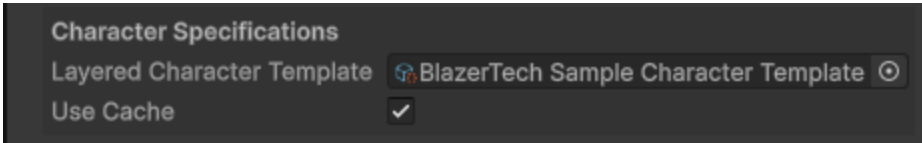
If **Load Character on Start** is enabled, the character is automatically created and rendered during the `Start()` method

If disabled, the character must be loaded manually through script using the [GetAndShowCharacter\(\)](#) method

## Assign Template

Assign a **Layered Character Template** asset.

This will be used to create a new character based on the specifications of the template.



## Use Cache

Determines whether previously created characters are reused.

- **Enabled**  
If a character has already been created using the same template, the cached instance is reused.

- **Disabled**

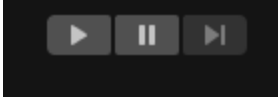
A new character will always be created regardless of if one already exists.

In most cases keep this enabled for improved performance.

Disable when using a **Randomized Template**, so a new random character is created each time.

---

## Hit Play



Enter Play Mode.

If **Load Character On Start** is enabled in **Loading Settings**, the character will be automatically created and displayed.

# Unified Character Template Renderer Component

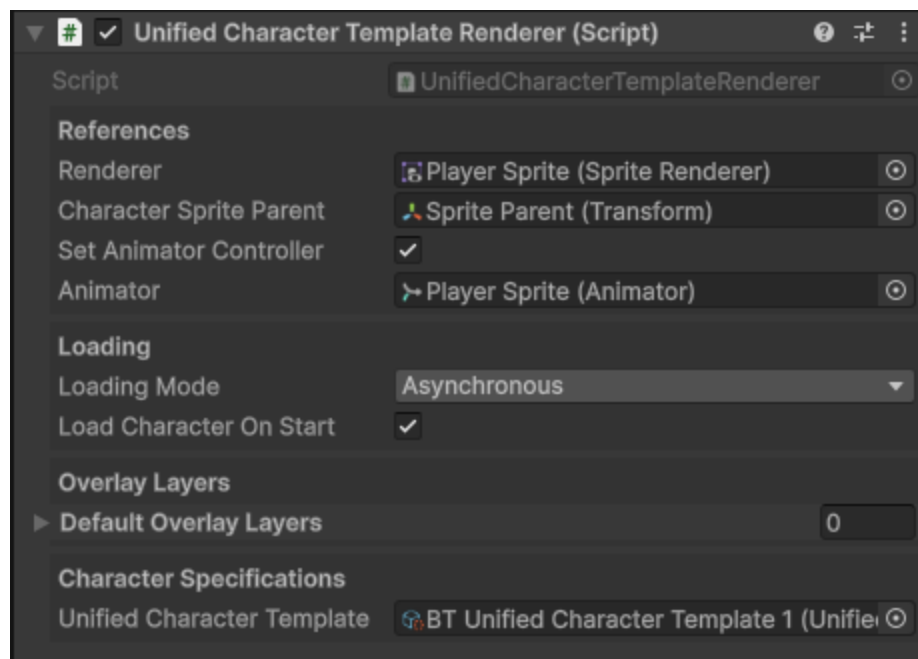
The **Unified Character Template Renderer** creates a new **Unified Character** from a **Unified Character Template** and renders it in-game.

This component is responsible for:

- Creating a **unified character**
- Applying the **Character Shader**
- Assigning the **Animator Controller** (Optional)
- Managing **Overlay Layers**

## Requirements:

- A [Unified Character Type](#)
- At least one [Unified Character Template](#)



## Quick Setup Overview

Quick overview of how to setup the **Unified Character Tempalte Renderer** component.

1. Add the component to a **GameObject**.
2. Assign **Renderer** (Usually a Sprite Renderer).
3. optionally assign **Animator**.
4. Configure **Loading Settings**.
5. Assign the **Unified Character Template** you want to use.

6. Play your game and if **Load Character On Start** is enabled, your character will be displayed.

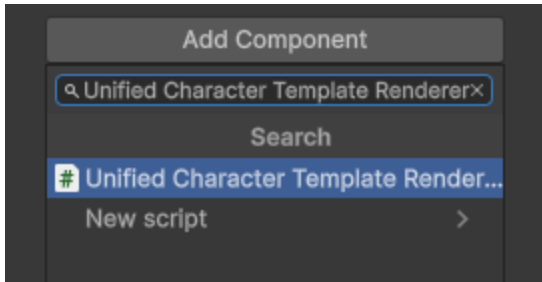
---

## Setup

Full step-by-step guide to setup the **Unified Character Template Renderer** component.

## Add Component

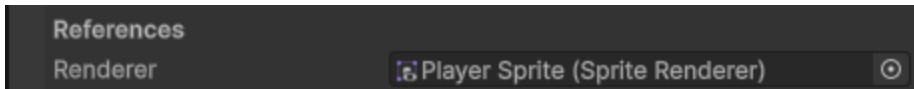
Add the component to a GameObject by clicking **Add Component** in the inspector window.



## Assign Renderer Component

Assign the component responsible for rendering the character's sprites.

In most cases, this will be a **Sprite Renderer**. The **Character Shader** will be applied to this renderer.

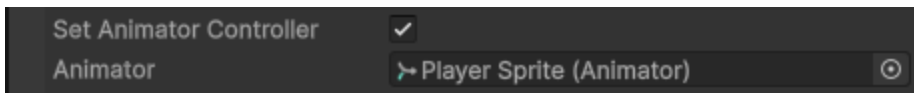


## Assign Animator Component (Optional)

If your **Character Type** asset includes an **Animator Controller**, it can be applied automatically when the character is loaded.

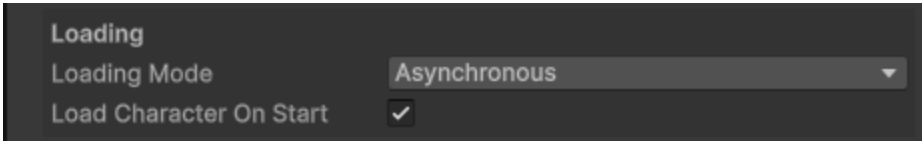
To enable this:

1. Enable **Set Animator Controller**.
2. Assign the target Animator component.



## Configure Loading Settings

The **Loading Mode** determines how the character is loaded at runtime. This can significantly impact performance.



| Loading Mode | Description                             | Advantages                                        | Cons                                                    |
|--------------|-----------------------------------------|---------------------------------------------------|---------------------------------------------------------|
| Asynchronous | Loads the character in the background.  | No frame stutters or freezes.                     | Character is temporarily invisible while loading.       |
| Synchronous  | Loads the character on the main thread. | Character is immediately rendered the same frame. | May cause frame stutters depending on spritesheet size. |

## Recommended Usage

- Use **Asynchronous** loading for background NPCs or characters that aren't time sensitive.
- Use **Synchronous** loading when the character must appear Immediately, such as a player character.

## Load Character On Start

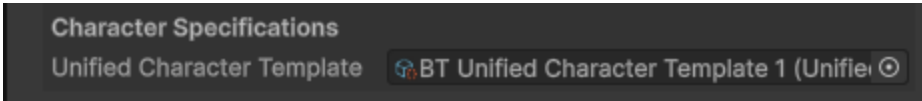
If **Load Character on Start** is enabled, the character is automatically created and rendered during the `Start()` method

If disabled, the character must be loaded manually through script using the [GetAndShowCharacter\(\)](#) method

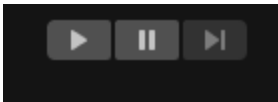
## Assign Template

Assign a **Unified Character Template** asset.

This will be used to create a new character based on the specifications of the template.



## Hit Play



Enter Play Mode.

If **Load Character On Start** is enabled in **Loading Settings**, the character will be automatically created and displayed.

# Random Layered Character Renderer Component

The **Random Layered Character Renderer** creates a new completely randomized **Layered Character** at runtime by selecting a random **Layer Option** for each layer of the character.

## NOTE

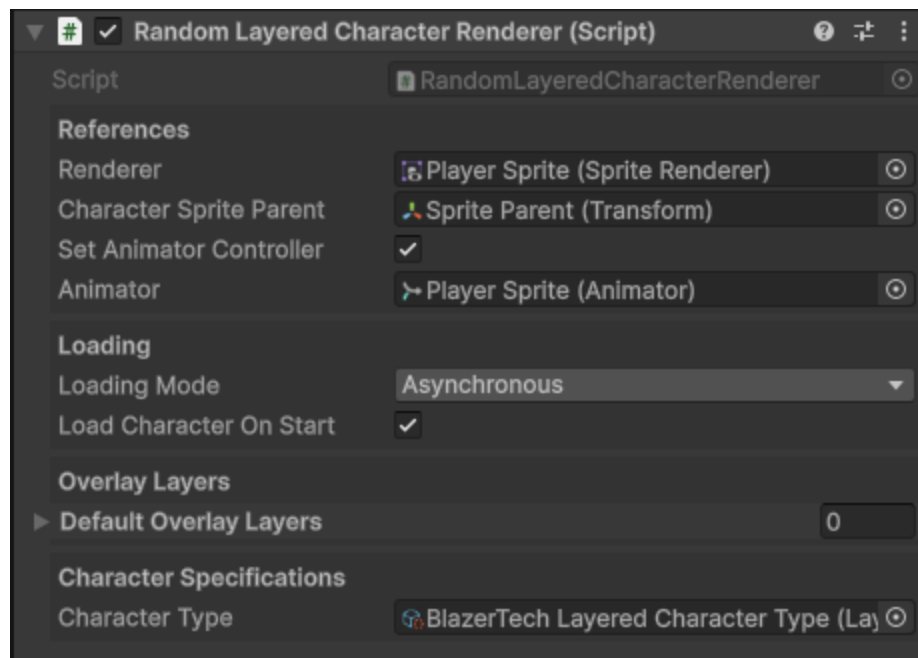
This component contains no options for controlling how the character is randomized. For advanced character randomization use a [Randomized Layered Character Template](#).

This component is responsible for:

- Creating a random **layered character**
- Applying the **Character Shader**
- Assigning the **Animator Controller** (Optional)
- Managing **Overlay Layers**

## Requirements:

- A [Layered Character Type](#)



## Quick Setup Overview

Quick overview of how to setup the **Random Layered Character Renderer** component.

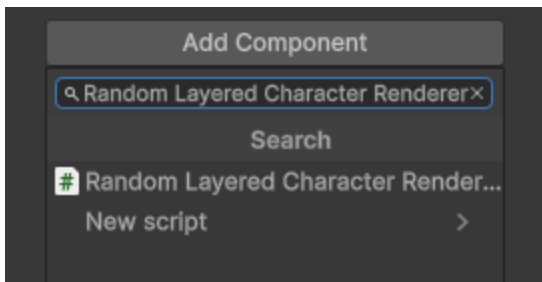
1. Add the component to a **GameObject**.
  2. Assign **Renderer** (Usually a Sprite Renderer).
  3. optionally assign **Animator**.
  4. Configure **Loading Settings**.
  5. Assign the **Layered Character Type** you want to use.
  6. Play your game and if **Load Character On Start** is enabled, a new completely randomized character will be displayed.
- 

## Setup

Full step-by-step guide to setup the **Random Layered Character Renderer** component.

## Add Component

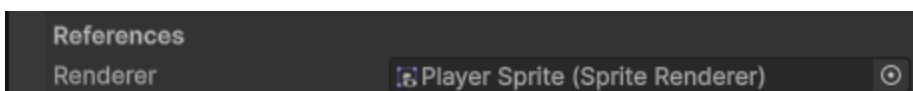
Add the component to a GameObject by clicking **Add Component** in the inspector window.



## Assign Renderer Component

Assign the component responsible for rendering the character's sprites.

In most cases, this will be a **Sprite Renderer**. The **Character Shader** will be applied to this renderer.

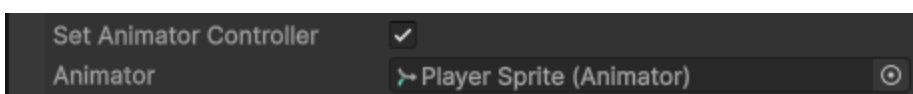


## Assign Animator Component (Optional)

If your **Character Type** asset includes an **Animator Controller**, it can be applied automatically when the character is loaded.

To enable this:

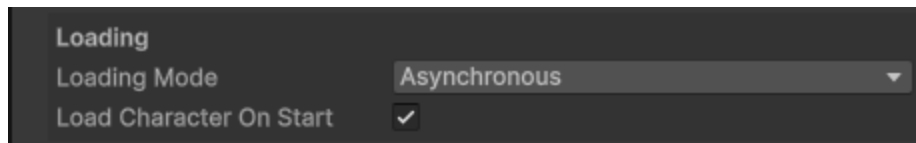
1. Enable **Set Animator Controller**.
2. Assign the target Animator component.



---

## Configure Loading Settings

The **Loading Mode** determines how the character is loaded at runtime. This can significantly impact performance.



| Loading Mode | Description                             | Advantages                                        | Cons                                                    |
|--------------|-----------------------------------------|---------------------------------------------------|---------------------------------------------------------|
| Asynchronous | Loads the character in the background.  | No frame stutters or freezes.                     | Character is temporarily invisible while loading.       |
| Synchronous  | Loads the character on the main thread. | Character is immediately rendered the same frame. | May cause frame stutters depending on spritesheet size. |

## Recommended Usage

- Use **Asynchronous** loading for background NPCs or characters that aren't time sensitive.
- Use **Synchronous** loading when the character must appear Immediately, such as a player character.

---

## Load Character On Start

If **Load Character on Start** is enabled, the character is automatically created and rendered during the **Start()** method

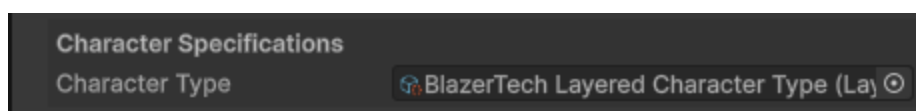
If disabled, the character must be loaded manually through script using the [GetAndShowCharacter\(\)](#) method

---

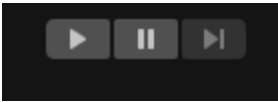
## Assign Character Type

Assign a **Layered Character Type** asset.

The renderer will create a randomized character using the **Character Type** asset you assign.



## Hit Play



Enter Play Mode.

If **Load Character On Start** is enabled in **Loading Settings**, the character will be automatically created and displayed.

# Layered Character Group Entry Renderer Component

The **Layered Character Group Entry Renderer** loads a **Layered Character** from a **Layered Character Group** and renders it in-game.

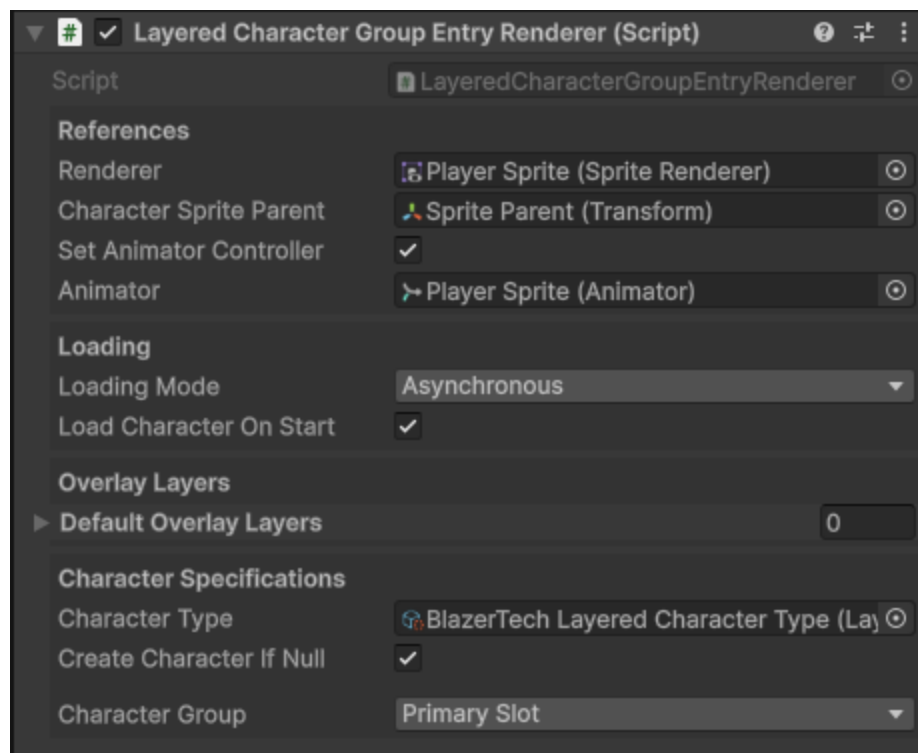
This component is responsible for:

- Loading a **layered character** from a group
- Applying the **Character Shader**
- Assigning the **Animator Controller** (Optional)
- Managing **Overlay Layers**

## Requirements:

- A [Layered Character Type](#)
- At least one **Layered Character** saved in a group

[Read More → Character Groups](#)



## Quick Setup Overview

Quick overview of how to setup the **Layered Character Group Entry Renderer** component.

1. Add the component to a **GameObject**.

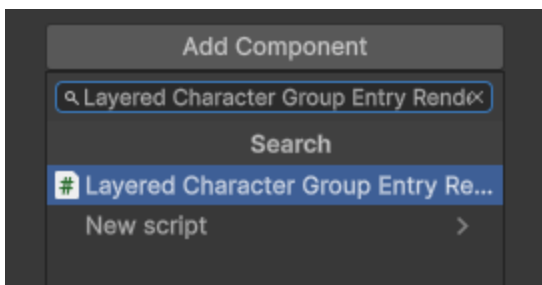
2. Assign **Renderer** (Usually a Sprite Renderer).
  3. optionally assign **Animator**.
  4. Configure **Loading Settings**.
  5. Assign the **Character Type** you want to load a character from.
  6. Select the **Character Group** (Primary, Flexible or Fixed) and configure parameters.
- 

## Setup

Full step-by-step guide to setup the **Layered Character Group Entry Renderer** component.

## Add Component

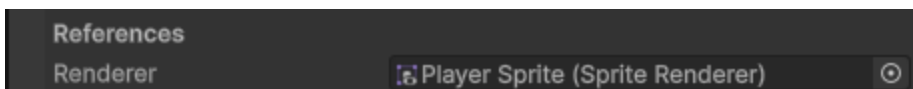
Add the component to a GameObject by clicking **Add Component** in the inspector window.



## Assign Renderer Component

Assign the component responsible for rendering the character's sprites.

In most cases, this will be a **Sprite Renderer**. The **Character Shader** will be applied to this renderer.

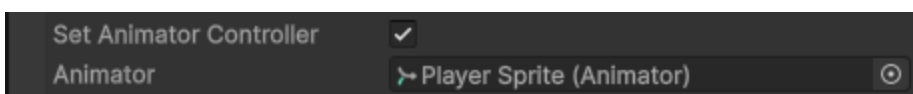


## Assign Animator Component (Optional)

If your **Character Type** asset includes an **Animator Controller**, it can be applied automatically when the character is loaded.

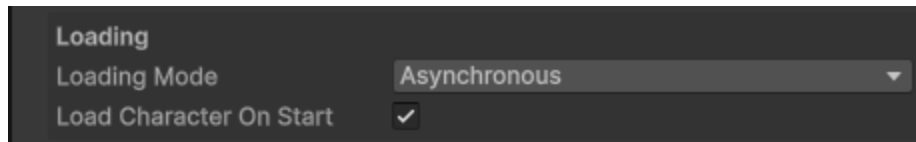
To enable this:

1. Enable **Set Animator Controller**.
2. Assign the target Animator component.



## Configure Loading Settings

The **Loading Mode** determines how the character is loaded at runtime. This can significantly impact performance.



| Loading Mode | Description                             | Advantages                                        | Cons                                                    |
|--------------|-----------------------------------------|---------------------------------------------------|---------------------------------------------------------|
| Asynchronous | Loads the character in the background.  | No frame stutters or freezes.                     | Character is temporarily invisible while loading.       |
| Synchronous  | Loads the character on the main thread. | Character is immediately rendered the same frame. | May cause frame stutters depending on spritesheet size. |

## Recommended Usage

- Use **Asynchronous** loading for background NPCs or characters that aren't time sensitive.
- Use **Synchronous** loading when the character must appear immediately, such as a player character.

## Load Character On Start

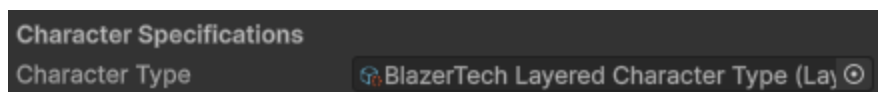
If **Load Character on Start** is enabled, the character is automatically created and rendered during the `Start()` method

If disabled, the character must be loaded manually through script using the [GetAndShowCharacter\(\)](#) method

## Assign Character Type

Assign a **Layered Character Type** asset.

This will be used when looking for a character group.



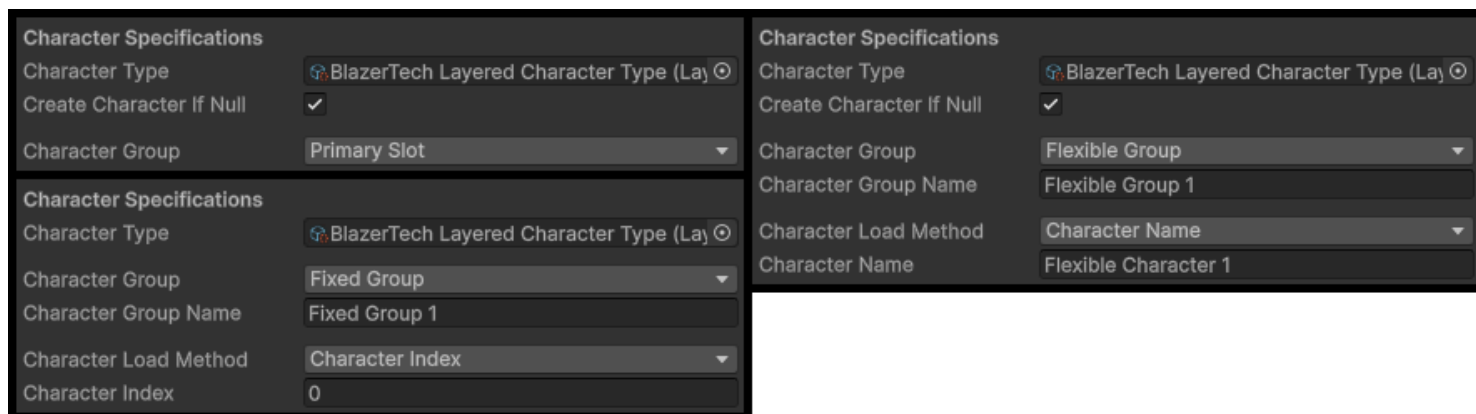
## Set Character Group Type

Choose the type of group you want to load from.

| Group Type                    | Description                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Primary Character Slot</b> | A single character attached to the Character Type asset. No additional parameters required.           |
| <b>Flexible Group</b>         | A group of characters that can be added, removed, or edited at any time.                              |
| <b>Fixed Group</b>            | A group with a preset number of characters. New characters cannot be added or removed after creation. |

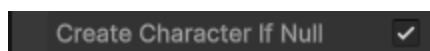
If **Flexible Group** or **Fixed Group** is selected, the following parameters are required:

| Parameter                    | Type   | Description                                                                                                                                                                                                                                                                                                                                       |
|------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Character Group Name</b>  | String | A unique name used to find the Fixed or Flexible group.                                                                                                                                                                                                                                                                                           |
| <b>Character Load Method</b> | Enum   | Determines how a character is selected from the group: <ul style="list-style-type: none"> <li>- <b>Character Name</b> &gt; Load a character by its saved name.</li> <li>- <b>Character Index</b> &gt; Load a character by its index position in the group.</li> <li>- <b>Randomized</b> &gt; Randomly load a character from the group.</li> </ul> |



## Create Character If Null

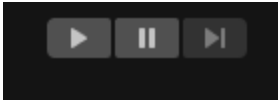
If enabled and a group was successfully found but no character was found in the group matching your criteria, a new blank character will be created and added to the group.



### NOTE

Only affects the **Primary Slot** and **Flexible Character Groups**. **Fixed Character Groups** can't have characters added to them after creation.

## Hit Play



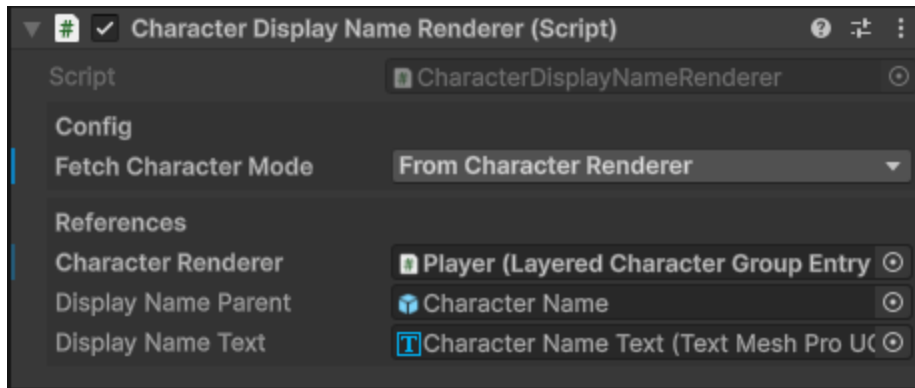
Enter Play Mode and if:

1. **Load Character On Start** is enabled in **Loading Settings**
2. The renderer is able to find the character group

Then the character will be automatically loaded and displayed.

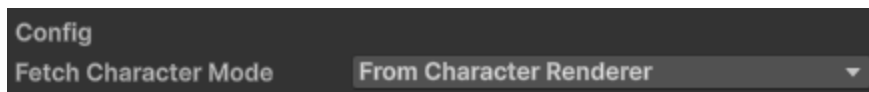
# Character Display Name Renderer

Provides an easy way to show the Display Name of a character in-game.



## Config

Choose how you'll get a reference to a character. There are two options:



## From Character Renderer

Uses a [Character Renderer component](#) to find a character.

Assign a reference to a **Character Renderer component**.

When the Character Renderer component loads a character:

1. The **Display Name Renderer** will be notified and retrieve the character.
2. The **display Name Renderer** will get the **Display Name** of the character and set the text of the assigned text component to the characters display name.

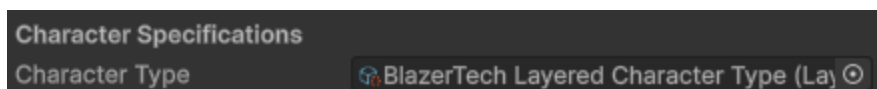
## From Character Group

Attempts to find a character from a character group at runtime.

## Assign Character Type

Assign a **Layered Character Type** asset.

This will be used when looking for a character group.



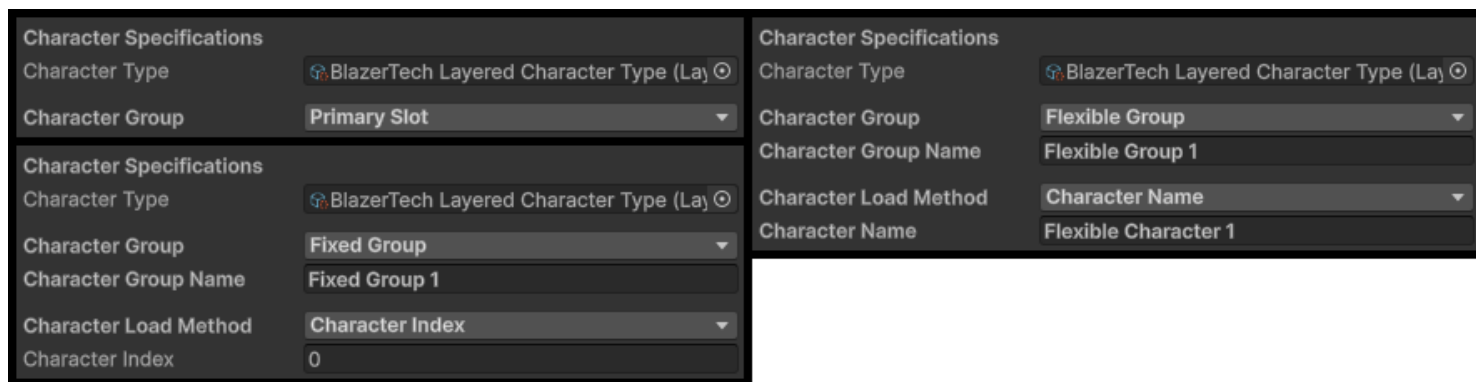
## Set Character Group Type

Choose the type of group you want to load from.

| Group Type                    | Description                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Primary Character Slot</b> | A single character attached to the Character Type asset. No additional parameters required.           |
| <b>Flexible Group</b>         | A group of characters that can be added, removed, or edited at any time.                              |
| <b>Fixed Group</b>            | A group with a preset number of characters. New characters cannot be added or removed after creation. |

If **Flexible Group** or **Fixed Group** is selected, the following parameters are required:

| Parameter                    | Type   | Description                                                                                                                                                                                                                                                                                                                                   |
|------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Character Group Name</b>  | String | A unique name used to find the Fixed or Flexible group.                                                                                                                                                                                                                                                                                       |
| <b>Character Load Method</b> | Enum   | Determines how a character is selected from the group: <ul style="list-style-type: none"><li>- <b>Character Name</b> &gt; Load a character by its saved name.</li><li>- <b>Character Index</b> &gt; Load a character by its index position in the group.</li><li>- <b>Randomized</b> &gt; Randomly load a character from the group.</li></ul> |



## References

| Field                      | Type       | Description                                                        |
|----------------------------|------------|--------------------------------------------------------------------|
| <b>Display Name Parent</b> | GameObject | Parent GameObject used to disable text when Display Name is blank. |

| Field             | Type     | Description                                                             |
|-------------------|----------|-------------------------------------------------------------------------|
| Display Name Text | TMP_Text | Reference to a TMP Text component used to render the Display Name text. |

| References          |                                                                                                                                                                                                           |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Display Name Parent |  Character Name                         |
| Display Name Text   |  Character Name Text (Text Mesh Pro UC  |

 **TIP**

Use a canvas with a **Render Mode** set to **World Space**. This lets you display UI elements within the world. In this case it'll sit on top of the character and follow along with it.

# Character Animator Handler Components

**Animator Handler components** control **parameters** within an **Animator Controller**.

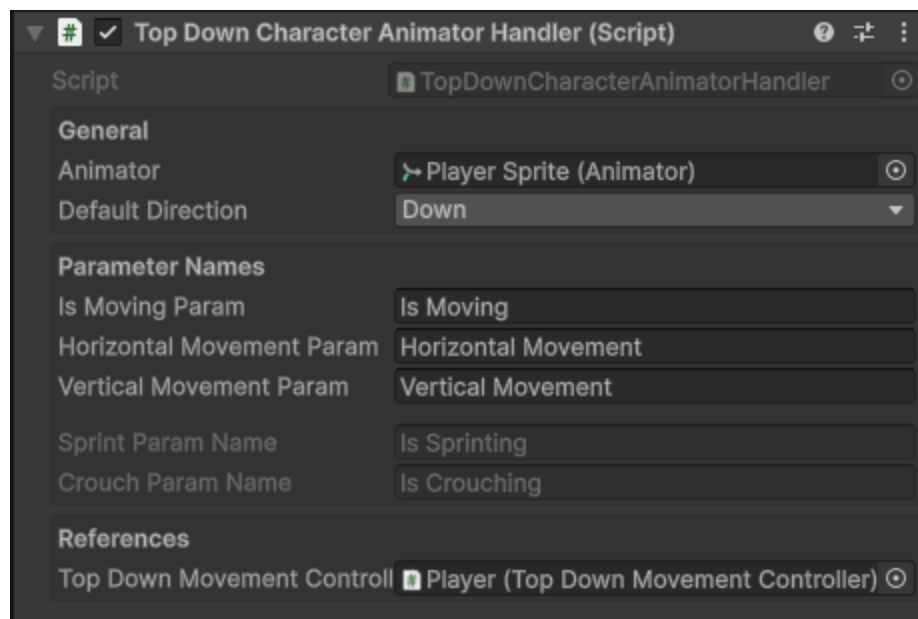
All **Animator Handler components** require a reference to an **Animator component**.

## TIP

An **Animator Controller** can be assigned in any **Character Type** asset and be automatically used when a character of that type is used.

## Top Down Character Animator Handler

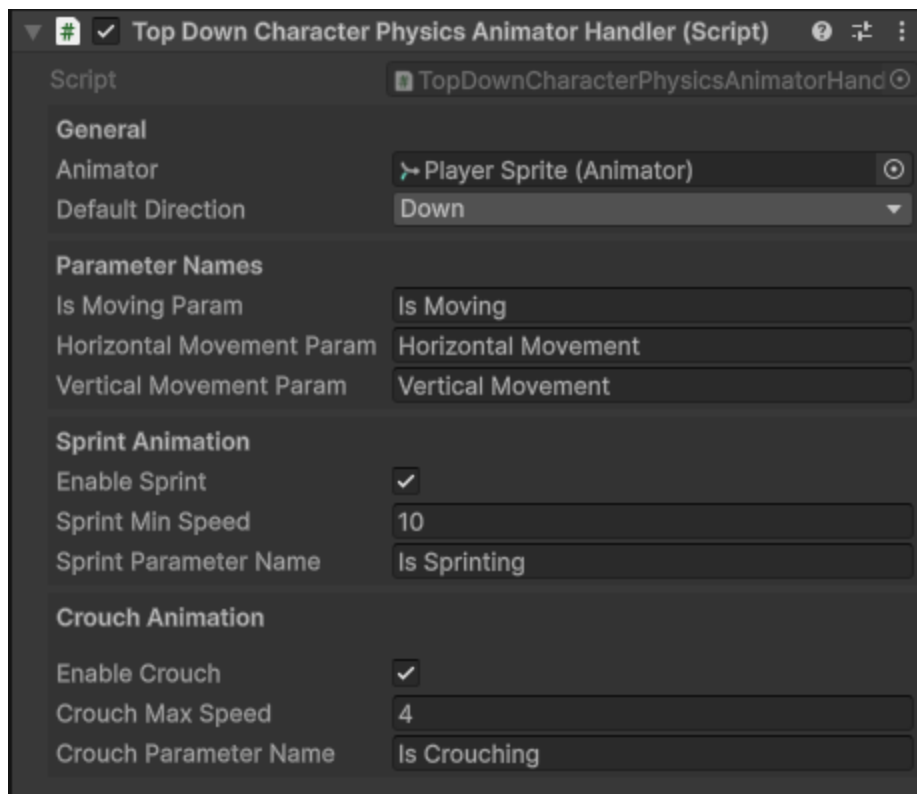
Reads input from a [Top Down Movement Controller](#) component to update **parameters** in an **Animator Controller**.



- [Read More → Top Down Character Animator Handler](#)

## Top Down Character Physics Animator Handler

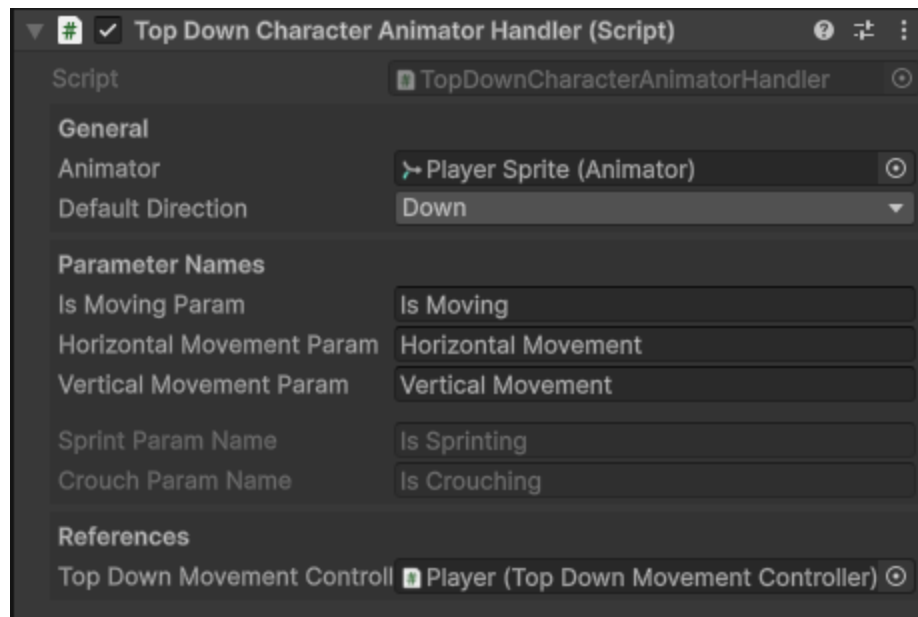
A physics driven component, every frame the GameObject's position is compared to its position last frame to determine **direction** and **speed**. This information is then used to update **parameters** in an **Animator Controller**.



- [Read More → Top Down Character Physics Animator Handler](#)

# Top Down Character Animator Handler

Syncs **parameters** in an **Animator Controller** to values within the [Top Down Movement Controller](#) component.



## How It Works

- Reads movement and state data from the assigned **Top Down Movement Controller** component.
- Updates **Animator Controller parameters** in-real time to reflect the character's state.
- **Animator Controller** uses those **parameters** to play specific animations.

## Inspector Fields

### References

| Parameter                  | Type                                   | Description                                                                    |
|----------------------------|----------------------------------------|--------------------------------------------------------------------------------|
| <b>Animator</b>            | <code>Animator</code>                  | The <b>Animator component</b> used to set parameters.                          |
| <b>Default Direction</b>   | <code>Enum</code>                      | The default direction the character should face.                               |
| <b>Movement Controller</b> | <code>TopDownMovementController</code> | The <b>Top Down Movement Controller</b> component used to listen for movement. |

### Parameter Names

| Parameter                        | Type   | Description                                                                                                                                                                                                                                       |
|----------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Is Moving Param</b>           | Bool   | The <b>Animator parameter name</b> used to indicate whether the character is moving.<br>Set to <b>true</b> when movement is detected.                                                                                                             |
| <b>Horizontal Movement Param</b> | String | The <b>Animator parameter name</b> for horizontal input.<br><b>-1</b> when moving left and <b>1</b> when moving right.                                                                                                                            |
| <b>Vertical Movement Param</b>   | String | The <b>Animator parameter name</b> for vertical input.<br><b>-1</b> when moving down and <b>1</b> when moving up.                                                                                                                                 |
| <b>Sprint Param</b>              | Bool   | The <b>Animator parameter name</b> used to indicate when the character is <b>sprinting</b> .<br><b>True</b> when sprinting, otherwise <b>false</b> .<br>Only available when <b>Sprint</b> is enabled in the <b>Top Down Movement Controller</b> . |
| <b>Crouch Param</b>              | Bool   | The <b>Animator parameter name</b> used to indicate when the character is <b>crouching</b> .<br><b>True</b> when crouched, otherwise <b>false</b> .<br>Only available when <b>Crouch</b> is enabled in the <b>Top Down Movement Controller</b> .  |

## Public Methods

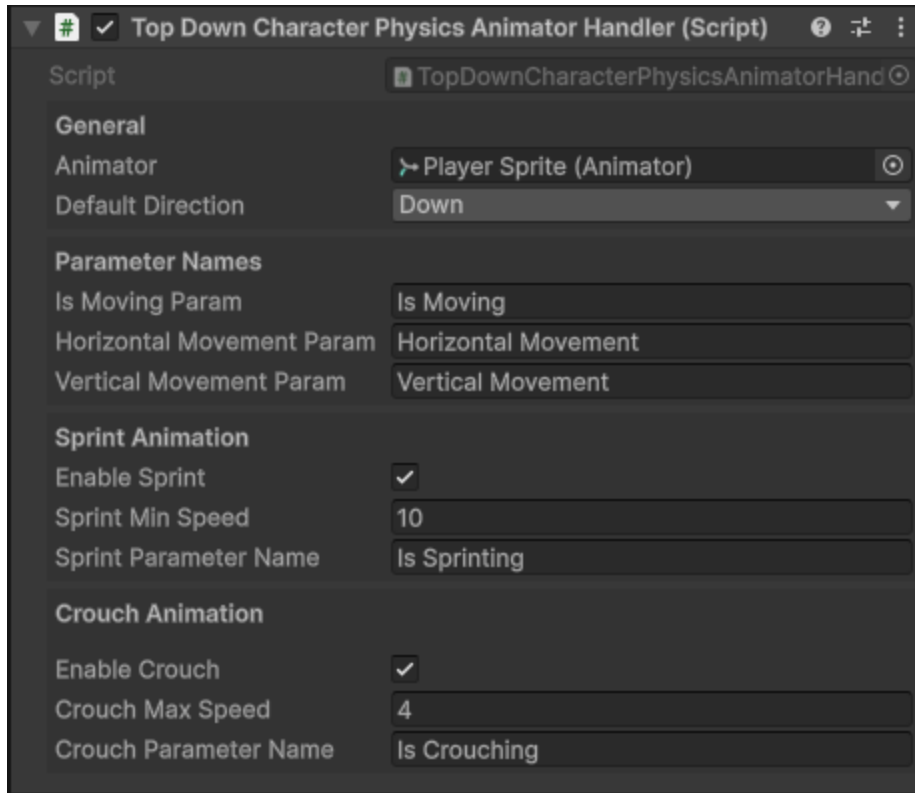
All of these methods can be called directly from any instance of the **Top Down Character Animator Handler** component.

- **ChangeDirection(FourDirectional/EightDirectional)**: Immediately update the direction the character is facing.
- **PlayAnimation(string)**: Play a specific animation or blend tree by state name.
- **PlayDefaultAnimation()**: Play the default animation state defined in the **Animator Controller**.

# Top Down Character Physics Animator Handler

The **Top-Down Character Physics Animator Handler** is physics-driven, Every fixed update the position of the GameObject the component is attached to is compared to its position last frame.

This is used to determine the **direction** the character is moving and **speed** the character is traveling at.



## ⊗ IMPORTANT

Due to the nature of how the **Top-Down Character Physics Animator Handler** works, crouch animations can only be played when the character is moving.

## How It Works

- Calculates movement each `FixedUpdate()` by comparing current and previous positions.
- Determines direction and speed, then updates **Animator parameters**.
- Optionally toggles **sprint/crouch** states when enabled.
- **Animator Controller** uses those parameters to play specific animations.

## Inspector Fields

## References

| Parameter       | Type            | Description                                           |
|-----------------|-----------------|-------------------------------------------------------|
| <b>Animator</b> | <i>Animator</i> | The <b>Animator component</b> used to set parameters. |

## Parameter Names

| Parameter                        | Type          | Description                                                                                                                           |
|----------------------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Is Moving Param</b>           | <i>Bool</i>   | The <b>Animator parameter name</b> used to indicate whether the character is moving.<br>Set to <i>true</i> when movement is detected. |
| <b>Horizontal Movement Param</b> | <i>String</i> | The <b>Animator parameter name</b> for horizontal input.<br><i>-1</i> when moving left and <i>1</i> when moving right.                |
| <b>Vertical Movement Param</b>   | <i>String</i> | The <b>Animator parameter name</b> for vertical input.<br><i>-1</i> when moving down and <i>1</i> when moving up.                     |

## Sprint Animation

| Field                        | Type          | Description                                                                                |
|------------------------------|---------------|--------------------------------------------------------------------------------------------|
| <b>Enable Sprint</b>         | <i>Bool</i>   | If true, enables sprinting animations when speed is greater than <b>Sprint Min Speed</b> . |
| <b>Sprint Min Speed</b>      | <i>Float</i>  | Minimum speed required to trigger sprint animation.                                        |
| <b>Sprint Parameter Name</b> | <i>String</i> | The <b>Animator parameter name</b> for the sprint state (default: "Is Sprinting").         |

## Crouch Animation

| Field                        | Type          | Description                                                                             |
|------------------------------|---------------|-----------------------------------------------------------------------------------------|
| <b>Enable Crouch</b>         | <i>Bool</i>   | If true, enables crouching animations when speed is less than <b>Crouch Max Speed</b> . |
| <b>Crouch Max Speed</b>      | <i>Float</i>  | Maximum speed for crouch animations to be used.                                         |
| <b>Crouch Parameter Name</b> | <i>String</i> | The <b>Animator parameter name</b> for the crouch state (default: "Is Crouching").      |

## Public Methods

All of these methods can be called directly from any instance of the **Top Down Character Physics Animator Handler** component.

- `ChangeDirection(FourDirectional/EightDirectional)`: Immediately update the direction the character is facing.
- `PlayAnimation(string)`: Play a specific animation or blend tree by state name.
- `PlayDefaultAnimation()`: Play the default animation state defined in the **Animator Controller**.

# Character Movement Controller Components

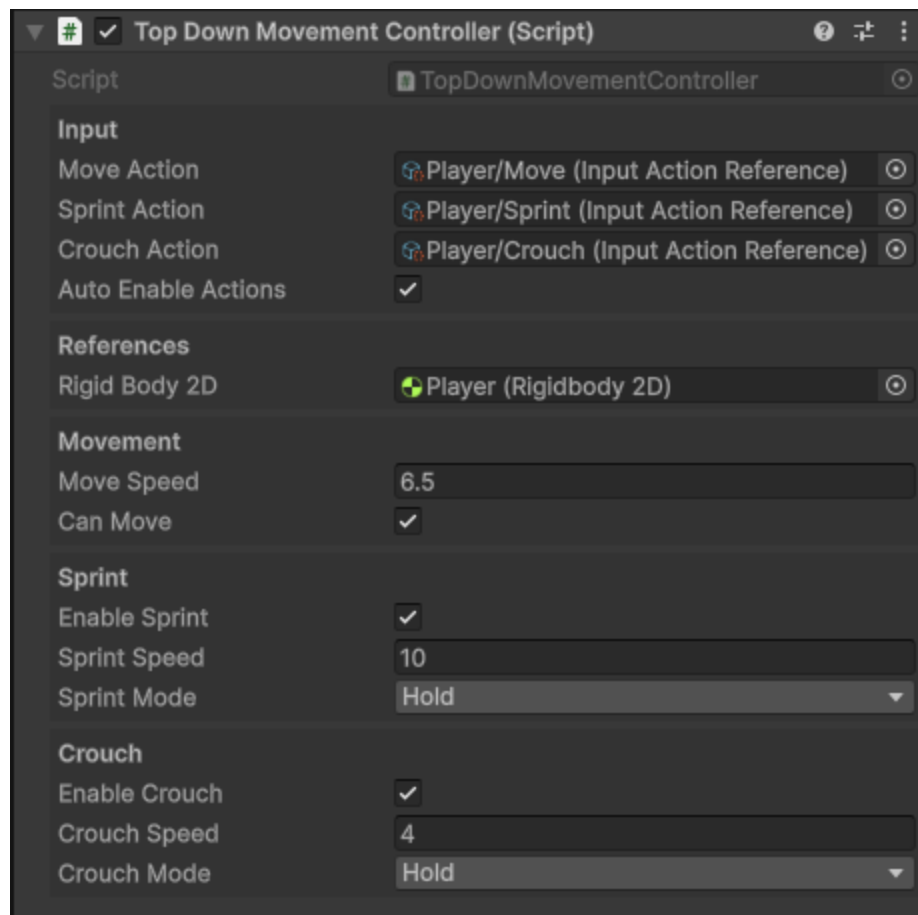
**Character Controllers** are components which let the player control the movement of a GameObject.

When used with a [Character Animator Handler](#) you can both control and animate any character with ease.

## Top Down Character Movement Controller

Handles character movement along the X and Y axis for 2d top-down games.

- Uses the **New Input System**.
- Optional **Sprint** and **Crouch** support
- Handles movement through a **Rigidbody2D** component.



- [Read More → Top Down Character Movement Controller](#)

# Top Down Character Movement Controller

Handles player movement for 2d top-down games where there are four directions the player can move (Left, right, up, down).

## Input Configuration

This component uses Unity's **New Input System**. Every input action is configurable.

[Input Actions](#) are used to easily modify what inputs are used for each action.

## Input Actions

| Input Action         | Type    | Usage                                                                    |
|----------------------|---------|--------------------------------------------------------------------------|
| <b>Move Action</b>   | Vector2 | The input action used to control player movement along the X and Y axes. |
| <b>Sprint Action</b> | Button  | The input action used to let the player sprint. (If enabled)             |
| <b>Crouch Action</b> | Button  | The input action used to let the player crouch. (If enabled)             |

A default **Input Action asset** is included under the `/Input Actions` subfolder.

This asset contains the default input actions for moving, sprinting and crouching.

## Auto Enable Actions

If **Auto Enable Actions** is enabled, the component automatically enables and disables the assigned input actions when the GameObject is enabled or disabled.

This is useful when not using a **PlayerInput** component or a [project-wide input actions asset](#) that handles enabling automatically.

## Movement Settings

| Field             | Type  | Description                                                                                      |
|-------------------|-------|--------------------------------------------------------------------------------------------------|
| <b>Move Speed</b> | Float | Base walk speed (Default: 6.5).                                                                  |
| <b>Can Move</b>   | Bool  | Toggles whether the character can currently move. Can be changed via script or in the inspector. |

## Sprinting & Crouching

Optional sprinting and crouching systems can be toggled on via their corresponding booleans. Each system has customizable speed and button mode options.

| Field        | Type  | Description                                                                    |
|--------------|-------|--------------------------------------------------------------------------------|
| <b>Speed</b> | Float | Movement speed while sprint or crouching.                                      |
| <b>Mode</b>  | Enum  | Determines whether the button must be held or toggled. (Default: <b>Hold</b> ) |

## References

| Reference          | Type        | Description                                    |
|--------------------|-------------|------------------------------------------------|
| <b>Rigidbody2D</b> | Rigidbody2D | Required reference used for applying movement. |

## Runtime Properties

| Property           | Type    | Description                                |
|--------------------|---------|--------------------------------------------|
| <b>IsMoving</b>    | bool    | True if the player is currently moving.    |
| <b>IsSprinting</b> | bool    | True if the player is currently sprinting. |
| <b>IsCrouching</b> | bool    | True if the player is currently crouching. |
| <b>Movement</b>    | Vector2 | Current normalized movement direction.     |

### TIP

Designed to be used along with a [Character Animator Handler](#) component. When used together they provide both character movement and animation functionality.

# Character Templates

**Character Templates** are Scriptable Objects that act as blueprints for creating characters at runtime.

Instead of manually configuring characters in code, Templates let you define reusable character configurations directly in the Editor. They provide an easy way to use pre-made characters anywhere in your project.

**Character Templates** can be created by **right clicking** the project window and navigating to **Create > BlazerTech > Character Management System > Character Templates**.

|                  |                               |                           |                                       |
|------------------|-------------------------------|---------------------------|---------------------------------------|
| BlazerTech >     | Character Management System > | Character Templates >     | Unified Character Template            |
| D3D12 Settings > |                               | Layered Character Types > | Layered Character Template            |
| Input Actions    |                               | Character Overlay Layer   | Randomized Layered Character Template |
|                  |                               | Unified Character Type    | Layered Character Group Template      |

## What is a Character Template?

A Character Template defines:

- The **Character Type** the character will use
- The default **Name** and optional **Display Name**
- How the character's appearance is configured

At runtime, a **Renderer component** can be used to easily create and display a character from a template.

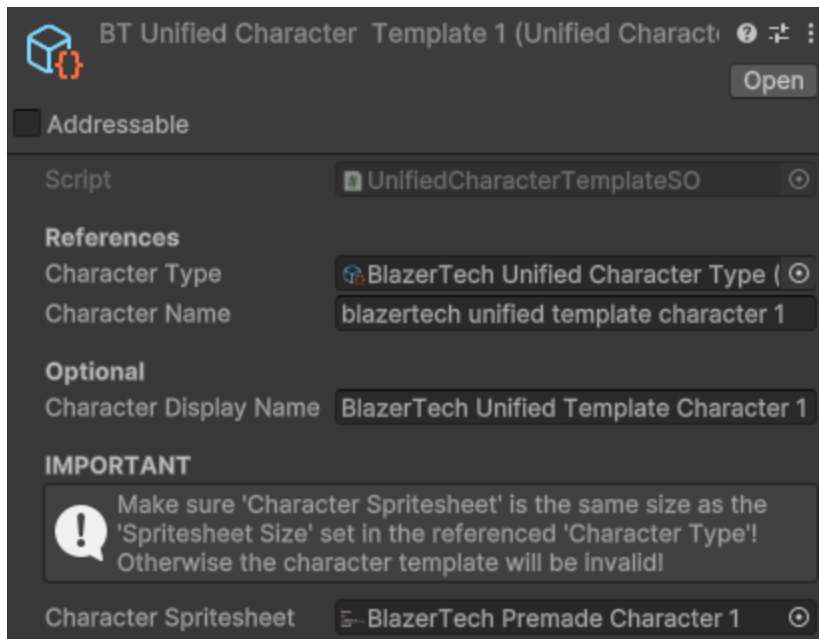
[Read More → Character Renderer Components](#)

## Character Template Variants

There are three types of Character Templates.

### Unified Character Template

Used to create characters with a single, fully assembled spritesheet. They are linked to a [Unified Character Type](#) asset.



Unified characters use a single spritesheet which contains all animations for the character. That spritesheet is assigned to the **Unified Character Template** and is used by any characters created by that template.

#### **NOTE**

The **Character Spritesheet** does **NOT** need to be sliced. All frame and slicing data is contained within the **Unified Character Type** the template is linked to.

[Read More → Unified Character Templates](#)

## Layered Character Template

Used to create modular characters composed of multiple layers.

They are linked to a [Layered Character Type](#) asset.

BT Character Template 1 (Layered Character Template) Open

☐ Addressable

**References**

Character Type: BlazerTech Layered Character Type

Character Name: blazertech template character 1

**Optional**

Character Display Name: BlazerTech Template Character 1

**Body**

Search:

Options (7): Body\_01\_Peach

**Outfit**

Search:

Options (29): Outfit\_05\_Dress\_01

**Hairstyle**

Search:

Options (29): Hairstyle\_02\_Shortlock\_04

**Accessory**

Search:

Options (30): Accessory\_03\_Straw-Hat\_01

Recreate List

A **Layered Character Template** lets you assign which Layer Option each layer should use. This gives you full control over exactly how the character will look.

When used, a new **Layered Character** is created using the **Layer Options** you assign for each layer of the character.

[Read More → Layered Character Templates](#)

---

## Randomized Layered Character Templates

Used to create randomized modular character based on a set of pre-defined rules. Unlike a standard [Layered Character Template](#) where the final character is the same every time, a Randomized Layered Character Template creates a new random character every time it's used.


Blazer Tech Randomized Sample Character Template (Randomized)
Open

☐ Addressable

**References**  
Character Type: BlazerTech Layered Character Type (Layered)  
Character Name: randomized-template-character

**Optional**  
Character Display Name: Randomized Template Character

Body

Selection Mode: Any  
Allow Blank: ☐

Outfit

Selection Mode: Regex  
Allow Blank: ☐  
Regex Pattern: Flannel|T-shirt  
Case Sensitive: ☐  
Invert Match: ☐  
▶ Regex Matches

Hairstyle

Selection Mode: Fixed  
Search:   
Options (29): Hairstyle\_01\_Fine\_Cut\_01

Accessory

Selection Mode: Allow List  
Blacklist: ☐  
▶ Layer Options (22/30)

Recreate List

### Randomized Templates work great for:

- Background NPC generation
- Characters with slightly randomized appearances
- Situations that need controlled randomness with features for additional filtering

### Key Characteristics:

- Per-layer selection rules
- White/Black list support
- Weighing support

- [Regex-based filtering](#)

[Read More → Randomized Layered Character Templates](#)

---

## Related

- [Character Types](#)
- [Character Renderer Components](#)

# Unified Character Templates

A **Unified Character Template** is a **Scriptable Object** and is the most common method to create **Unified Characters** at runtime.

It acts as a reusable blueprint that defines:

- The **Character Type asset** the character will use
- The **default name** assigned when a character is created
- The **spritesheet** used for rendering and animation

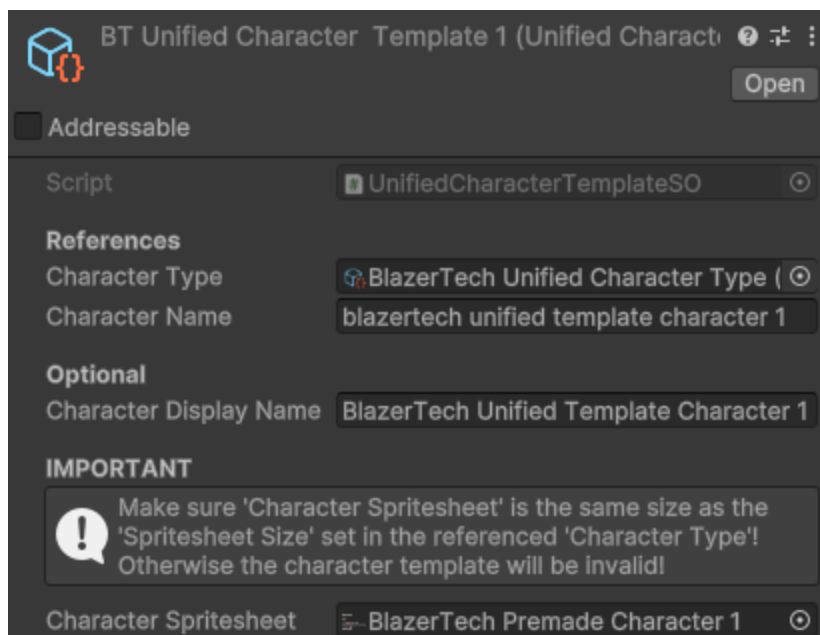
## Workflow Overview

1. Create a **Unified Character Template** asset
2. Assign a **Character Type**
3. Set the default name
4. Assign a properly structured spritesheet
5. Attach the template to a **Unified Character Renderer** component

The template can now be reused anywhere in your project to consistently create that character.

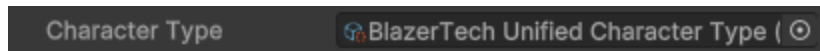
## Setup Create

To create a Unified Character Template **right click** the **Project window** and navigate to **Create > BlazerTech > Character Management System > Character Templates > Unified Character Template**.



## Character Type

Assign the **Character Type asset** that all characters created from this template will use.



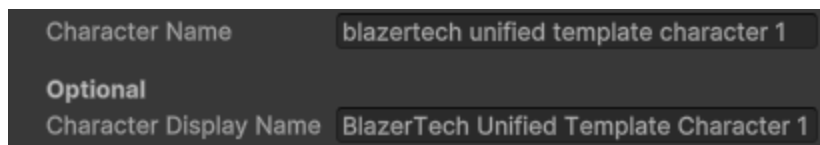
This is used to determine:

- If the **Character Spritesheet** is valid
- The **Animator Controller** used to animate the character

## Default Character Name

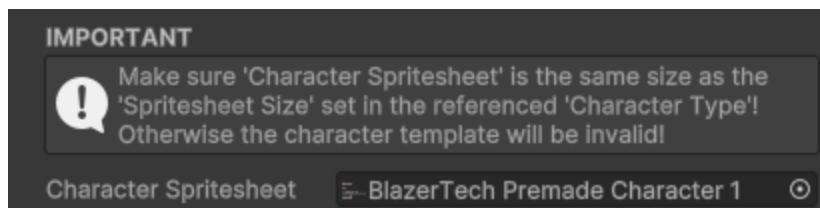
Set the default name assigned when a character is created from this template.

You may also optionally set a **Display Name** for use alongside created characters in-game.



## Character Spritesheet

Assign a spritesheet that has the same size and layout as the Character Spritesheet in the attached Character Type.



### **NOTE**

When the **Character Spritesheet** is assigned, the spritesheet is marked as **Addressable** automatically, allowing it to be loaded/unloaded during runtime.

## Spritesheet Requirements

When a character is created from the template the spritesheet will replace the **Base Spritesheet** used in the **Character Type**, this means the spritesheet must contain the same animations and layout.

Read more about the [Character Shader](#) for more info.

The assigned spritesheet must:

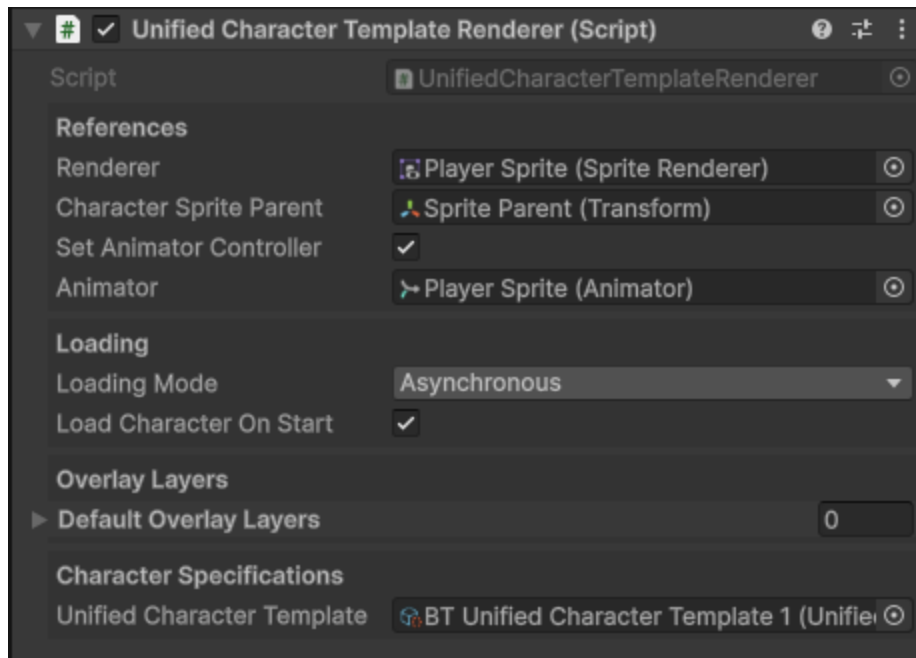
- Be the same size as the **Base Spritesheet** in the Character Type
- Contain all the same animations and frames in the **Base Spritesheet** in the Character Type
- Have the **Sprite Mode** set to **Single**

### ⊗ CAUTION

If the assigned spritesheet is not the correct size it will be rejected at runtime.

## Runtime Usage

Once the **Character Template** is setup it can be assigned in a **Unified Character Template Renderer component** which at runtime will create a character from the template and animate it using the **Animator Controller** assigned in the **Character Type**.



[Read More → Unified Character Template Renderer component](#)

# Layered Character Templates

A **Layered Character Template** is a **Scriptable Object** used to create **Layered Characters** at runtime.

It Acts as a reusable blueprint that defines:

- The **Character Type** asset the character will use
  - The **default name** assigned when a character is created
  - The **Layer Option** used for each layer of the character
- 

## Workflow Overview

1. Create a **Layered Character Template** asset
2. Assign a **Character Type**
3. Set the default name
4. Choose **Layer Options** for each layer
5. Attach the template to a **Layered Character Renderer** component

The template can now be reused to consistently generate the same layered character every time.

---

## Setup

### Create

To create a Layered Character Template **right click** the **Project window** and navigate to **Create > BlazerTech > Character Management System > Character Templates > Layered Character Template**.

BT Character Template 1 (Layered Character Template) Open

☐ Addressable

**References**

Character Type BlazerTech Layered Character Type

Character Name blazertech template character 1

**Optional**

Character Display Name BlazerTech Template Character 1

**Body**

Search

Options (7) Body\_01\_Peach

**Outfit**

Search

Options (29) Outfit\_05\_Dress\_01

**Hairstyle**

Search

Options (29) Hairstyle\_02\_Shortlock\_04

**Accessory**

Search

Options (30) Accessory\_03\_Straw-Hat\_01

Recreate List

## Character Type

Assign the **Character Type asset** that all characters created from this template will use.

Character Type BlazerTech Layered Character Type

This is used to determine:

- The available **layers** of the character
- The available **Layer Options** for each layer.
- The **Animator Controller** used to animate the character

## Default Character Name

Set the default name assigned when a character is created from this template.

You may also optionally set a **Display Name** for use alongside created characters in-game.

|                        |                                 |
|------------------------|---------------------------------|
| Character Name         | blazertech template character 1 |
| Optional               |                                 |
| Character Display Name | BlazerTech Template Character 1 |

## Select Layer Options

When a **Character Type** has been assigned, a list of layers appears.

|              |                           |
|--------------|---------------------------|
| Body         |                           |
| Search       |                           |
| Options (7)  | Body_01_Peach             |
| Outfit       |                           |
| Search       |                           |
| Options (29) | Outfit_05_Dress_01        |
| Hairstyle    |                           |
| Search       |                           |
| Options (29) | Hairstyle_02_Shortlock_04 |
| Accessory    |                           |
| Search       |                           |
| Options (30) | Accessory_03_Straw-Hat_01 |

Each entry respresents a layer defined in the **Character Type**.

Each layer has an assigned **Layer Option** which is the spritesheet used for that layer.

You can change the **Layer Option** used for each layer here.

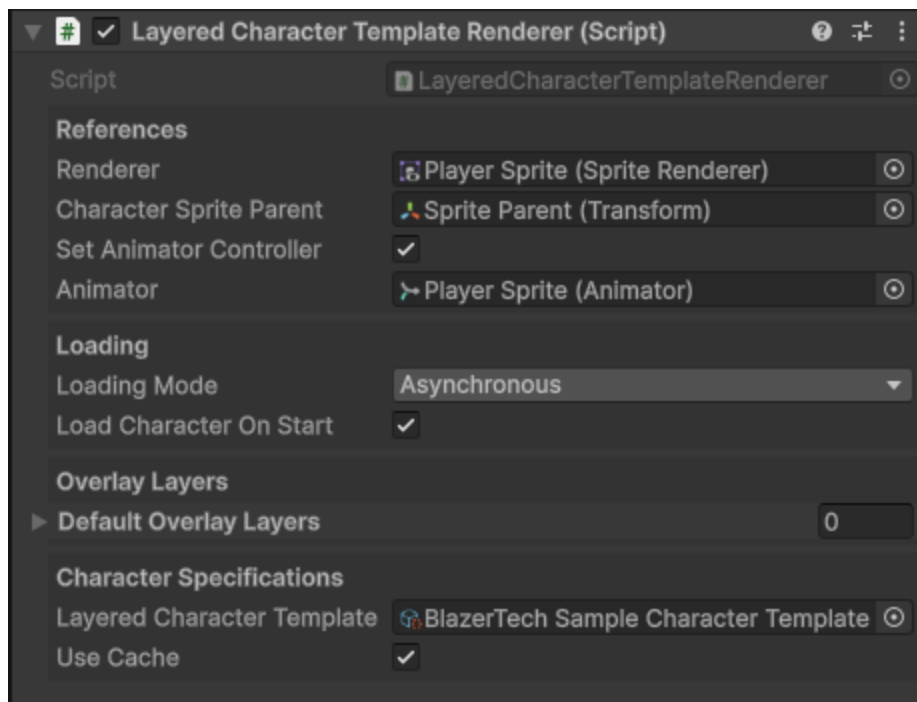
A dropdown shows all available options and the search bar can be used to narrow down results.

When a character is created from this template, all selected **Layer Options** are applied automatically.

---

## Runtime Usage

Once the **Character Template** is setup it can be assigned in a **Layered Character Template Renderer component** which at runtime will create a character from the template and animate it using the **Animator Controller** assigned in the **Character Type**.



[Read More → Layered Character Template Renderer component](#)

# Randomized Layered Character Templates

A **Randomized Layered Character Template** is a **Scriptable Object** used to create **Randomized Layered Characters** at runtime.

Unlike standard Character Templates where the output is always the same result, a **Randomized Character Template** creates a random character each time based on a set of pre-configured rules.

---

## When Should You Use this?

Use a **Randomized Layered Character Template** when:

- You need randomized NPC generation.
- You want varied background characters with set rules
- You want controlled randomness with filtering or weighting

This system gives you fine control over how each layer is selected.

---

## Workflow Overview

1. Create a **Randomized Layered Character Template**
2. Assign a **Character Type**
3. Set the default name
4. Configure Selection Mode for each layer
5. Assign the template to a **Layered Character Renderer**

The template can now procedurally generate controlled, reusable character variations.

---

## Basic Setup

### Create

To create a Randomized Layered Character Template **right click** the **Project window** and navigate to **Create > BlazerTech > Character Management System > Character Templates > Randomized Layered Character Template**.


Blazer Tech Randomized Sample Character Template (Randomized)
Open

☐ Addressable

**References**

Character Type
BlazerTech Layered Character Type (Layered)

Character Name
randomized-template-character

**Optional**

Character Display Name
Randomized Template Character

Body

Selection Mode
Any

Allow Blank
☐

Outfit

Selection Mode
Regex

Allow Blank
☐

Regex Pattern
Flannel|T-shirt

Case Sensitive
☐

Invert Match
☐

▶ Regex Matches

Hairstyle

Selection Mode
Fixed

Search

Options (29)
Hairstyle\_01\_Fine\_Cut\_01

Accessory

Selection Mode
Allow List

Blacklist
☐

▶ Layer Options (22/30)

Recreate List

## Character Type

Assign the **Character Type asset** that all characters created from this template will use.

Character Type
BlazerTech Layered Character Type (Layered)

This is used to determine:

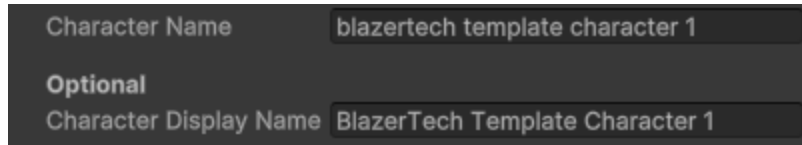
- The available **layers** of the character
- The available **Layer Options** for each layer.
- The **Animator Controller** used to animate the character

Once assigned, a configurable list of layers will appear.

## Default Character Name

Set the default name assigned when a character is created from this template.

You may also optionally set a **Display Name** for use alongside created characters in-game.

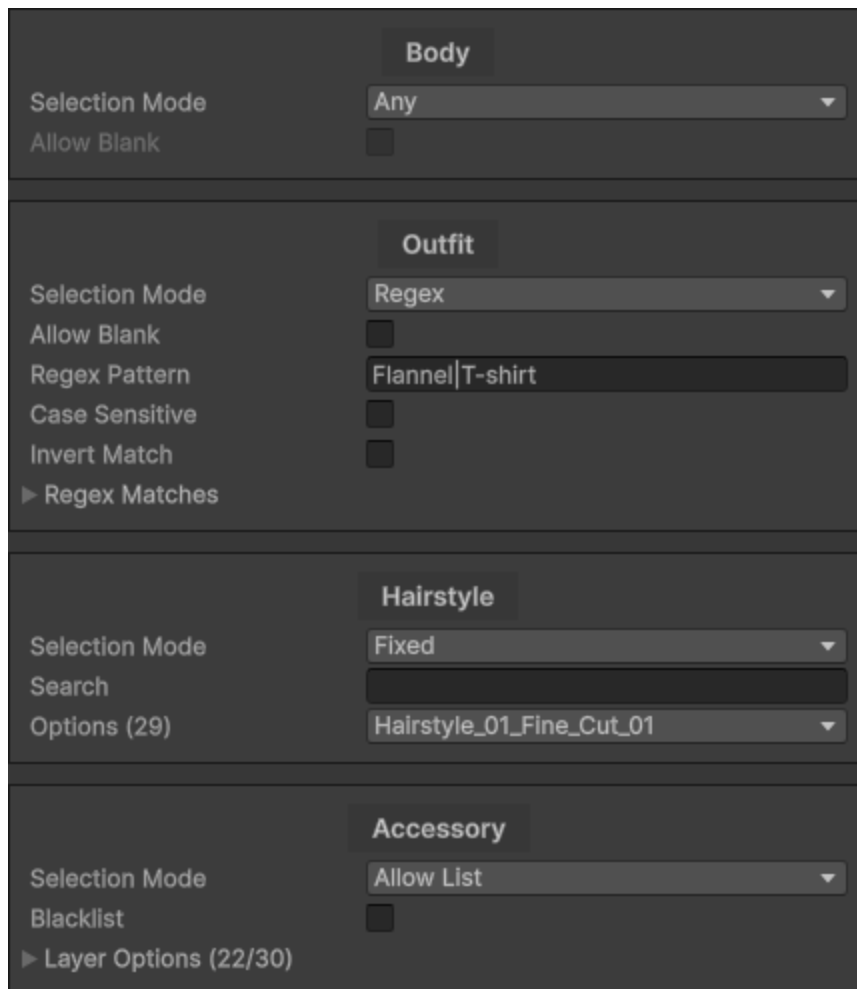
A screenshot of a settings panel for character names. It has a dark background. The first section is labeled 'Character Name' and contains a text input field with the value 'blazertech template character 1'. Below this is an 'Optional' section labeled 'Character Display Name' with a text input field containing 'BlazerTech Template Character 1'.

---

## Layer Randomization Settings

When a **Character Type** has been assigned, a list of layers appears.

Each entry represents a layer defined in the **Character Type**.

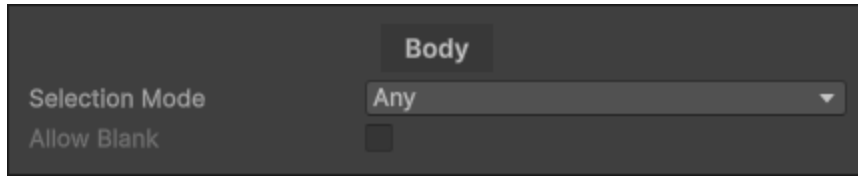
A screenshot of a settings panel for layer randomization. It has a dark background and is divided into four sections: 'Body', 'Outfit', 'Hairstyle', and 'Accessory'. Each section has a title bar at the top. The 'Body' section has 'Selection Mode' set to 'Any' and 'Allow Blank' unchecked. The 'Outfit' section has 'Selection Mode' set to 'Regex', 'Allow Blank' unchecked, 'Regex Pattern' set to 'Flannel|T-shirt', 'Case Sensitive' unchecked, 'Invert Match' unchecked, and a '► Regex Matches' button. The 'Hairstyle' section has 'Selection Mode' set to 'Fixed', an empty 'Search' field, and 'Options (29)' set to 'Hairstyle\_01\_Fine\_Cut\_01'. The 'Accessory' section has 'Selection Mode' set to 'Allow List' and 'Blacklist' unchecked, with a '► Layer Options (22/30)' button.

Each entry has a **Selection Mode** which determines how a **Layer Option** is chosen for that layer. Each layer can be configured independently.

---

## Selection Mode: Any

Selects a **Layer Option** completely at random from all available options.  
This is the default and simplest option.



**Allow Blank** determines if a blank **Layer Option** can be selected.

### *i* NOTE

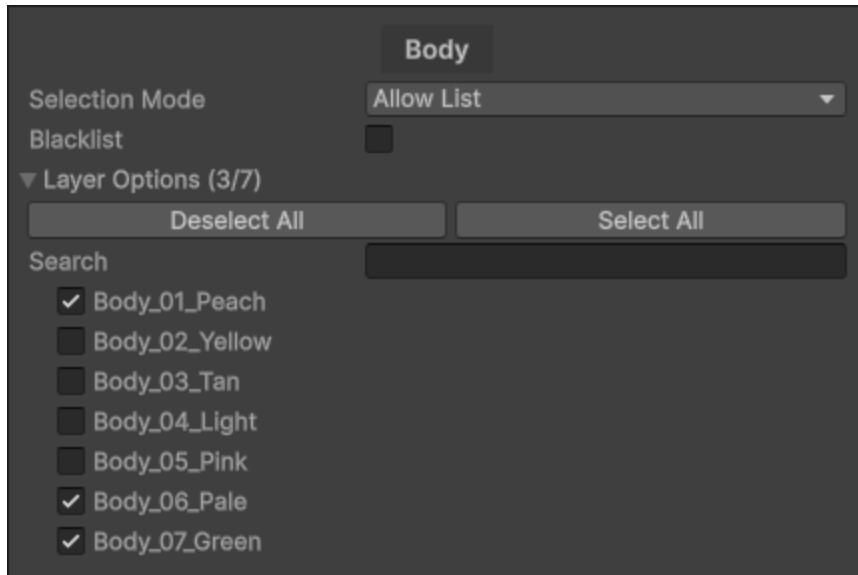
**Allow Blank** can only be enabled if the layer itself supports blank options.

**Best for:** Fully random generation with minimal setup.

---

## Selection Mode: Allow List

Selects randomly from a specific list of allowed Layer Options.



### Pros:

- Precise control over available options

### Cons:

- Can be hard to manage if **Layers Options** are change often.

## Selection Mode: Weighted Allow List

Works like the standard [Allow List](#) with the addition of a **Weight** property for each **Layer Option** selected.

The screenshot shows a user interface for the 'Weighted Allow List' selection mode. At the top, there is a tab labeled 'Body' and a dropdown menu set to 'Weighted Allow List'. Below this, there are buttons for 'Deselect All' and 'Select All'. A search bar is present. The main area lists seven options, each with a checked checkbox, a label, and a weight input field. The options are: Body\_01\_Peach (Weight: 1), Body\_02\_Yellow (Weight: 1), Body\_03\_Tan (Weight: 1), Body\_04\_Light (Weight: 1), Body\_05\_Pink (Weight: 2), Body\_06\_Pale (Weight: 1), and Body\_07\_Green (Weight: 1).

| Option         | Weight |
|----------------|--------|
| Body_01_Peach  | 1      |
| Body_02_Yellow | 1      |
| Body_03_Tan    | 1      |
| Body_04_Light  | 1      |
| Body_05_Pink   | 2      |
| Body_06_Pale   | 1      |
| Body_07_Green  | 1      |

Higher weights increase the likelihood of selection.

**Example:** If Option A has weight 2 and Option B has weight 1, Option A is twice as likely to be selected.

### Pros:

- Precise control over available options
- Adjustable probability

### Cons:

- Can be hard to manage if **Layers Options** are added/removed often.
- Requires manual balancing

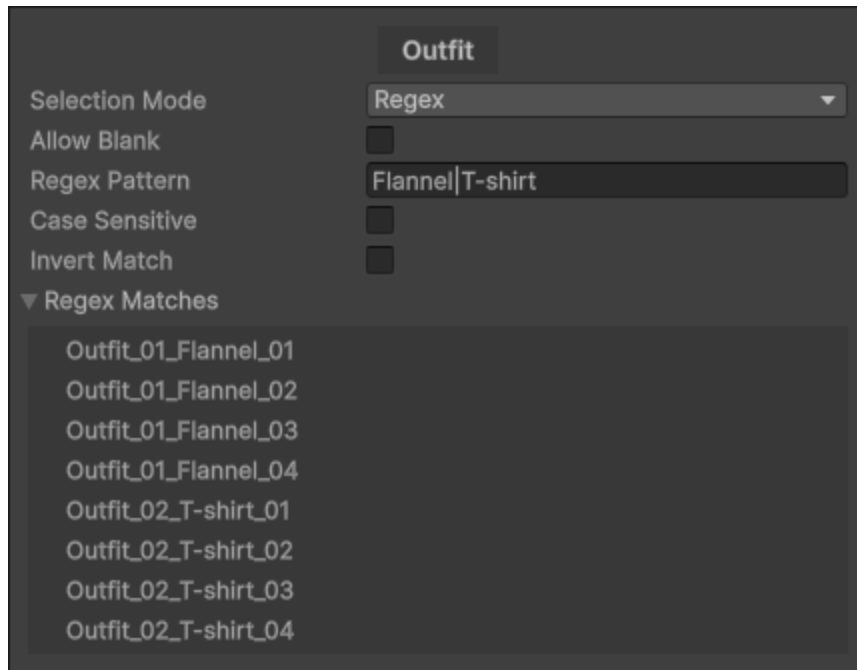
**Best for:** Common vs rare visual elements.

---

## Selection Mode: Regex

Uses a **Regular Expression** pattern to dynamically match Layer Options by name.

[Learn more about Regular Expressions here](#)↗



The regex pattern filters **Layer Options** at runtime and randomly chooses from the list.

Example pattern: **Flannel|T-shirt**

This matches any **Layer Option** containing either "Flannel" or "T-shirt".

The **Regex Matches** list shows which options currently match your pattern.

#### **NOTE**

matches are evaluated at runtime, not cached in the editor. The **Regex Matches** list is for editor visualization only.

## Additional Settings

- **Allow Blank** — Include blank as a possible result
- **Case Sensitive** — Toggle case sensitivity
- **Invert Match** — Treat the pattern as a blacklist

### Pros:

- Highly flexible
- Automatically adapts when **Layer Options** change

### Cons:

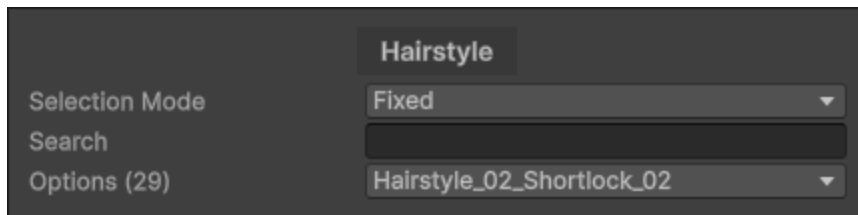
- Steep learning curve if new to Regular Expressions

**Best for:** Situations where **Layer Options** are frequently added or removed.

---

## Selection mode: Fixed

Always selects a specific Layer Option.

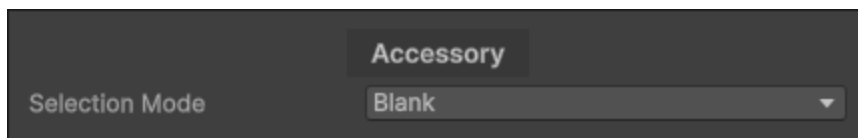


**Best for:** Layers that should never change.

---

## Selection Mode: Blank

Always selects a blank Layer Option.



### NOTE

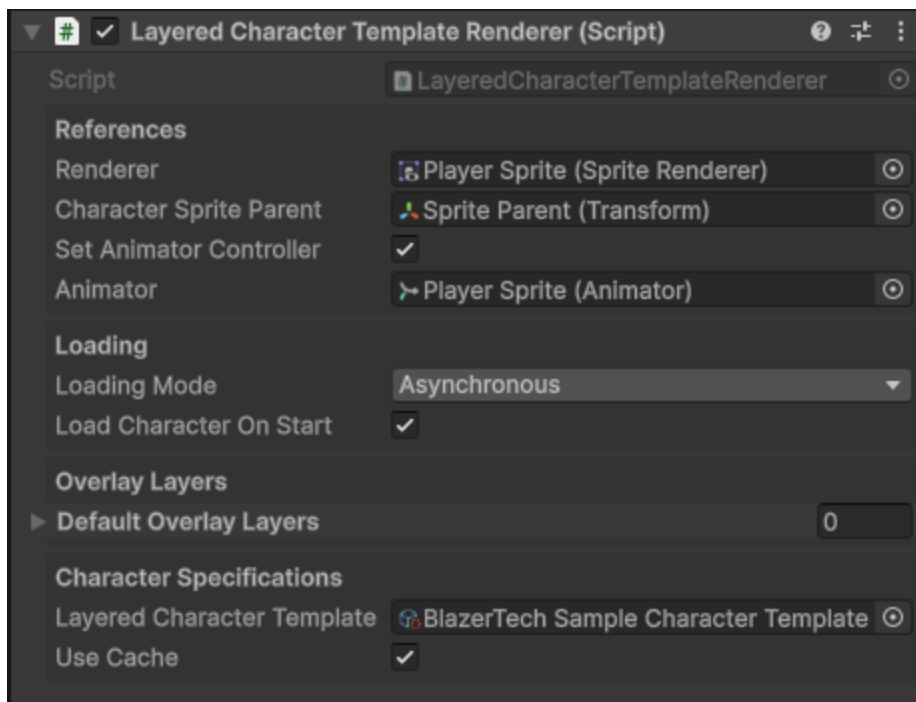
Only available if the layer is allowed to be blank. Can be toggled in the [Layer Definition Asset](#)

**Best for:** Explicitly disabling a layer.

---

## Runtime Usage

**Randomized Layered Character Templates** can be used in the same **Layered Character Renderer component** that the standard **Layered Character Templates** use.



This component will create a character from the template and animate it using the **Animator Controller** assigned in the **Character Type**.

**NOTE**

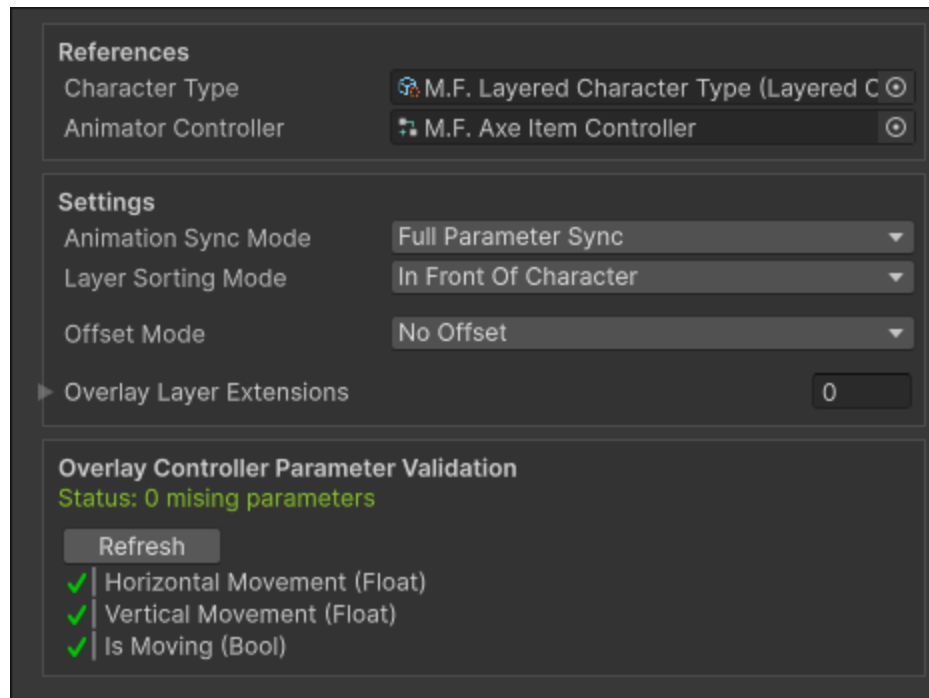
If **Use Cache** is enabled it will use the same character every time after the first character is created.  
**If you want to create a new randomized character each time, disable this setting.**

[Read More → Layered Character Template Renderer component](#)

# Character Overlay Layers

Character Overlay Layers allow you to add additional spritesheets that are layered above or below a character.

These work on both Unified and Layered Characters.



They are typically used for equipment, armor, or other optional elements for a character.

**Overlay Layers** use a completely separate **spritesheet** and **Animator Controller**.

This is the only instance where a spritesheet does **NOT** need to be the same size or follow the same layout as the **Base Spritesheet**.

**Overlay Layer Animator Controllers** can optionally have their parameters be synced to the parameters of the **Character Type Animator Controller** during runtime.

This means **Overlay Layers** can be synced to the character or be animated separately.

## When To Use Overlay Layers

Overlay Layers are useful when you need to add additional elements to a character without modifying the base character.

## Common Use Cases

| Use Case  | Example                          |
|-----------|----------------------------------|
| Equipment | Weapons, shields, any held items |

| Use Case                          | Example                                      |
|-----------------------------------|----------------------------------------------|
| Additional Clothing               | Armor, helmet                                |
| Effects                           | Auras, glow effects                          |
| Damage/Health Indicators          | Scatches, blood, burn marks                  |
| NPC Identification and Indicators | Quest markers, team colors, enemy highlights |

## How Overlay Layers Work

An **Overlay Layer** is a scriptable object that lives in your assets folder, this asset stores all info needed for an **Overlay Layer** to be used.

Every **Character Overlay Layer** contains its own:

- Spritesheet
- Animator Controller

The **Animators parameters** can be synchronized to the paramters in the **Character Type Animator Controller**.

This allow the **Overlay Layer** to be synced to the Charcter or animated on its own.

## Setup & Usage

- [Character Overlay Layer Configuration](#) - Learn how to create and setup a Character Overlay Layer
- [Character Overlay Layer Extensions](#) - Learn how to extend the functionality of a Character Overlay Layer with custom logic.
- [Character Overlay Layer Usage](#) - Learn how to apply your Character Overlay Layer to a charactar during runtime.

# Character Overlay Layer Configuration

This page explains how to create and configure a **Character Overlay Layer asset**.

Overlay Layers let you add additional visual elements to a character. They can be used for equipment, visual effects, accessories, or any other visuals that need extra functionality or need to be animated separately from the character.

Not sure what a **Character Overlay Layer** is?

[Read More → Character Overlay Layers](#)

## Quick Setup Overview

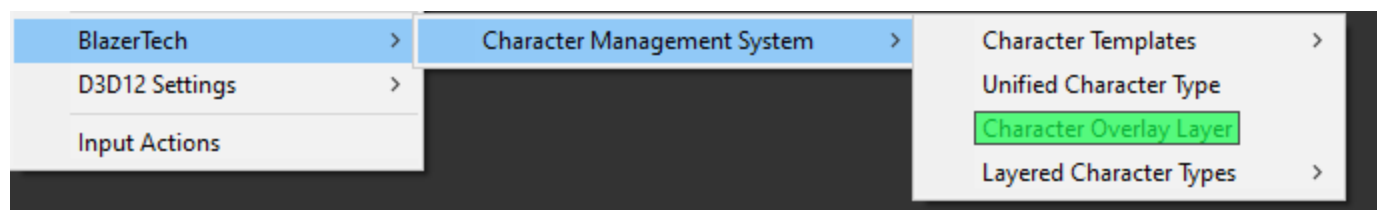
A typical workflow for configuring an overlay layer looks like this:

1. Create the **Overlay Layer asset**
2. Assign a **Character Type** asset
3. Assign an **Animator Controller** (Optional)
4. Choose the appropriate **Animation Sync Mode**
5. Set the **Layer Sorting Mode**
6. Adjust **Offsets** if the overlay does not line up correctly
7. Add any **Overlay Layer Extensions** if additional logic is required

## Creating an Overlay Layer

To create a new Overlay Layer asset:

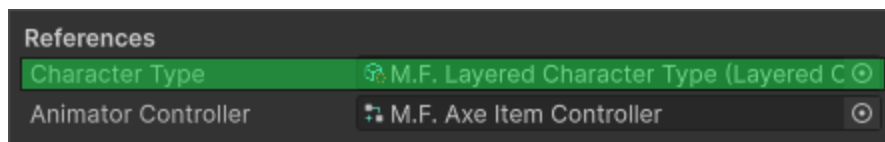
1. **Right click** the **Project window**
2. Navigate to: **Create > BlazerTech > Character Management System > Character Overlay Layers**



## Overlay Layer Setup

### Character Type

Assign the **Character Type asset** that this **Overlay Layer** will be used with.



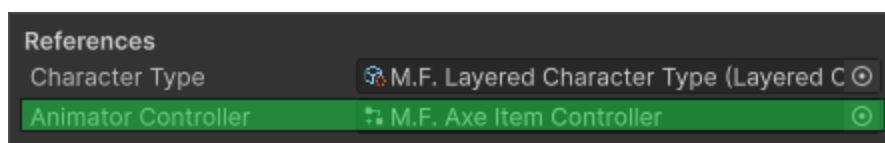
Overlay Layers are tied to a specific Character Type so the system knows which characters the Overlay Layer is compatible with.

### NOTE

Overlay Layers can only be used with their assigned Character Type.

## Animator Controller (Optional)

Assign the **Animator Controller** used to animate the **Overlay Layer**.



Overlay Layers use their own Animator Controller, separate from the controlled used by the character itself.

This controller can:

- Play its own independant animations
- Synchronize paramters with the characters Animator Controller
- Combine both approaches

How the controller behaves depends on the [Animation Sync Mode](#) selected.

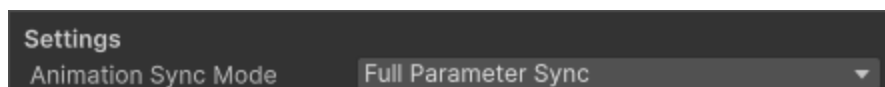
## Animation Sync Mode

Determines whether the **Overlay Layer Animator Controller** synchronizes parameters with the **Character Type Animator Controller**.

### Full Parameter Sync

All parameters in the **Overlay Layer Animator Controller** are synchronized with the parameters in the **Character Type Animator Controller**.

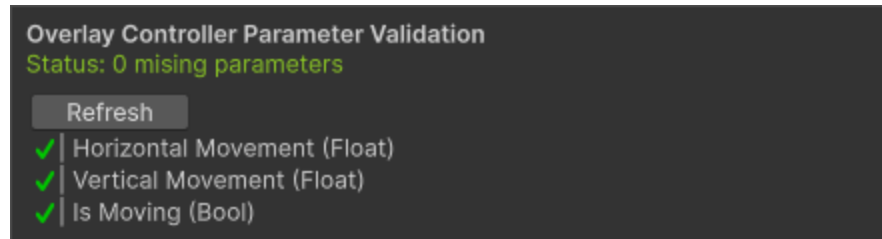
This mode assumes both controllers contain the **same parameters with identical names and types**.



When this mode is selected, a new section appears in the inspector called:

### Overlay Layer Parameter Validation

This section compares the two Animator Controllers and checks whether the Overlay Controller is missing any parameters required for synchronization.

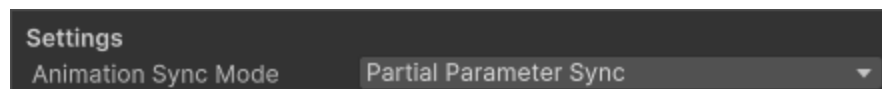


If any parameters are not included in the Overlay Controller at runtime, an error will be thrown for each missing parameter.

---

## Partial Parameter Sync

Only selected parameters are synchronized between the two Animator Controllers.



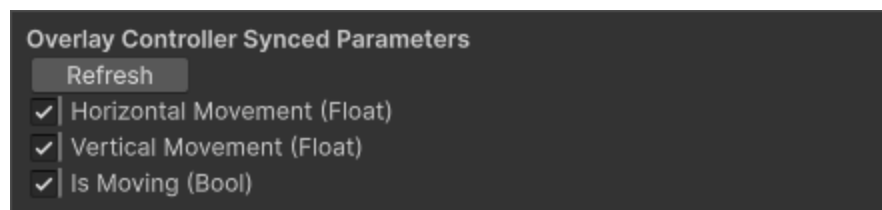
When this mode is selected, a new section appears in the inspector called:

### Overlay Controller Synced Parameters

This section lists all parameters found in the **Character Type Animator Controller**.

Parameters can be enabled if a parameter with the same **name** and **type** exist in the **Overlay Layer Animator Controller**.

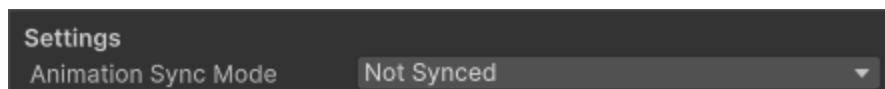
If the parameter does not exist in the **Overlay Controller**, the parameter cannot be enabled and a notice will be displayed.



---

## Not Synced

The **Overlay Layer Animator Controller** runs completely independently from the character.

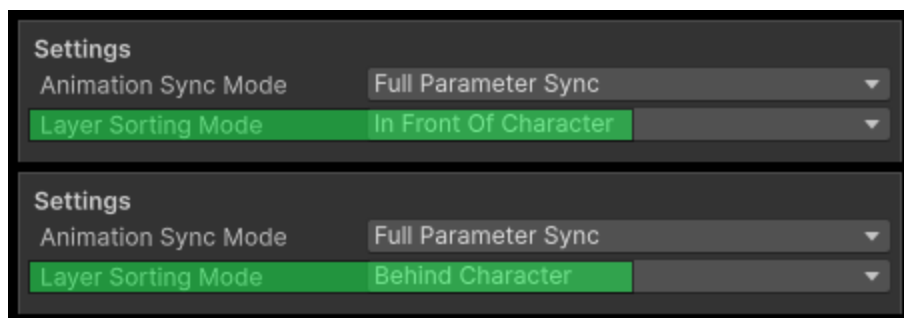


This mode is great for visual elements that need to play their own animations and don't care about the state of the character.

---

## Layer Sorting Mode

Determines how the **Overlay Layer** is rendered in relation to the Character.



### In Front of Character

The **Overlay Layer** is rendered **in front of the character**.

This is typically used for things like:

- Weapons
  - Held objects
  - Damage/Health Indicators
- 

### Behind Character

The **Overlay Layer** is rendered **behind the character**.

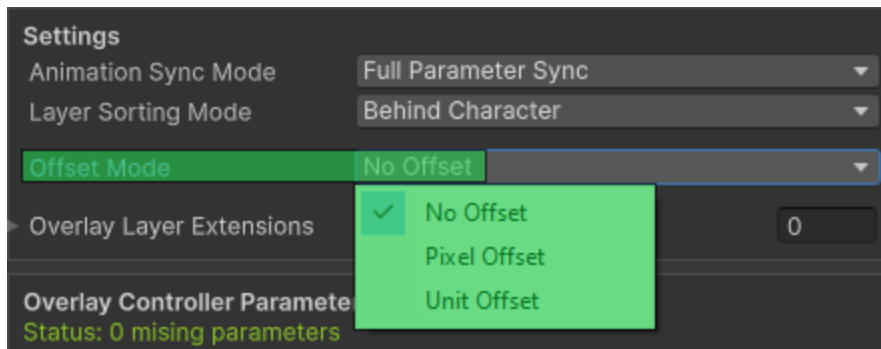
This is useful for things like:

- Back-mounted equipment
  - Capes
  - Wings
- 

## Offset Mode

Sometimes the Overlay Layer might not line up correctly with the character sprite.

Offsets allow you to manually adjust its position.



The **Offset Mode** determines how the offset values are interpreted.

1. **Pixel Offset** - Applies the offset in pixels, which are converted to **Unity Units** at runtime.
2. **Unit Offset** - Applies the offset directly in **Unity World Units**.

---

## Next Steps

- Learn how to apply an Overlay Layer to a character at runtime.  
[Read More → Character Overlay Layer Usage](#)
- Learn how to extend Overlay Layers with custom behavior  
[Read More → Character Overlay Layer Extensions](#)

# Character Overlay Layer Extensions

**Overlay Layers** can be extended to add additional functionality to them.

In the **Character Overlay Layer** asset is an **Overlay Layer Extensions** list.

Each entry in the list is a **Prefab** which must contain a script that inherits from **OverlayLayerExtension**

---

## The Overlay Layer Extension Class

**Overlay Layer Extensions** allow you to attach custom logic to a **Character Overlay Layer**.

An Overlay Layer Extension is a **Prefab containing a script** that inherits from `OverlayLayerExtension`. When an **Overlay Layer** is used, the extension is instantiated and given access to important components such as the **Sprite Renderer**, **Animator**, and **GameObject** used by the Overlay Layer.

This allows Overlay Layers to provide their own gameplay logic or interactions.

---

## How Overlay Layer Extensions Work

Each Character Overlay Layer asset contains a list of Overlay Layer Extensions.

Every entry in the list is a Prefab that:

1. Contains a script that inherits from the `OverlayLayerExtension` class
2. Implements the logic for the extension
3. Is automatically instantiated when the Overlay Layer is created at runtime.

When the Overlay Layer Extension is initialized, it receives an `OverlayLayerContext` object.

This object provides references to the key components used by the Overlay Layer.

These components are:

- `GameObject` of the Overlay Layer
  - `Sprite Renderer` Component
  - `Animator` Component
- 

## OverlayLayerContext Class

`OverlayLayerContext` is passed to every extension when it's initialized.

**It contains references to the core objects used by the Overlay Layer.**

```
public sealed class OverlayLayerContext
{
    public SpriteRenderer SpriteRenderer { get; }
    public Animator Animator { get; }
    public GameObject GameObject { get; }

    public OverlayLayerContext(GameObject gameObject, SpriteRenderer spriteRenderer,
        Animator animator)
    {
        GameObject = gameObject;
        SpriteRenderer = spriteRenderer;
        Animator = animator;
    }
}
```

## Available References

| Property              | Description                                                            |
|-----------------------|------------------------------------------------------------------------|
| <b>GameObject</b>     | The <b>GameObject</b> created for the overlay layer                    |
| <b>SpriteRenderer</b> | The <b>SpriteRenderer</b> component rendering Overlay Layer sprites    |
| <b>Animator</b>       | The <b>Animator</b> component controlling the Overlay layer animations |

These references allow Overlay Layer Extensions to full intereact with the **Overlay Layer**.

Examples:

- **Trigger animations**
- **Change sprites or materials**
- **Enable or disable the layer**
- **Add additional components**

---

## Setup Guide

### Create the Extension Script

Create a new script that inherits from `OverlayLayerExtension`.

**Example:**

```

using BlazerTech.CharacterManagement.Core;
using UnityEngine;

public class ExampleOverlayLayerExtension : OverlayLayerExtension
{
    public override void Initialize(OverlayLayerContext context)
    {
        base.Initialize(context);

        Debug.Log("Initialized!");
    }
}

```

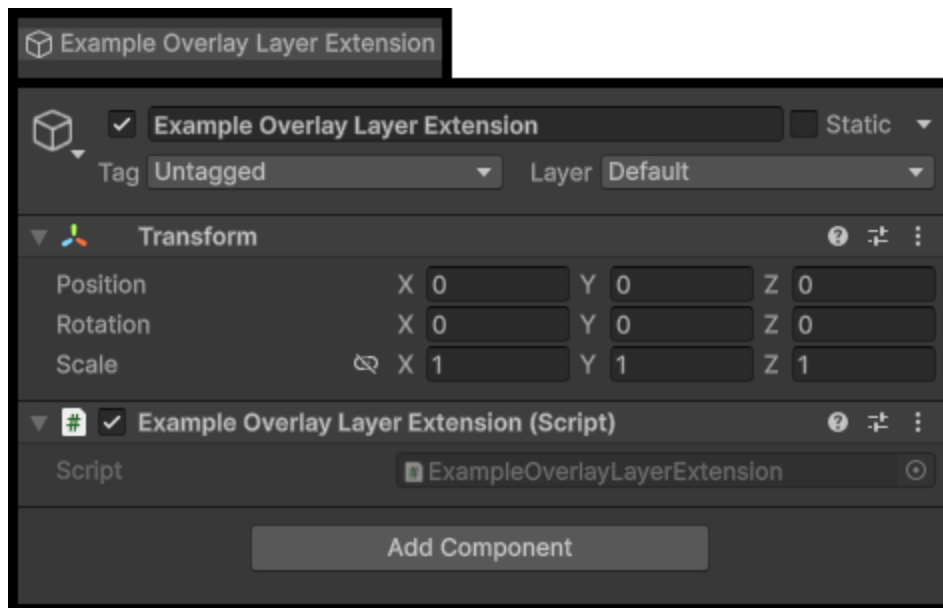
The `OverlayLayerContext` class gives you access to the **Overlay Layer components**.

---

## Create the Extension Prefab

Now we need to create a new GameObject prefab that contains our new Extension script.

1. Create a new **GameObject** in your scene and name it.
2. Attach your **Overlay Layer Extension script**.
3. Configure any variable on your component.
4. Drag the GameObject into your **Project window** to create a **Prefab**.



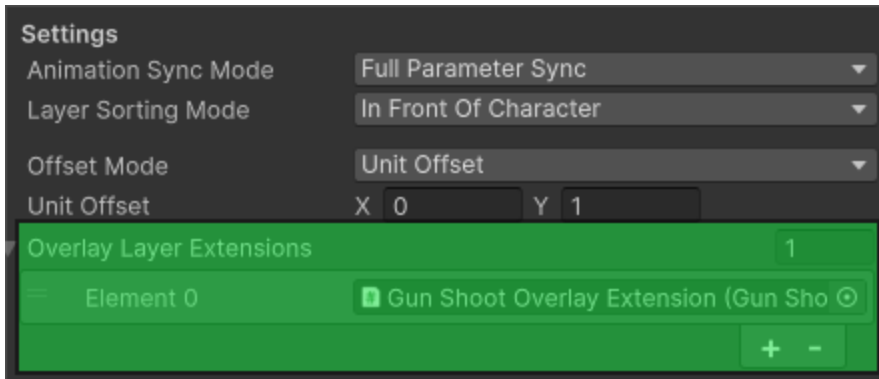
This prefab will be instantiated when the Overlay Layer is created during runtime.

---

## Add the Extension Prefab to the Overlay Layer

Lets attach the **Overlay Layer Extension prefab** we just created to the **Character Overlay Layer asset**.

1. Select your **Character Overlay Layer asset**.
2. Locate the **Overlay Layer Extensions** list.
3. Add a new element to the list.
4. Assign your **Extension Prefab**.



When the **Overlay Layer** is used during runtime, all **Overlay Layer Extensions** will be instantiated in the order listed as **children of the Overlay Layer** and initialized.

## Overlay Layer Extension Example

The follow example is used in the Modern Interiors Integration package.

It simulates a character firing a weapon.

The Overlay Layer only contains the characters hands, allowing the shooting animation to play independently from the main character body.

When the **E key** is pressed, the extension triggers a **Shoot** animation on the **Overlay Layer**.

```
using BlazerTech.CharacterManagement.Core;
using UnityEngine.InputSystem;

public class GunShootOLE : OverlayLayerExtension
{
    private void Update()
    {
        if (overlayLayerContext == null) return;

        if (Keyboard.current.eKey.wasPressedThisFrame)
        {
            overlayLayerContext.Animator.SetTrigger("Shoot");
        }
    }
}
```

```
}  
}
```

In this example:

1. The extension checks input every frame.
  2. When **E** is pressed, it activates a trigger titled **Shoot**.
  3. Only the Overlay Layer animates, letting the rest of the body animate separately.
- 

## Related

- [Character Overlay Layers](#)
- [Character Overlay Layer Configuration](#)
- [Character Overlay Layer Usage](#)

# Character Overlay Layer Usage

This page explains how to apply **Overlay Layers** to a character using a [Character Renderer Component](#).

Overlay Layers can be applied in two ways:

1. Automatically when the character loads
2. Dynamically at runtime through code

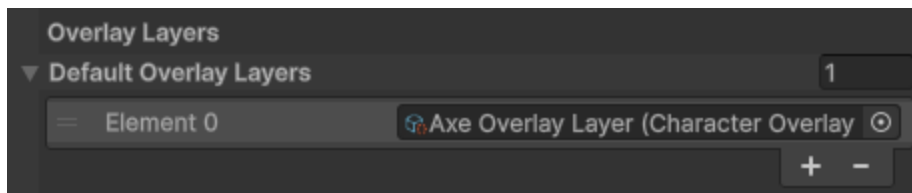
---

## Applying Overlay Layers Automatically

### Default Overlay Layers

Every **Character Renderer component** contains a **Default Overlay Layers** list.

Any Overlay Layer in this list is automatically applied when the character is loaded.



This is useful when a character should always spawn with specific visual elements.

**Overlay Layers are applied in the order listed.**

#### ⊗ CAUTION

**Overlay Layers** must use the **same Character Type** as the character they are applied to. If the **Character Types** do not match, the **Overlay Layer** will not be applied.

---

## Managing Overlay Layers Through Code

Overlay Layers can also be **added, removed, or reordered at runtime**.

All functionality is provided through the **OverlayLayerHandler**, which exists on every **Character Renderer component**.

This handler manages all overlay layers currently applied to the character.

---

## Accessing the OverlayLayerHandler

Every **Character Renderer component** exposes the handler through the `OverlayLayerHandler` property.

Example reference:

```
CharacterRenderer.OverlayLayerHandler
```

Once accessed, the handler can be used to manager **Overlay Layers**.

---

## Adding an Overlay Layer

Adds a new Overlay Layer to the character.

```
AddOverlayLayer(CharacterOverlayLayerSO overlayLayer)
```

### Behavior

- Instantiates the Overlay Layer
- Adds it to the end of list of active Overlay Layers
- Initializes the Overlay Layer

If the same Overlay Layer is added multiple times, **duplicates will be created**.

### Example

```
characterRenderer.OverlayLayerHandler.AddOverlayLayer(overlayLayer);
```

---

## Removing an Overlay Layer

Removes an overlay Layer from a character.

```
RemoveOverlayLayer(CharacterOverlayLayerSO overlayLayer)
```

### Behavior

- Searches for the first instance of the Overlay Layer
- Removes it if found

If multiple instances of the same overlay layer exist, **only the first instance is removed**.

### Example

```
characterRenderer.OverlayLayerHandler.RemoveOverlayLayer(overlayLayer);
```

---

## Changing Overlay Layer Order

Overlay Layers are rendered based on their GameObject position in the hierarchy

You can change the position of an Overlay Layer using:

```
ChangeOverlayLayerIndex(CharacterOverlayLayerSO overlayLayer, int newIndex)
```

## Behavior

- Finds the Overlay Layer
- Moves it to the specified index
- Updates the GameObjects position in the hierarchy

## Example

```
characterRenderer.OverlayLayerHandler.ChangeOverlayLayerIndex(overlayLayer, 1);
```

If the index is outside the valid range, the value is **clamped automatically**.

---

## Example Script

The following example demonstrates adding an **Overlay Layer** and moving it to a specific position.

```
using BlazerTech.CharacterManagement.Components;
using UnityEngine;
using UnityEngine.InputSystem;

public class AddOverlayLayerOnPress : MonoBehaviour
{
    [SerializeField] CharacterRendererBase characterRenderer;

    [SerializeField] CharacterOverlayLayerSO overlayLayer;

    void Update()
    {
        if (Keyboard.current.digit1Key.wasPressedThisFrame)
        {
            characterRenderer.OverlayLayerHandler.AddOverlayLayer(overlayLayer);
            characterRenderer.OverlayLayerHandler.ChangeOverlayLayerIndex(overlayLayer, 1);
        }
    }
}
```

```
}  
  }  
}
```

---

## Related

- [Character Overlay Layers](#)
- [Character Overlay Layer Configuration](#)
- [Character Overlay Layer Extensions](#)

# Built-In Characters



## BlazerTech Modular Characters

Sample modular characters are included within the **BlazerTech Character Management System** and are available to be used in any commercial or non-commercial project. These characters were made to match the style of the [Modern Series](#) created by [LimeZu](#).

These characters are modular and consist of 4 layers:

1. **Body (7 Options)**
2. **Outfit (28 Options)**
3. **Hairstyle (28 Options)**
4. **Accessory (29 Options)**

A **Layered Character Type** and **Unified Character Type** are pre-setup in the **/BlazerTech Modular Characters** subfolder. The **Unified Character Type** contains a few premade characters created by combining random options of each of the 4 layers.

The **BlazerTech Modular Characters** were created by **Jammie**. Check out more of his work [here](#)!

### TIP

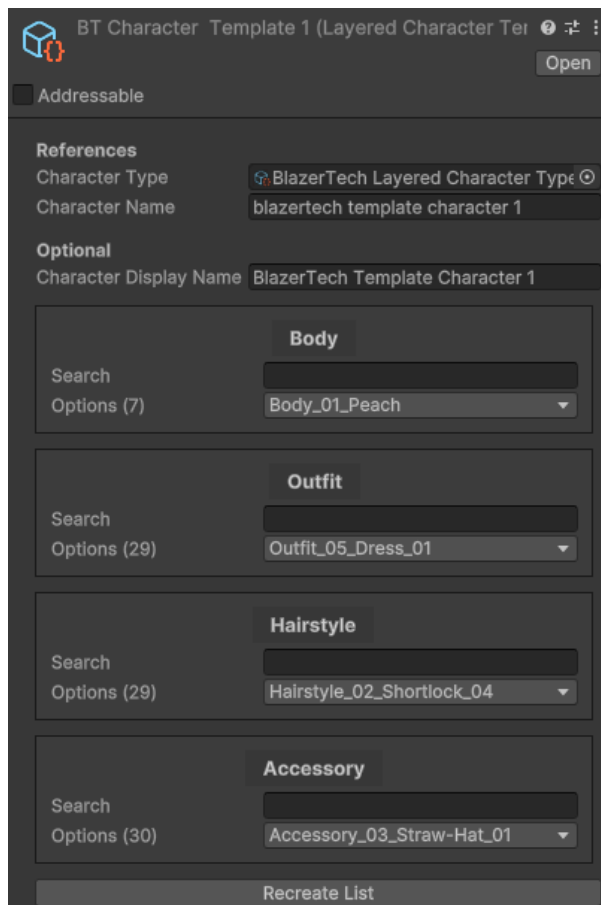
Upon the full release of the **BlazerTech Character Management System** these characters will also be available for purchase separately at a discounted price.

# Character Usage

The easiest way to create and use a **BlazerTech character** is to create a **character template**. It's basically a blueprint for the character which we can then use to create the character later during runtime.

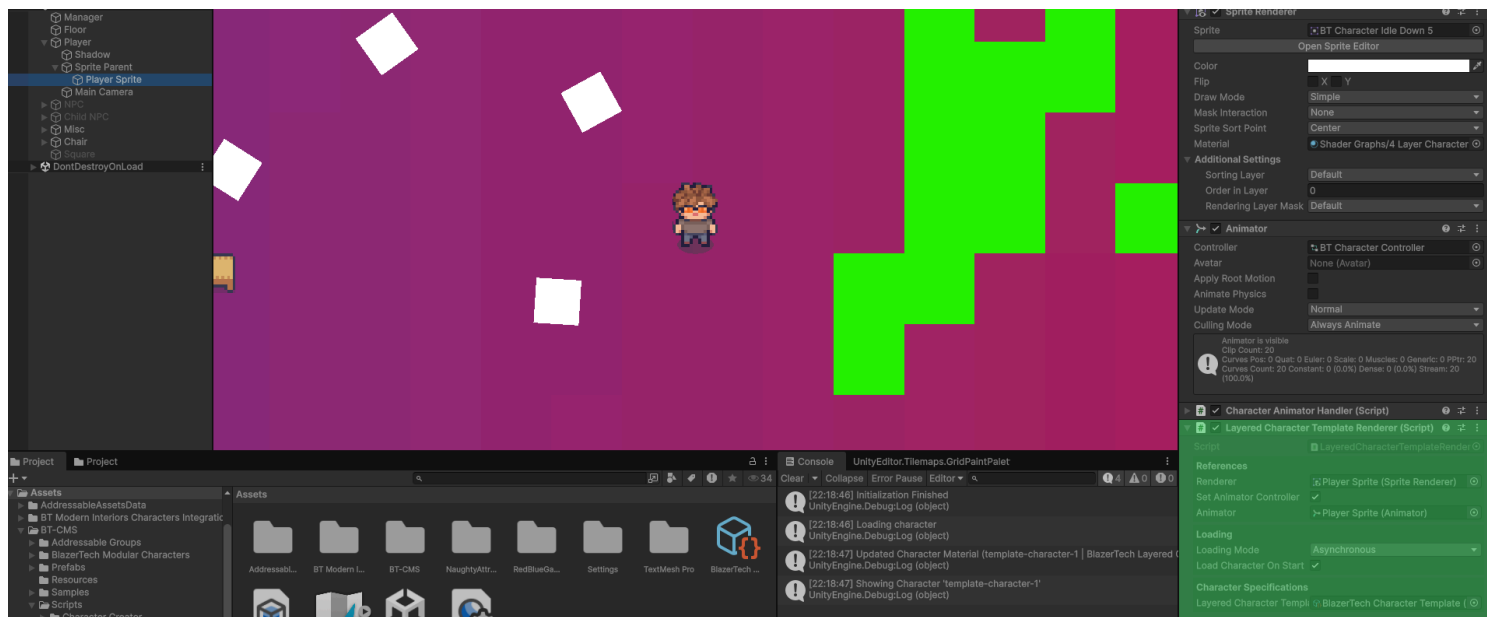
To create a character template **right click** the **Project window** and navigate to **Create > BlazerTech > Character Management System > Character Templates > Layered Character Template**.

Once created, assign the **BlazerTech Layered Character Type** to the **Character Type** field. A list will appear with 4 entries, one for each layer. Assign the option you want to use for each layer of the character.



When you want to use your template add the [Layered Character Template Renderer](#) component to a GameObject and assign a reference to the **Layered Character Template** you just created.

Then enter play mode and you'll see the character you created from the template.



## Related

- [Character Templates](#)

# LimeZu Modern Interiors Support

Don't know who LimeZu is? Check him out [here](#)! He's an insanely talented and dedicated man.

Fun fact, the **BlazerTech Character Management System** was originally planned to be a support tool for **Modern Interiors**. Then I got bigger ideas for it and it branched off into it's own thing.

However because of that the Character Management System provides an easy implementation for the modular characters within **Modern Interiors** and **Modern Farm**.

LimeZus characters are modular, meaning they're built of multiple layers that when put on top of each other form a finished character. This is exactly how Layered Characters work in the Character Management System.



Follow the guide for setting up a Layered Character Type [here](#) to learn how to use LimeZus characters with the Character Management System

# Character Creator Overview

The **Character Creator** is a modular, prefab based framework for building **Character Creation Menus**.

A Character Creation Menu allows players to create and customize characters for use in your game. For example, it can be used to create a player character or customize the appearance of an NPC.

The Character Creator is designed to be flexible. Each part of the menu is provided as a separate module, allowing you to combine, configure, and customize the modules to create the menu you want.

## NOTE

The Character Creator is designed to work for **Layered Characters** only. Its customization features work by modifying the individual layers that make up a Layered Character.

## Character Creator Modules

The Character Creator is **modular**, it's split into a bunch of smaller pieces we call modules, each provides a specific piece of functionality. When these modules are put together they form a complete and functioning **Character Creation Menu**.

| Module                                 | Description                                                                                 |
|----------------------------------------|---------------------------------------------------------------------------------------------|
| <a href="#">Layer Selectors</a>        | Allow the player to change the visual elements for each character layer.                    |
| <a href="#">Character Preview</a>      | Displays an animated or static preview of the character that updates when changes are made. |
| <a href="#">Menu Controls</a>          | Essential controls for the menu such as Save, Back and Reset.                               |
| <a href="#">Character History</a>      | Records changes and allows players to revert to previous versions of the character.         |
| <a href="#">Randomization Controls</a> | Provides options for randomizing the entire character or individual layers.                 |
| <a href="#">Loading Screens</a>        | Hides the menu while it's being initialized and can display loading progress.               |
| <a href="#">Character Display Name</a> | Allows the player to enter a display name for their character.                              |

# Layer Selector Module

A **Layer Selector** is a UI element that allows the playerto change a specific **layer** of a [Layered Character](#).

The following selector types are included.

| Selector Type                     | Description                                                                                                     |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <a href="#">Dropdown Selector</a> | Displays the available options in a dropdown list.                                                              |
| <a href="#">Carousel Selector</a> | Displays the current option and allows the player to cycle through other options using left and right controls. |
| <a href="#">Grid Selector</a>     | Displays layer options as a grid of preview thumbnails.                                                         |
| <a href="#">List Selector</a>     | Displays layer options in a vertical or horizontal list, optionally with preview images.                        |
| <a href="#">Tab Selector</a>      | Works alongside another selector and allows the player to switch which layer that selector controls.            |



[Read More → Layer Selector Setup](#)

# Character Preview Module

The **Character Preview** displays the character currently being customized.

Character previews can be configured with several options:

- **Static Sprite Preview** – Displays the **preview sprite** defined by the Character Type.
- **Animated Preview** – Uses an Animator Controller to play character animations.
- **Animation Swapping** – Allows additional animations defined by the Character Type to be selected from the menu.
- **Rotate Character** – Adds controls that allow the player to view the character from different directions.



[Read More → Character Preview](#)

## Menu Controls Module

**Menu Controls** provide the buttons and controls used to interact with the Character Creation Menu, along with some additional niceties.

These controls include:

- **Back/Close Menu button:** Closes the menu.
- **Save button:** Saves the character's changes.
- **Reset button:** Resets the character.
- **Randomize button:** Randomizes all layers of the character.



[Read More → Menu Controls](#)

## Character History Module

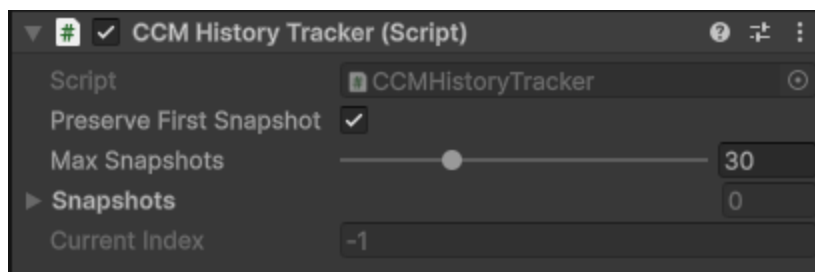
The **Character History System** records changes made while customizing a character.

The **CCM History Tracker** component stores a **snapshot** of the character whenever a change is made. These snapshots can then be used to undo changes or return to an earlier version of the character.

## History Tracker

The History Tracker provides the following settings:

- **Snapshot Limit:** Determines how many snapshots can be stored. The limit can be set between **1 and 100**, with a default of **30**.
- **Preserve First Snapshot:** Prevents the first snapshot from being overwritten when the snapshot limit is reached.

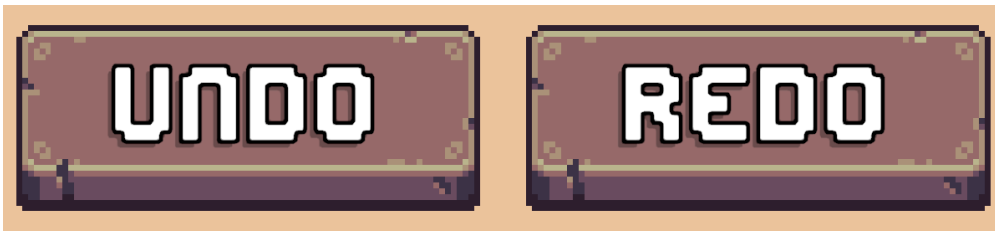


[Read More → History Tracking System](#)

## Undo & Redo

The **Undo & Redo** functionality allows players to move backward and forward through their character's history.

Use the **CCM Timeline Button Handler** component on a button to connect it to a **History Tracker**.



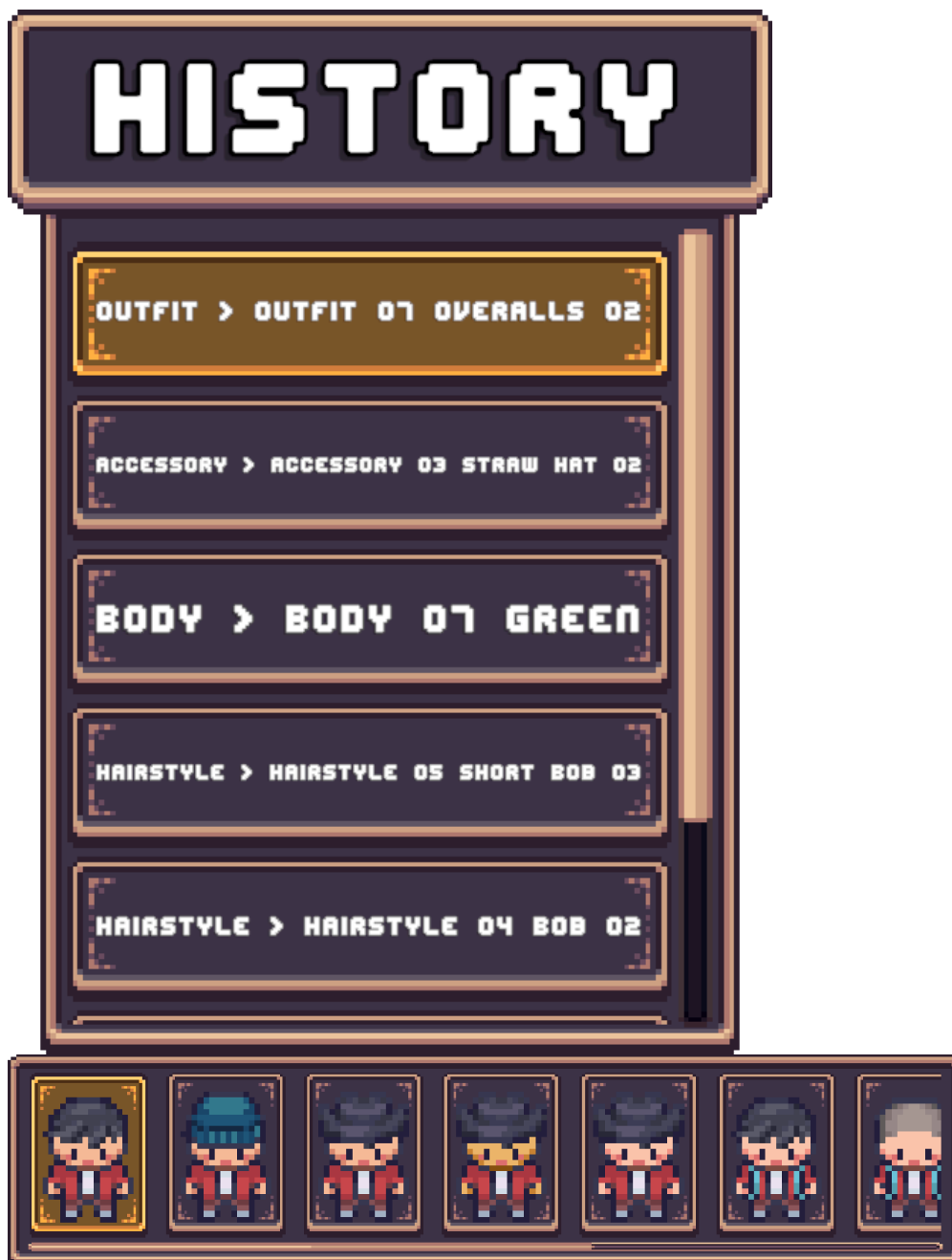
[Read More → History Undo/Redo](#)

## History Panels

A **History Panel** displays the snapshots recorded by the History Tracker. Players can select an entry to revert the character to that version.

The Character Creator supports several presentation styles:

| Panel Type          | Description                                                      |
|---------------------|------------------------------------------------------------------|
| <b>Text Based</b>   | Displays text describing what changed in each snapshot.          |
| <b>Sprite Based</b> | Displays a preview image of the character for each snapshot.     |
| <b>Hybrid</b>       | Displays both a description and preview image for each snapshot. |



[Read More → History Panels](#)

## Randomization Controls Module

Character randomization functionality can be added in multiple different ways

- **Randomize Button** - Randomizes all layers of the character.
- **Controlled Randomization** - Allows the player to choose which layers should be randomized.
- **Layer-Specific Randomization** - Allows individual layers to be randomized directly from [Layer Selectors](#).



[Read More → Character Randomization](#)

## Loading Screen Module

Loading screens hide the **Character Creation Menu** while it's being setup.

Loading Screens are modular, allowing additional elements such as loading bars and progress text to be added or removed as needed.



[Read More → Loading\\_Screen](#)

## Character Display Name Module

The Character Display Name module allows the player to give their character a display name.

The display name can later be displayed anywhere in your game, such as above the character's head.



[Read More → Character Display Name](#)

# Character Creator Setup

The **Character Creator** provides a modular system for creating an in-game **Character Creation Menu**.

This page walks through the different ways to set up a Character Creation Menu, from using a premade menu to building your own.

[Read Also → Character Creator Overview](#)

## Choosing a Setup Method

There are two ways to setup a Character Creation Menu:

| Option               | Best For                                                        |
|----------------------|-----------------------------------------------------------------|
| Use a Premade Menu   | Getting a complete Character Creation Menu working quickly      |
| Create Your Own Menu | Building a custom interface using the Character Creator modules |

### TIP

If you're new to the Character Creator, start with a **Premade Character Creation Menu**. You can customize or replace individual modules later.

- [Use a Premade Character Creation Menu](#)
- [Create Your Own Character Creation Menu](#)

## The Character Creation Menu Manager

Every Character Creation Menu is managed by a `CharacterCreationMenuManager`.

The manager controls when the menu is opened and closed and provides the information required to initialize it.

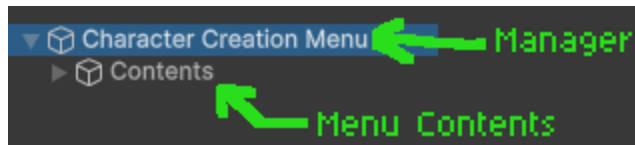
## Setting Up the Manager

Create a `GameObject` for the `CharacterCreationMenuManager` and keep it **enabled at runtime**.

The manager requires two fields:

| Field                         | Description                                                                                                         |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Default Character Type</b> | The <code>LayeredCharacterTypeSO</code> used when the menu is opened. This can be overridden when opening the menu. |
| <b>Menu Contents</b>          | The <code>GameObject</code> containing the actual Character Creation Menu.                                          |

A typical menu is structured like this:



The **Menu Manager** should exist separately from the **Menu Contents**.

### ⊗ IMPORTANT

The Menu Manager should remain enabled at runtime. The Menu Contents `GameObject` can be enabled or disabled.

## Opening the Menu

The `CharacterCreationMenuManager` provides four methods for opening the Character Creation Menu.

Most of these methods use the **Character Grouping System** to determine which character is being created or edited.

[Learn More → Character Grouping System](#)

| Method                                                     | Purpose                                                                                       |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <a href="#">EnableMenu_EditOrCreatePrimaryCharacter</a>    | Opens the menu using the Primary Character Slot for the specified Character Type.             |
| <a href="#">EnableMenu_EditCharacter</a>                   | Opens the menu to edit an existing Layered Character.                                         |
| <a href="#">EnableMenu_CreateOrReplacePrimaryCharacter</a> | Opens the menu with a new character that overrides the Primary Character Slot.                |
| <a href="#">EnableMenu_CreateCharacterInFlexibleGroup</a>  | Opens the menu with a new character and places it in the specified Flexible Group when saved. |

## Menu Events

The `CharacterCreationMenuManager` provides two Unity Events that can be used to connect the Character Creator to the rest of your game.

### On Character Saved

Called when the character in the Character Creation Menu is saved.

The saved character is provided as an event argument.

For example, you can use this event to close the Character Creation Menu by calling `CharacterCreationMenuManager.DisableMenu`.

### On Menu Disabled

Called after the Character Creation Menu has been disabled and closed.

Use this event to open another menu or perform other actions after the Character Creation Menu closes.

---

## Premade Character Creation Menus

Premade Character Creation Menus provide a complete starting point with the modules needed for a functional character creator.

At minimum, a premade menu contains:

| Component           | Purpose                                                |
|---------------------|--------------------------------------------------------|
| Layer Selectors     | Allow the player to change the character's appearance. |
| Character Preview   | Shows the character while it's being edited.           |
| Loading Screen      | Covers the menu while it's being initialized.          |
| Save & Back Buttons | Allow the player to save changes and leave the page.   |

## Premade Menu Prefabs Location

Premade Character Creation Menus are located at:

[Prefabs](#) > [Character Creator](#) > [Premade Menus](#)

Four premade menus are included.

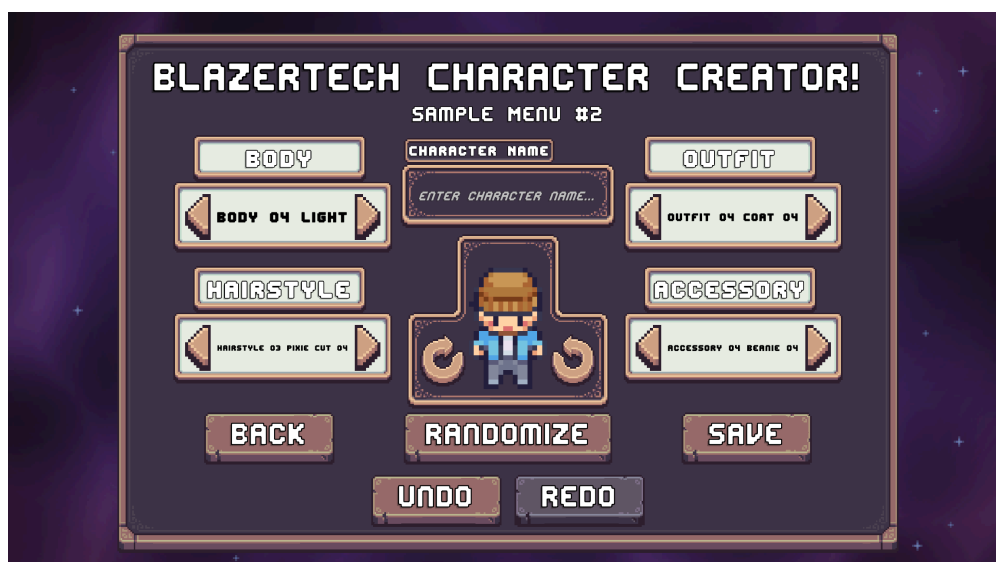
## Premade Character Creation Menu #1

- **Dropdown Selectors [Initialize Existing]** with four dropdown selectors
- **Character Preview** with rotation and animation controls
- **Controlled Randomization**
- **Save and Back Controls**
- **History Panel (Horizontal)**
- **Loading Screen #1**



## Premade Character Creation Menu #2

- **Carousel Selectors [Initialize Existing]** with four carousel selectors
- **Character Preview** with rotation controls
- **Save & Back Controls**
- **Randomize Entire Character** button
- **Character Display Name Field**
- **Undo/Redo Controls**
- **Loading Screen #2**



## Premade Character Creation Menu #3

- **Tab Selectors [Initialize Existing]** with four tab selectors
- **Grid Layer Selector**
- **Character Preview**
- **Save and Back Controls**
- **Loading Screen #3**

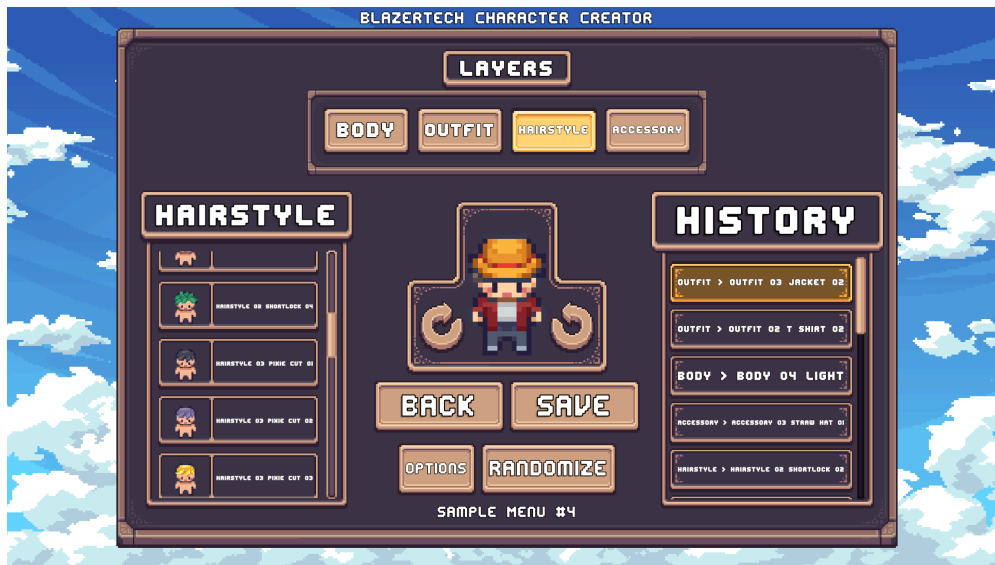
The selected tab determines which layer is displayed by the Grid Layer Selector.



## Premade Character Creation Menu #4

- **Tab Selectors [Initialize Existing]** with four tab selectors
- **List Layer Selector**
- **Character Preview** with rotation controls
- **Save and Back Controls**
- **History Panel (Vertical)**
- **Controlled Randomization**
- **Loading Screen #4**

The selected tab determines which layer is displayed by the List Layer Selector.



## Creating Your Own Character Creation Menu

A custom Character Creation Menu consists of two main GameObjects:

1. **Menu Manager**

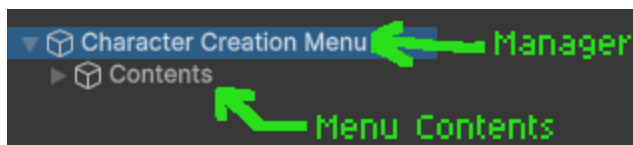
Contains the `CharacterCreationMenuManager` component and controls the menu.

2. **Menu Contents**

Contains the actual UI and Character Creator components.

The **Menu Contents** GameObject is assigned to the **Menu Contents** field on the `CharacterCreationMenuManager`.

The two GameObjects should be separate, as shown below:



### ⊗ IMPORTANT

The Menu Manager should remain enabled at runtime. The Menu Contents GameObject can be enabled or disabled.

Once the manager is set up, the next step is to add the components that make up the menu.

## Character Creation Menu Contents

The Character Creator is modular. Each part of the Character Creation Menu is provided as a separate prefab/module.

Most modules can simply be added as children of the **Menu Contents** GameObject without additional setup.

The following sections separate the modules into **Essentials** and **Optional Features**.

## Essentials

The following modules provide the basic functionality needed for a usable Character Creation Menu.

### 1. Layer Selectors

**Layer Selectors** allow the player to change the selected option for each character layer.

Layer Selector prefabs are located at:

**Prefabs > Character Creator > Modules > Layer Selectors**

Choose the type of selector you want to use, then use a prefab from its **/Pre-Setup** folder. These prefabs contain a set of already-configured Layer Selectors.

[Learn More → Layer Selectors](#)

### 2. Character Preview

The **Character Preview** displays a live preview of the character while it's being edited.

Character Preview prefabs are located at:

**Prefabs > Character Creator > Modules > Character Preview**

Use any prefab inside the **/Pre-Setup** folder.

Three prefabs are available, a base version and two others with additional features.

[Learn More → Character Previews](#)

### 3. Menu Controls

At minimum, the menu should provide:

- **Back:** Closes the menu.
- **Save Character:** Saves the character's changes.

These buttons can be created manually or added using the included prefabs.

### Creating a Button Manually

To create a button manually:

1. Add a Unity **Button** to the Character Creation Menu.
2. Add a **CCMRelay** component to the button's GameObject.
3. Add an **On Click Event** to the button.
4. Call one of the following methods:
  - **CCMRelay.DisableMenu()** for the Back button.
  - **CCMRelay.SaveCharacter()** for the Save Character button.

Menu Control prefabs are also available at:

**Prefabs > Character Creator > Modules > Menu Controls**

Additional menu controls can be added as needed.

[Learn More → Menu Controls](#)

## 4. Loading Screen

A **Loading Screen** covers the Character Creation Menu while it's being initialized.

Loading Screen prefabs are located at:

**Prefabs > Character Creator > Modules > Loading Screen**

There are three folders for the loading screen module:

- **Loading Screen Core** provides a basic black loading screen.
- **Loading Screen Additions** are prefabs that can be added as children of the Loading Screen Core to add additional functionality.
- **Pre-Setup Loading Screens** provide pre-configured loading screens with additional features such as loading bars and text.

Addon components require a reference to the **Loading Screen Handler** found on the Loading Screen Core.

The folders are organized as follows:

```
Loading Screen
├── Core
├── Additions
└── Pre-Setup
```

[Learn More → Loading Screens](#)

# Optional Features

Once the essential modules are working, additional features can be added to your Character Creation Menu.

## Character Randomization

Character Randomization allows the player to quickly generate a character without manually selecting every layer.

There are several ways to add randomization:

- **Randomize Entire Character** randomizes all layers.
- **Controlled Randomization** allows individual layers to be enabled or disabled before randomizing.
- Some Layer Selector variants include a button for randomizing an individual layer.

Randomization prefabs are located at:

**Prefabs > Character Creator > Modules > Randomization**

A simple **Randomize Entire Character** button can also be created using a **Button** and **CCMRelay**, in the same way as other Menu Controls.

[Learn More → Character Randomization](#)

## Character History

The **History Tracking System** records changes made while editing a character.

To enable history tracking, add the **CCMHistoryTracker** component to a GameObject inside the Character Creation Menu Contents.

This is required for all modules part of the History Tracking System.

History prefabs are located at:

**Prefabs > Character Creator > Modules > History**

The included history prefabs are divided into two categories:

### Undo/Redo

The **/Undo-Redo** folder contains prefabs with buttons that allow the player to undo and redo character changes.

### History Panels

The **/History Panels** folder contains panels that display the character's edit history.

Two panel types are included:

- **Vertical History Panel:** Displays text describing each change.
- **Horizontal History Panel:** Displays a character preview for each change.

Selecting an entry in a History Panel reverts the character to that state.

[Learn More → History Tracking System](#)

## Character Display Name

A **Character Display Name Field** allows the player to enter a name for their character.

The display name can optionally be displayed near the character later.

The prefab is located at:

Prefabs > Character Creator > Modules > Character Display Name Field

---

## Recommended Setup Order

For a first Character Creation Menu, the following order is recommended:

### 1. Create the Menu Manager

- Add **CharacterCreationMenuManager** to a GameObject.
- Set the **Default Character Type**.
- Assign the **Menu Contents** GameObject.

### 2. Add Layer Selectors

- Choose a Layer Selector type.
- Add its **Pre-Setup** prefab.

### 3. Add the Character Preview

- Add the core Character Preview prefab.

### 4. Add Menu Controls

- Add **Save** and **Back** buttons.

### 5. Add a Loading Screen

- Use a Pre-Setup prefab if you want a ready-to-use setup.

### 6. Test the menu

- Open the menu using one of the **CharacterCreationMenuManager** methods.
- Verify that the character loads and can be edited and saved.

## 7. Add optional features

- Randomization
- Character History
- Display Name

---

## Related

The following pages cover individual systems of the Character Creator in more detail:

- [Layer Selector Setup](#)
- [Character Preview](#)
- [Menu Controls](#)
- [Loading Screens](#)
- [Character Randomization](#)
- [Character History System](#)

# Character Preview

A **character preview** shows a live view of the character in the **Character Creation Menu**.

Whenever a layer of the character is modified, the preview is updated automatically.



## Character Preview Controller

If using the **Animated Preview Mode**, an **Animator Controller** is required.

This controller has specific guidelines that must be followed.

[Read More → Character Preview Controller](#)

## Character Preview Setup

- **Fastest Setup** > Use the [Pre-Setup Prefabs](#)
- **Custom Setup** > Follow the [Manual Setup](#)

Regardless of the setup method. The **Preview Mode** must be configured.

## Preview Mode

The Character Preview supports two display modes:

| Mode     | Description                                          |
|----------|------------------------------------------------------|
| Static   | Displays a single sprite                             |
| Animated | Uses an Animator Controller to animate the character |

## Static

1. Uses the **Preview Sprite** from the **Layered Character Type > Character Creator Settings**.
2. No **Animator Controller** required.

## Animated

- Requires an **Animator Controller**.
- Assigned in the **Layered Character Type > Character Creator Settings**.

### TIP

Animator Controller setup and requirements are explained here:

[Character Preview Controller](#)

## Additional Controls

Additional controls can be added to a **Character Preview** to add more functionality.

The **Character Preview** contains two types of additional controls:

## Rotation Controls

**Rotation Controls** allow the player to change the character's facing direction.



Two buttons can be added to the preview, allowing the player to rotate the character left or right.

**Rotation Controls** require **Animated Preview Mode**.

[Manual Setup](#)

---

## Animation Controls

**Animation Controls** allow the player to switch between different pre-defined animations.



Each animation is represented by a button. Selecting an animation causes the Character Preview to play the related animation.

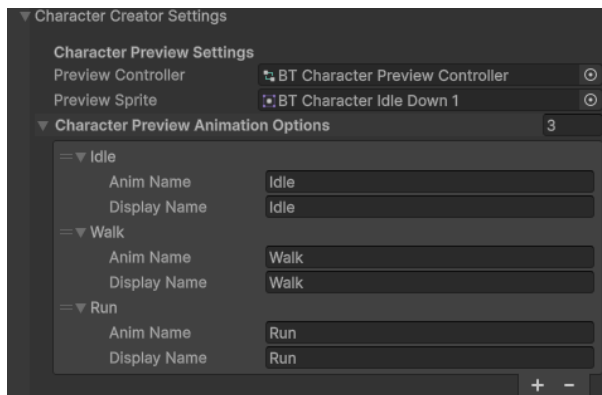
**Animation Controls** require **Animated Preview Mode**.

### [Manual Setup](#)

## Assigning Animations

To assign animations to be used in animation controls:

1. Navigate to your **Layered Character Type**.
2. Expand **Character Creator Settings**.
3. Add entries to the **Character Preview Animation Options** list.
4. Each entry needs an **Animation Name**, which **MUST** match the name of an animation or blend tree in the **Preview Controller**.
5. Optionally a **Display Name** can be set. This is the name that will be displayed by the **Animation Controls**. If not set, the **Animation Name** will be used instead.



# Prefabs

Prefabs offer easy implementation of character previews without the need to set them up yourself.

**Location:** Prefabs > Character Creator > Modules > Character Preview.

---

## Pre-Setup Prefabs

Located in the **Pre-Setup** subfolder.

These prefabs are fully configured and ready to use.

- **Character Preview** - Shows a preview of the character with no other controls.
- **Character Preview [+Rotation Controls]** - Includes buttons on the left and right to rotate the character.
- **Character Preview [+Rotation Controls, Anim Buttons]** - Includes rotation controls and buttons at the bottom to switch the animation the character is playing.

Drag and drop any of these prefabs into your menu, enter **Play Mode** and you'll have a functioning Character Preview.



---

## Animation Controls Prefabs

Located in the **Animation Controls** subfolder.

These are the same animation controls used in the full pre-setup prefab, provided separately for custom setups.

Four variations are provided, each in their own folder.

Variations contain different sprites but do not change any functionality.

Within each variation folder are two prefabs:

1. **Animation Button** - A single animation button, not functional on its own.
2. **Animation Controls [Initialize Existing]** - A complete setup using the **CCM Animation Switcher** component.

## Setup

1. Add the prefab to your scene.
2. Assign the **Character Preview Handler** to the **CCM Animation Switcher**.
3. Buttons will initialize automatically at runtime.

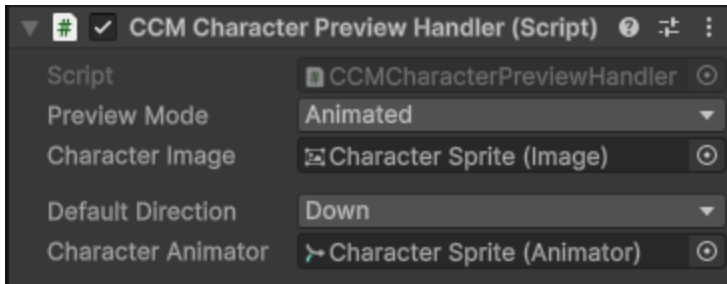
## Manual Setup

Want to create a **Character Preview** from scratch? Here's how to do that.

### Step 1 Create Preview Object

1. Create a new **GameObject**.
2. Add the **CCM Character Preview Handler** component

This is the component responsible for initializing and updating your character preview.



### Step 2 Select Preview Mode

Choose how the preview is displayed:

The **Preview Mode** determines how the character preview is displayed.

1. **Static**
  - Uses the **Character Preview Sprite** assigned in the **Character Creator Settings** section of the **Layered Character Type**.
2. **Animated**
  - Uses the **Character Preview Controller** assigned in the **Character Creator Settings** section of the **Layered Character Type** to animate the character preview.

If **Animated** is selected:

- Assign the **Character Animator** (Animator component).
- The **Preview Controller** will be assigned to this **Animator** at runtime.

---

## Step 3 Assign Character Image

Assign the **Character Image**. This is the Image component that the [Character Shader](#) will be applied to.

Now once you enter **Play Mode**, you'll see the character displayed in the preview you just created.

---

## Adding Rotation Controls

Rotation Controls allow the player to change the character's facing direction.

### Setup

1. Create a new **GameObject**.
2. Add the Unity **Button** component.
3. Assign the Buttons **On Click** event to the **CCMCharacterPreviewHandler** component and call the **RotateCharacterPreview()** method.
4. The **RotateCharacterPreview()** requires a bool, disabled = rotate left, enabled = rotate right.

Follow these steps twice to create two buttons, one for rotating left, the other for rotating right.

---

## Adding Animation Controls

Animation Controls allow for switching between animations at runtime.

### NOTE

Requires Animated Preview Mode.

1. Create a new **GameObject**.
2. Add the **CCM Animation Switcher** component to the GameObject.
3. Set **Animation Button Parent**. This is the parent GameObject for your Animation Buttons (Can be the same GameObject).
4. Set the Initialization Mode.
  - **Initialize Existing**: Will find pre-created animation buttons within the Animation Button Parent
  - **Auto Create**: Will instantiate new Animation Buttons as children of the Animation Button Parent. Uses the Animation Button Prefab you assign.

5. Assign **Character Preview Handler** reference. This is used to change the currently playing animation on the Character Preview.

## Animation Button Prefab

If the **CCM Animation Switcher** is set to **Auto Create**, it requires a reference to an **Animation Button prefab**.

### Use pre-existing Animation Button prefab:

- Inside the **Animation Controls** prefab folder is an **Animation Button prefab**. That can be assigned directly to the **CCM Animation Switcher**.

### Create your own Animation Button Prefab:

1. Create a new **GameObject**.
2. Add the **CCM Animation Preview Button Handler** component.
3. Add the Unity **Toggle** component.
4. Assign the **Toggle** to the **CCM Animation Preview Button Handler** component.
5. Add two children GameObjects, name one "**Highlight**" and the other "**Text**".
6. Add an Image component to the **Highlight** GameObject and assign the sprite you want the button to show when it's selected.
7. Assign the **Image** component to the **Graphic** field on the **Toggle**.
8. Add the **TextMeshPro - Text** component to the **Text** GameObject.
9. Assign the **TextMeshPro - Text** component to the **Text** field on the **CCM Animation Preview Button Handler**.

Drag and drop the **parent GameObject** into the **Project window** to turn it into a prefab. You can now assign your new prefab directly to the **CCM Animation Switcher**.

---

## Define Preview Animations

Animation buttons are generated based on the **Character Preview Animation Options** list.

## Location

**Layered Character Type > Character Creator Settings**

## Each Entry Contains

1. **Animation Name** - The name used to find the animation in the **Animator Controller**.
2. **Display Name** - The text shown on the button.

At runtime:

- One button is created per entry.
- Pressing a button plays the corresponding animation.

# Character Preview Controller

The **Character Preview Controller** is the **Animator Controller** used to animate the character within the **Character Creation Menu**.

It is specifically designed for use with the [Character Preview](#) module.

The controller is assigned in the **Layered Character Type** under the **Character Creator Settings** section.

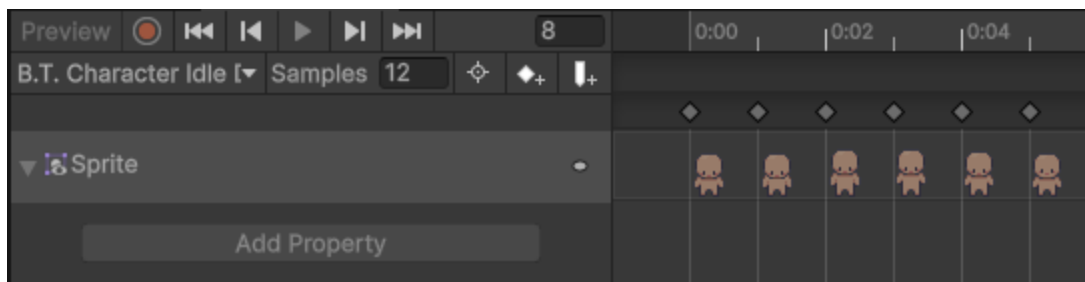
---

## UI Animation Requirement

The Character Preview is rendered using a **UI Image component**, not a **Sprite Renderer**.

When creating animations in Unity, the animated property is tied to the component type used at the time of recording. If an animation was created using a **Sprite Renderer**, it will only affect **Sprite Renderer** components.

As a result, using those animations in UI (which uses an **Image**) will not work.



## Requirements

- Animations must target an **Image** component.
- Animations that only target a **Sprite Renderer** will not function in the **Character Creation Menu**.

## Options

1. Create a new set of animations targeting an **Image** component.
  2. Modify existing animation to also animate an **Image** component.
- 

## Animator Controller Setup Requirements

If the Character Preview does **not** use **Rotation Controls** or **Animation Controls**:

- There are no setup requirements.
- The default animation will play automatically at runtime.

If using additional features:

- Rotation Controls > see [Rotation Controls](#).
  - Animation Controls > Animations are played directly by name, so state transitions should not be included.
- 

## Rotation Controls

**Rotation Controls** allow the character to change the direction they're facing direction using buttons.

When implemented, the **Animator Controller** requires one parameter:

Type: **Float** Name: **Direction**

## Direction Values

| Value | Direction |
|-------|-----------|
| 0     | Forward   |
| 1     | Backward  |
| 2     | Left      |
| 3     | Right     |

The value of this paramter is updated at runtime when rotation buttons are pressed.  
It's used by the Animator to determine which directional animation should be played.

---

## Recommended Setup (Blend Trees)

**Blend Trees** are the recommended approach for handling directional animations.

They allow a single animation state (such as **Idle** or **Walk**) to automatically switch between directions based on the **Direction** paramter.

---

## Concept

Instead of creating separate states like:

- Idle\_Forward
- Idle\_Backward
- Idle\_Left
- Idle\_Right

A **Blend Tree** combines all of these into a single state:

- **Idle (Blend Tree)**

The Animator then selects the correct animation based on the **Direction** value.

---

## Step 1: Create Blend Tree State

1. Open the **Animator Controller**
  2. Create a new state (e.g. **Idle**).
  3. Create new Blend Tree: **Right click** > **Create State** > **From New Blend Tree**.
  4. Give the Blend Tree a name (e.g. **Idle**).
- 

## Step 2: Configure Blend Tree

- Set **Blend Type**: **1D**.
  - Set **Parameter**: **Direction**.
- 

## Step 3: Add Motions

Add four motions to the Blend Tree:

| Motion        | Threshold |
|---------------|-----------|
| Idle_Forward  | 0         |
| Idle_Backward | 1         |
| Idle_Left     | 2         |
| Idle_Right    | 3         |

Each motion is a separate animation clip.

The **Threshold** defines which value of the **Direction** paramter will trigger that animation.

---

## How It Works

At runtime:

1. The **Character Preview Handler** updates the **Direction** paramter.
2. The Blend Tree evaluates its thresholds.
3. The matching directional animation is played.

Example:

**Direction** = 2 > plays **Idle\_Left**.

---

## Extending to Multiple Animations

This structure can be reused for additional animations.

For example:

- Idle
- Walk
- Run

If you have [Animation Controls](#) setup they can call these other **Blend Trees**, when called they will play the correct directional animation depending on the value of **Direction**.

# Character Creator Menu Controls

Menu Controls are UI buttons that provide core user actions such as saving, closing, randomizing and resetting a character.

The included prefabs can be quickly added to a **Character Creation Menu** or replaced with custom implementations using the **CCM Relay** component.



## Included Actions

Menu Controls provide the following actions:

- Close or exit the Character Creation Menu.
- Save or confirm character changes.
- Reset the character to its original state.
- Randomize the characters appearance.

## Prefabs

**Location:** Prefabs > Character Creator > Modules > Menu Controls.

Ready-made prefabs are included and can be used without any additional setup. Drag and drop them directly into your Character Creation Menu hierarchy and they're immediately ready to be used.

## Variants

Within the prefabs folder you'll see 4 variation folders. The only difference between variants is the sprites used.

**Available prefabs:**

1. Menu Controls [Back, Save, Randomize, Reset]
2. Menu Controls [Back, Save, Randomize]
3. Menu Controls [Back, Save]

The brackets dictate what actions are included in the prefab.

---

## Generic Buttons

Inside the same prefabs directory, the **Generic Buttons** folder contains individual button prefabs with no default assigned functionality.

Each prefab includes:

- A Unity **Button** component
- A **CCM Relay** component

When using the generic button prefabs, add an **On Click** event to the button and assign the **CCM Relay** component. You can then call different methods on the **CCM Relay** component to perform actions such as saving changes or exiting the menu.

---

## CCM Relay Component

The **CCM Relay** (Character Creation Menu Relay) component acts as a bridge between the UI and the Character Creation Menu system.

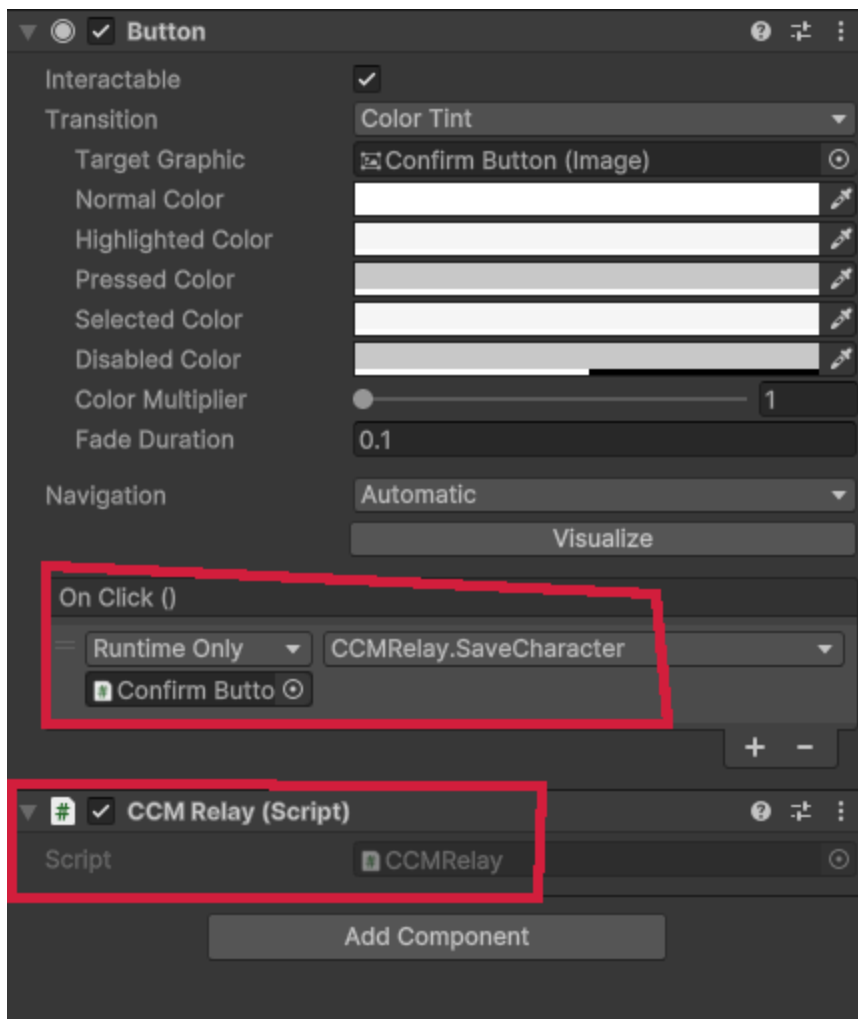
It forwards button events to the **active Character Creation Menu Manager instance**, making it easy to setup simple functionality without writing any code.

## Usage

To create a custom button using the **CCM Relay**:

1. Add the **CCM Relay** component to a GameObject
2. Add the Unity **Button** component
3. Assign the Buttons **On Click** event to call a method on the **CCM Relay** component

## Example



## CCM Relay Methods

The following public methods are available on the **CCM Relay** component:

### DisableMenu()

Closes the disables the **Character Creation Menu**.

### SaveCharacter()

Saves changes made to the currently edited character.

### RandomizeEntireCharacter()

Randomizes all character layers of the character currently being edited.

### ResetCharacter()

Restores the character to the state it was in when the **Character Creation Menu** was first opened.

---

## CCM Save Character Button Component

The **CCM Save Character Button** component provides a ready-to-use save button for your Character Creation Menu. It automatically handles saving the current character and manages its interactable state based on whether there are unsaved changes.

### Setup

1. Create a new **GameObject**.
2. Add an **Image** component.
3. Add the **CCM Save Character Button** component.
  - The component inherits from Unity's **Button** component, so it provides all of the same properties and functionality.
4. Assign the **Target Graphic** and configure the button's **Transition** settings as you would with any other Unity button.

### Behavior

- **Disable When No Changes** - When enabled, the button is automatically disabled if no changes have been made to the character since it was last saved.
- An **On Click** event is **not required**. When clicked, the button automatically saves the current character.
- You can still add **On Click** event actions if you want the button to perform additional actions after saving. For example, you could use an On Click event that calls the [DisableMenu](#) method on a **CCM Relay** to close the Character Creation Menu after the character has been saved.

# Layer Selector Setup

A **Layer Selector** is a UI element that lets the player modify a specific **layer** of a [Layered Character](#).

Each selector is responsible for controlling one layer of the character. (e.g. Body, Outfit, Hairstyle, etc.)

## Layer Selector Types

The following selector types are included:

| Selector Type                     | Description                                                                                            |
|-----------------------------------|--------------------------------------------------------------------------------------------------------|
| <a href="#">Dropdown Selector</a> | a Simple dropdown listing all options for a layer.                                                     |
| <a href="#">Carousel Selector</a> | Displays the current option with button to cycle left/right.                                           |
| <a href="#">Grid Selector</a>     | A grid of preview thumbnails for each layer option.                                                    |
| <a href="#">List Selector</a>     | A vertical or horizontal list of options, optionally with preview images.                              |
| <a href="#">Tab Selector</a>      | Works alongside another selector. Clicking a tab switches which layer another selector is controlling. |



# How Layer Selectors Work

A **Layer Selector** cannot function on its own.

All selectors must be initialized and managed by the **CCM Character Layer Selection Manager**.

This component:

- Initializes Layer Selectors.
  - Assigns each selector to a specific layer.
  - Handles Layer Selector overflows.
- 

## Bulk Selector Setup

The most common setup is to use **multiple Layer Selectors**, one for each layer with all Layer Selectors shown on screen at once.

There are two ways to setup Layer Selectors:

- [Pre-Setup Prefabs](#) - Use prefabs which are already full setup and functional.
  - [Manual Setup](#) - Setup Layer Selectors from scratch.
- 

## Prefabs

**Location:** **Prefabs** > **Character Creator** > **Modules** > **Layer Selectors**.

Each Layer Selector type has its own folder containing two subfolders:

---

### Core Folder

Contains individual Layer Selector prefabs.

- Usually includes multiple variations.
  - Not functional on their own.
  - Intended for manual setup.
- 

### Pre-Setup Folder

Contains collections of the **Layer Selector** fully configured with the **Character Layer Selection Manager component**.

They can be dropped straight into any **Character Creation Menu** and are functional without any other setup.

Two variations of the **Pre-Setup** prefab are provided:

---

## [Initialize Existing]

Prefabs that contain **[Initialize Existing]** at the end of the name include a set of **Layer Selectors** within the prefab.

**Vertical**, **Horizontal** or **Grid Layout Groups** are included and configured. These are used to space out Layer Selectors automatically.

These prefabs are useful when you want to control exact placement of each Layer Selector. The downside is if not enough Layer Selectors are included, not all layers of the character will be editable and a warning will be logged.

### NOTE

If too many **Layer Selectors** are included, by default **unused Layer Selectors will be disabled and hidden**. This action can be changed in the **Character Layer Selection Manager** component.

---

## [Auto Create]

Prefabs that contain **[Auto Create]** at the end of the name do **NOT** include **Layer Selectors** beforehand, instead they are automatically instantiated at runtime and are placed inside a child GameObject with a **Grid Layout Group** setup to organize them.

These can be useful for situations where the amount of layers a character will have can change. For example if the **Character Creation Menu** will be used with **multiple different Character Types** with different amounts of layers.

When the **Character Creation Menu** is opened, the appropriate amount of **Layer Selectors** will be instantiated automatically.

---

## Layer Selector Customization

All Layer Selectors can be customized:

- Sprites can be changed
- Fonts and text size can be changed
- Colors can be modified

Go wild! Make those Layer Selectors unique to your game.

---

# Manual Setup

Steps to setup your own collection of Layer Selectors with the **Character Layer Selection Manager**.

---

## Create Manager

1. Create a new **GameObject** named "**Layer Selectors**".
  2. Add the **CCM Character Layer Selection Manager** component.
- 

## Choose Initialization Mode

There are two Initialization Modes:

---

### Auto Create

At runtime, the manager will instantiate a new **Layer Selector** for each character layer.

#### Required Fields:

##### 1. Layer Selector Prefab

- The prefab used when instantiating Layer Selectors.
- Found under: **Prefabs > Character Creator > Modules > Layer Selectors > [Type] > Core**.
- Unsure which Layer Selector to choose? [Check out the list here](#).

##### 2. Layer Selector Parent

- The parent GameObject where selectors will be instantiated.
  - Typically uses a **Layout Group component** for automatic spacing.
- 

## Initialize Existing

All Layer Selectors must already exist in the hierarchy.

#### Setup Steps:

1. Navigate to: **Prefabs > Character Creator > Modules > Layer Selectors**.
2. Open the folder for the Layer Selector you want to use.
3. Open the [Core](#) folder.
4. Drag selector prefabs into your scene.
5. Make them children of the [Layer Selectors Parent](#) GameObject.

Unsure which Layer Selector to choose? [Check out the list here](#).

---

## Layer Selector Parent

- Reference to the parent GameObject containing all Layer Selectors.
- Required for initialization.
- Can be the same GameObject as the manager.

## Unused Selector Action

Determines what happens when there are more selectors than layers:

| Option                   | Behavior                                               |
|--------------------------|--------------------------------------------------------|
| Nothing                  | Leaves the selector unchanged.                         |
| Disable                  | Disables interactivity but keeps it visible.           |
| Disable and Change Alpha | Disables interactivity and fades using a Canvas Group. |
| Hide (Default)           | Completely hides the selector.                         |

---

## Test

1. Enter **Play Mode**.
2. Open the **Character Creation Menu**.

If setup correctly:

- Layer Selectors will initialize automatically.
- Each selector will control a different layer.
- Changing a selector updates the character in real-time.

# Dropdown Layer Selector

A **Dropdown Layer Selector** is a standard dropdown UI element.

When opened, it displays a list of all available options for the assigned character layer.



## Usage

The **Dropdown Layer Selector** can be used in two ways.

1. Use in a **bulk setup** alongside a set of other **Dropdown Layer Selectors**.
  - [Read the setup guide here.](#)
2. Use a single dropdown with **Tab Layer Selectors**.
  - [Read more about Tab Layer Selectors here](#)

# Prefabs

**Location:** Prefabs > Character Creator > Modules > Layer Selectors > Dropdown Selector.

---

## Core Prefabs

Located in the **Core** subfolder

Contains individual Dropdown Layer Selector prefabs.

- **Layer Dropdown Selector** – Basic dropdown selector.
- **Layer Dropdown Selector [+Randomize]** – Includes a randomize button next to the dropdown.
- **Layer Dropdown Selector [+Title]** - Includes a title above the dropdown with the layer name.

These prefabs are **not** functional on their own.

---

## Pre-Setup Prefabs

Located in the **Pre-Setup** subfolder.

These are bulk prefabs that contain multiple **Dropdown Layer Selectors**.

Prefabs that contain a pre-configured [Character Layer Selection Manager](#).

These prefabs will work out of the box without any extra setup.

- **Dropdown Selectors [Auto Create]** – Instantiates dropdown selectors at runtime. Uses a Grid Layout Group component to sort them.
  - **Dropdown Selectors [Initialize Existing]** – Uses dropdown selectors already present in the prefab hierarchy.
- 

## Pre-Setup Tab

Located in the **Pre-Setup Tab** subfolder.

Pre-configured with the **CCM Selected Layer Tab Handler** component and ready to be used with [Tab Layer Selectors](#).

Included prefabs:

- **Dropdown Layer Tab**

Drag and drop a prefab into the **Character Creation Menu** and assign the **Character Layer Selection Manager**.

---

## Limitations

- The **Dropdown Layer Selector** only shows **text**. If you need visual previews, consider the [Grid Layer Selector](#) or [List Layer Selector](#).

# Carousel Layer Selector

A **Carousel Layer Selector** displays the currently selected layer option in the middle with left/right controls to the side used to cycle through other options available for the assigned layer.



---

## Usage

The **Carousel Layer Selector** can be used in two ways.

1. Use in a **bulk setup** alongside a set of other **Carousel Layer Selectors**.
  - [Read the setup guide here.](#)
2. Use a single Carousel with a **Tab Layer Selector**.
  - [Read more about Tab Layer Selectors here](#)

---

## Prefabs

**Location:** Prefabs > Character Creator > Modules > Layer Selectors > Carousel Selector.

---

## Core Prefabs

Located in the **Core** subfolder.

Contains individual Carousel Layer Selector prefabs.

- **Layer Carousel Selector** – Base carousel selector.
- **Layer Carousel Selector [+Randomize]** – Includes a randomize button below the carousel.
- **Layer Carousel Selector [+Title]** – Includes a title above the carousel with the layer name.
- **Layer Carousel Selector [-Background]** - Basic carousel selector without a background.

These prefabs are **not** functional on their own.

---

## Pre-Setup Prefabs

Located in the **Pre-Setup** subfolder.

These are bulk prefabs that contain multiple **Carousel Layer Selectors**.

**Pre-setup** prefabs already include a [Character Layer Selection Manager](#).

These will work out of the box without any extra setup required.

- **Carousel Selectors [Auto Create]** – Instantiates carousel selectors at runtime. Uses a Grid Layout Group component to sort them.
  - **Carousel Selectors [Initialize Existing]** – Uses carousel selectors already present in the prefab hierarchy.
- 

## Pre-Setup Tab

Located in the **Pre-Setup Tab** subfolder.

Pre-configured with the **CCM Selected Layer Tab Handler** component and ready to be used with [Tab Layer Selectors](#).

Included prefabs:

- **Carousel Layer Tab**

Drag and drop a prefab into the **Character Creation Menu** and assign the **Character Layer Selection Manager**.

---

## Limitations

- The **Carousel Layer Selector** only shows **text**. If you need visual previews, consider the [Grid Layer Selector](#) or [List Layer Selector](#).

# Grid Layer Selector

A **Grid Layer Selector** contains a grid where each element in the grid represents an option of the assigned character layer. When an element is selected it will be applied to the character immediately.



## Usage

The Grid Layer Selector can be used in two ways.

1. Use a single Grid Layer Selector with **Tab Layer Selectors** (Recommended).
  - [Read more about Tab Layer Selectors here](#)
2. Use in a **bulk setup** alongside a set of other **Grid Layer Selectors**.
  - [Read the setup guide here](#).

## Prefabs

**Location:** Prefabs > Character Creator > Modules > Layer Selectors > Grid Selector

## Core Prefabs

Located in the **Core** subfolder.

Contains individual Grid Layer Selector prefabs.

- **Layer Grid Selector [Sprite]** – Most common grid selector. Each element in the grid is displayed as a sprite.
- **Layer Grid Selector [Text]** – Each element in the grid is displayed as text.
- **Layer Grid Selector [Sprite + Text]** - Hybrid. Each element in the grid is displayed as both sprite and text.
- **Layer Grid Selector [Vertical + Title]** – Grid is vertical & includes a title with the name of the assigned layer at the top.

These prefabs are **not** functional on their own.

---

## Pre-Setup Prefabs

Located in the **Pre-Setup** subfolder.

These are bulk prefabs that contain multiple **Grid Layer Selectors**.

Pre-setup prefabs already include a [Character Layer Selection Manager](#).

These will work out of the box without any extra setup required.

- **Grid Selectors [Auto Create]** – Instantiates grid selectors [sprite] at runtime. Uses a **Horizontal Layout Group component** to sort them.
  - **Grid Selectors [Initialize Existing]** – Uses grid selectors already present in the prefab hierarchy.
- 

## Grid Entries

Located in the **Grid Entries** subfolder.

These are the entries used in the grid for the **Layer Grid Selector**.

A **grid entry** is referenced in a **Layer Grid Selector** and defines how each entry in the grid looks and functions.

The following are entries that can be used in any **Layer Grid Selector**:

1. **Layer Option Grid Element [Sprite]** - Displays a sprite preview of what the layer option looks like.
2. **Layer Option Grid Element [Text]** - Displays text containing the name of the assigned layer option.
3. **Layer Option Grid Element [Text + Sprite]** Displays text containing the name of the assigned layer option & a preview of what the layer option looks like.

---

## Pre-Setup Tab

Located in the **Pre-Setup Tab** subfolder.

Pre-configured with the **CCM Selected Layer Tab Handler** component and ready to be used with [Tab Layer Selectors](#).

Included prefabs:

- **Grid Layer Tab**

Drag and drop a prefab into the **Character Creation Menu** and assign the **Character Layer Selection Manager**.

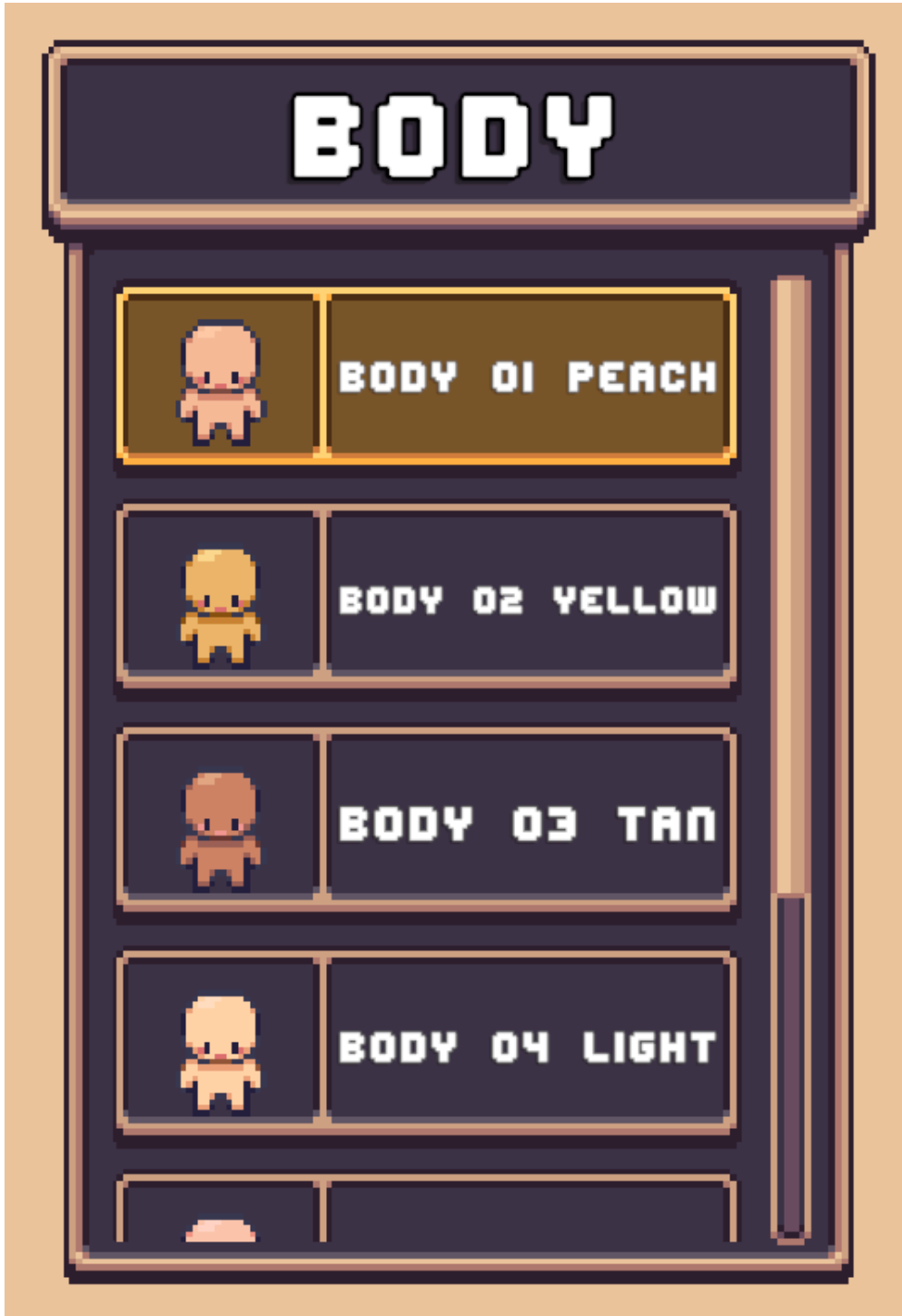
---

## Limitations

- The grid can be quite big and take up a lot of space at times. Works best when used with a [Tab Layer Selector](#).

# List Layer Selector

A **List Layer Selector** contains a vertical list where each element in the grid represents an option of the assigned character layer. When an element is selected it will be applied to the character immediately.



## Usage

The List Layer Selector can be used in two ways.

1. Use a single List Layer Selector with **Tab Layer Selectors** (Recommended).

- [Read more about Tab Layer Selectors here](#)
2. Use in a **bulk setup** alongside a set of other **Grid Layer Selectors**.
- [Read the setup guide here.](#)
- 

## Prefabs

**Location:** Prefabs > Character Creator > Modules > Layer Selectors > List Selector.

---

### Core Prefabs

Located in the **Core** subfolder.

Contains individual List Layer Selector prefabs.

- **Layer List Selector** – Most common list selector. Each element in the list is displayed as both text & sprite.
- **Layer List Selector [+Title]** – Includes a title with the name of the assigned layer at the top.

These prefabs are **not** functional on their own.

---

### Pre-Setup Prefabs

Located in the **Pre-Setup** subfolder.

These are bulk prefabs that contain multiple **List Layer Selectors**.

Pre-setup prefabs already include a [Character Layer Selection Manager](#).

These will work out of the box without any extra setup required.

- **List Selectors [Auto Create]** – Instantiates list selectors [+Title] at runtime. Uses a **Horizontal Layout Group component** to sort them.
  - **List Selectors [Initialize Existing]** – Uses list selectors already present in the prefab hierarchy.
- 

### List Entries

Located in the **List Entries** subfolder.

These are the entries used in the list for the **Layer List Selector**.

A **list entry** is referenced in a **Layer List Selector** and defines how each entry in the list looks and functions.

The following are entries that can be used in any **Layer List Selector**:

1. **Layer Option List Element [Text]** - Displays only text containing the name of the assigned layer option.
  2. **Layer Option List Element [Text + Sprite]** Displays text containing the name of the assigned layer option & a preview of what the layer option looks like.
- 

## Pre-Setup Tab

Located in the **Pre-Setup Tab** subfolder.

Pre-configured with the **CCM Selected Layer Tab Handler** component and ready to be used with [Tab Layer Selectors](#).

Included prefabs:

- **List Layer Tab**

Drag and drop a prefab into the **Character Creation Menu** and assign the **Character Layer Selection Manager**.

---

## Limitations

- The list can take up a lot of space when multiple are used at a time. Works best when used with a [Tab Layer Selector](#).

# Tab Layer Selector

A **Tab Layer Selector** is a button. It won't do much on its own. It must be paired with another **Layer Selector**.

When a Tab Layer Selector is pressed it will change the assigned layer of another Layer Selector.

The Tab Layer Selector can be used as an alternative to a [bulk layer selector setup](#). Instead only requiring one Layer Selector in the menu that can be changed to control different layers using Tab Layer Selectors.

On its own, a Tab Layer Selector does not modify the character.

Instead, it updates the **Selected Layer** in the **Character Layer Selection Manager**, which other selectors can respond to.



## How It Works

- Each tab represents a specific **character layer** (Body, Outfit, Hairstyle, etc.)
- When a tab is pressed:
  - The **Selected Layer** in the **Character Layer Selection Manager** is updated.
- A separate **Layer Selector** is setup with a handler which listens for changes and updates **the Layer Selector** to control the newly selected layer.

This allows you to:

- Use one **Layer Selector** to control **multiple layers**.
- Reduce UI clutter compared to having one selector per layer.

### NOTE

The **Tab Layer Selector** was originally designed to work with [Grid Layer Selectors](#) & [List Layer Selectors](#) but can work with any **Layer Selector**.

---

## Usage

**Tab Layer Selectors** are setup the same way as other Layer Selectors, using the **Character Layer Selection Manager**.

Follow this guide to setup your **Tab Layer Selectors**. [Bulk Selector Setup](#)

---

## Required Companion Setup

Tab Layer Selectors require one additional component to function properly:

**CCM Selected Tab Handler**

This component listens for changes to the **Selected Layer** and updates a target **Layer Selector**.

---

## Pre-Setup Tab Prefabs

Each Layer Selector type includes a Pre-Setup Tab folder.

These prefabs already include the **CCM Selected Layer Tab Handler** component.

## Setup

1. Drag a prefab into your **Character Creation Menu**.
2. Assign a reference to the **CCM Character Layer Selection Manager**.

Once assigned, setup is complete.

---

## Manual Setup

Follow these steps to manually setup **Tab Layer Selectors** with its **companion Layer Selector**.

---

### Step 1 Add Tab Selectors

Use the [Pre-Setup prefabs](#) or follow the [Manual Setup](#) guide to setup your **Tab Layer Selectors**.

---

### Step 2 Create Handler GameObject

1. Create a new **GameObject**.
  2. Add the **CCM Selected Layer Tab Handler** component.
  3. Assign the **Character Layer Selection Manager** reference.
-

## Step **3** Add Target Layer Selector

1. Add any Layer Selector to your Menu (From a [Core](#) folder)
2. Place it under your **handler GameObject** (recommended but not required)
3. Assign it to the **Layer Selector** field in the handler.

This selector will dynamically change which layer it controls.

Now enter **Play Mode** and test it. When you press any of the **Tab Layer Selectors**, the handler will update the assigned layer in the **target Layer Selector**.

---

## Prefabs

**Location:** [Prefabs](#) > [Character Creator](#) > [Modules](#) > [Layer Selectors](#) > [Tab Selector](#)

Four visual variation folders are included.

All variations contain identical functionality and differ only in appearance.

## Core Prefabs

Located in the **Core** subfolder.

- **Layer Tab Selector** – Basic tab selector (Just a button).

Not functional on its own.

---

## Pre-Setup Prefabs

Located in the **Pre-Setup** subfolder.

- **Tab Selectors [Initialize Existing]** – Uses tab selectors already present in the prefab hierarchy.

### NOTE

There is no **Auto Create** variant for **Tab Layer Selectors**.

Tab Layer Selectors require a reference to the [Character Layer Selection Manager](#) which must be set manually.

This means they **cannot** be created automatically at runtime.

---

# History Tracking System

The **History Tracking System** tracks every change made to the character in the Character Creation Menu while open. Changes can then be looked through and undone at anytime.

## Features Overview

| Feature                   | Description                                                                 |
|---------------------------|-----------------------------------------------------------------------------|
| <b>Automatic Tracking</b> | Every change to the character can be recorded as a snapshot.                |
| <b>Undo/Redo</b>          | Step backward or forward through the list of changes made to the character. |
| <b>Direct Selection</b>   | Select any snapshot to instantly restore the character.                     |
| <b>Snapshot Limit</b>     | Configurable maximum number of snapshots (1-100, default = 30).             |

## History Tracker Component

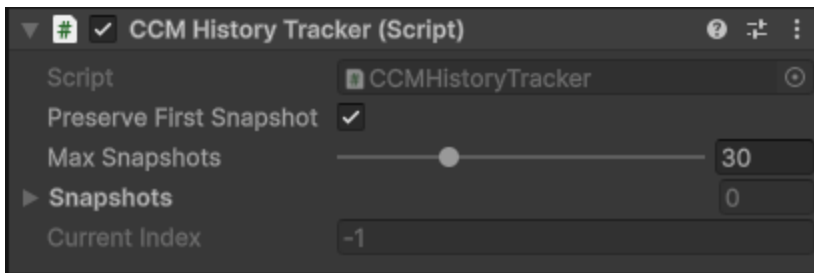
Introducing the [CCM History Tracker](#) component.

This component will track every time a layer on the character is changed and record the change as a snapshot.

- A snapshot contains all information required to restore a character to the exact state it was in when the snapshot was taken.
- Snapshots are saved in a list with newest entries added to the end of the list.
- **Max Snapshots** can be set anywhere between **1-100** (Default is **30**).
- When the max amount of snapshots is met, oldest snapshots will start being replaced.
- The **Preserve First Snapshot** bool can be toggled which will stop the first snapshot created from being deleted or replaced.

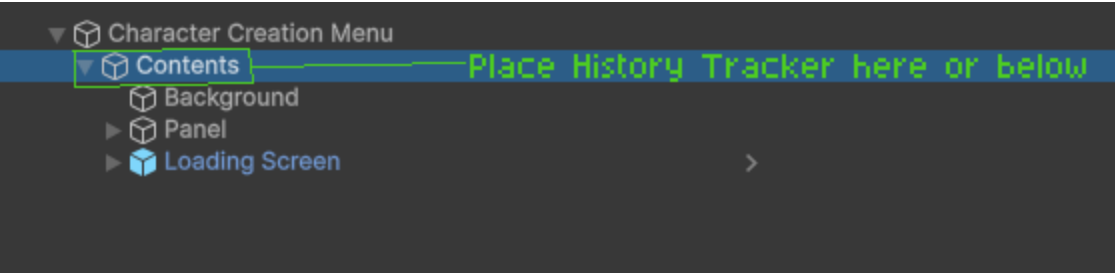
### NOTE

The **CCM History Tracker** component is required for both **History Panels** and **Undo/Redo controls** to function.



# Component Placement

The **CCM History Tracker** component should be placed inside the **Character Creation Menu Contents**. This is the **Menu Contents** GameObject the **Character Creation Menu Manager** component references.

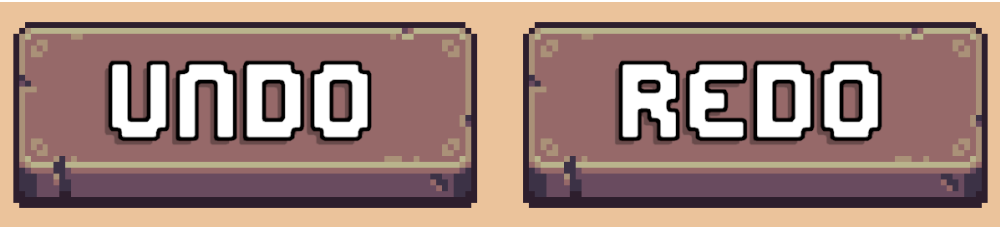


# Integrations

Additional UI elements which integrate with the **History Tracking System**.

## Undo/Redo Buttons

Buttons for un-doing or re-doing changes made to the character.



[Read More → History Undo/Redo](#)

## History Panels

Panels which display every change made in a clean list.



[Read More → History Panels](#)

# History Panels

A **History Panel** displays a list of snapshots recorded by the [History Tracker](#).

These snapshots represent previous states of a character, allowing the player to:

- Review past changes.
  - Revert to previous versions.
- 

## Panel Types

There are two primary types of **History Panels**. Both display a list of modifications made to the character but show that information in different ways.

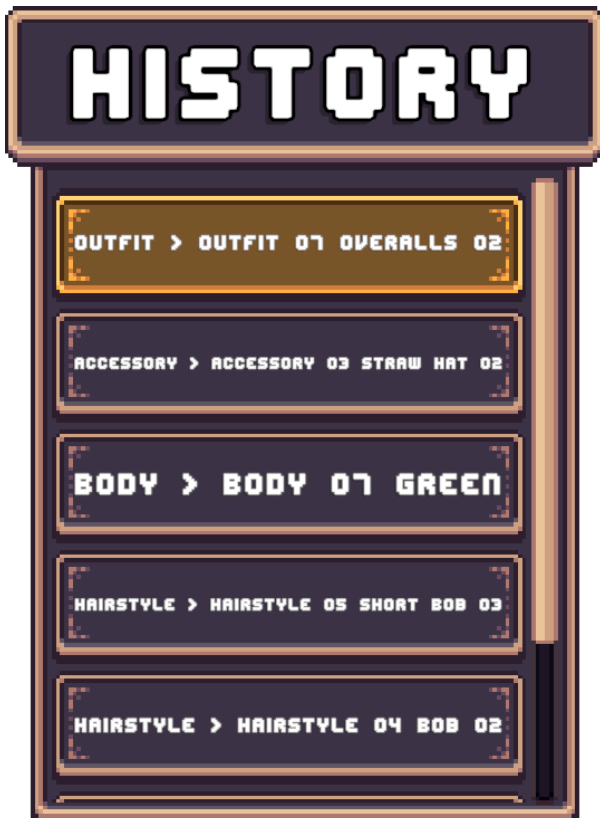
---

### Text-Based Panels

A **Text-Based Panel** describes each snapshot using text.

- **1 Layer Changed** > Displays the layer name and selected option.
- **2 Layers Changed** > Displays both layer names.
- **3+ layers changed** > Displays the number of layers modified.

**Typical Layout:** Vertical List.



## Sprite-Based Panels

A **Sprite-Based Panel** visually represents each snapshot using character previews.

Each entry reconstructs the characters appearance at that point in time by taking a single frame from each spritesheet contained in the snapshot and combining them together.

**Typical Layout:** Horizontal List.



### Preview Frame Setup

To define which frame is used for preview generation:

1. Select your **Layered Character Type** asset and go to the Inspector window.
2. Expand **Character Creator Settings**
3. Locate **Character Preview Settings**
4. Assign a **Preview Sprite**

#### *i* NOTE

The **Preview Sprite** MUST be a sprite from the **Base Spritesheet**.

The **rect** of this sprite determines the frame extracted from every layer.  
All extracted frames are combined to render the final preview.

## Prefabs

Location: [Prefabs](#) > [Character Creator](#) > [Modules](#) > [History](#) > [History Panels](#)

### Pre-Setup

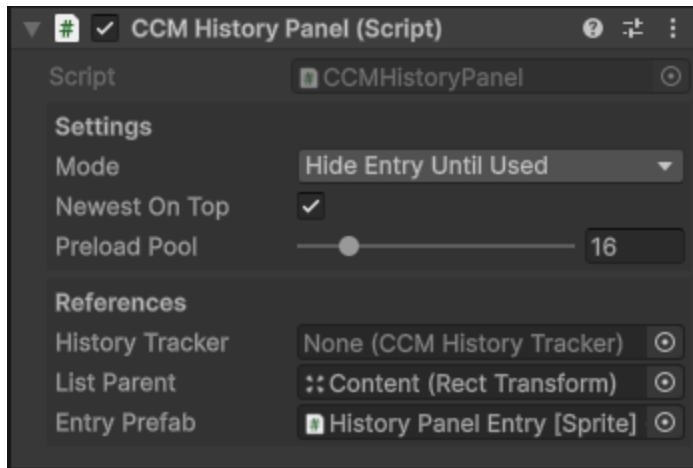
Located in the **Pre-Setup** folder.

These panels are ready to use:

1. Drag and drop into your Character Creation Menu.
2. Assign a **History Tracker**.

You'll need to assign a **History Tracker** in order for the panel to be functional.  
Don't have a **History Tracker** setup? [Read here](#).

The **History Tracker** field can be found in the **CCM History Panel** component at the root of the prefab.



---

## Entry Prefabs

Located in the **History List Entries** folder.

Included prefabs:

1. **History Panel Entry [Text]**
  - Displays a textual description of what changed in the snapshot.
2. **History Panel Entry [Sprite]**
  - Displays a collection of sprites with each sprite representing a layer of the character recorded in the snapshot.

These are the same prefabs used in the **Pre-Setup** History Panel prefabs.

---

## Creating a History Panel

All **History Panels** use the **CCM History Panel** component to populate entries from snapshot data.

## Setup Steps

1. Create a new **GameObject**.
2. Add the **CCM History Panel** component.
3. Assign a **History Tracker**.
  - [Setup Guide](#).
4. Assign a **List Parent**.
5. Assign an **Entry Prefab**.

[Existing entry prefabs](#) can be used or you can [create your own entry prefab](#).

---

## List Parent

The **List Parent** is the container where entries are instantiated.

A **Vertical** or **Horintal Layout Group** component can be used to automatically space entries.  
Or a **Scroll View** can be used which provides a scrollable list which overflow handling.

The **CCM History Panel** component will instantiate entries as children of this **GameObject**, however it's up to you to decide how these entries will be layed out.

---

## Panel Settings

| Setting              | Description                                                                                             |
|----------------------|---------------------------------------------------------------------------------------------------------|
| <b>Mode</b>          | Controls whether entries appear only when used or are always visible.                                   |
| <b>Newest On Top</b> | If enabled, inserts new entries at the top of the list. If disabled, inserts new entries at the bottom. |
| <b>Preload Pool</b>  | Number of entries pre-instantiated on menu initialization.                                              |

---

## Creating a History Panel Entry

Follow these steps to create your own **History Panel entry** from scratch:

### Setup Steps

1. Create a new **GameObject**.
2. Add the [History Panel Entry](#) component.
3. Set the **Display Mode**:
  - **Text** > Requires a text reference.
  - **Sprite** > Requires a sprite container reference (Parent GameObject).
  - **Text And Sprite** > Requires Both.
4. Add a **Toggle** component.
5. Assign the **Toggle** reference in the **History Panel Entry** component.
6. Add an **image** component (This will be the entries background).
7. Set the Toggle **Target Graphic** to this Image.

## Selection Highlight

8. Create a child **GameObject** and name it (e.g. **Highlight**).
  9. Add an **Image** component to it.
  10. Give it the sprite you want to be shown when selected.
  11. Assign this image to the Toggle **Graphic** field.
- 

## Finalizing

12. Drag the **GameObject** into the **Project window** to create a prefab.
13. Assign it to the **Entry Prefab** field on your **History Panel**.

### **TIP**

**Confused?** Check out one of the [included entry\\_prefabs](#) to understand how they're set up.

# History Undo/Redo

With the [History Tracker component](#) setup you can include buttons in your **Character Creation Menu** that undo or redo changes you've previously made.

## NOTE

A [CCM History Tracker](#) component **must** be present somewhere inside the **menu contents** for the Undo/Redo buttons to be functional.



## Prefabs

Location: [Prefabs](#) > [Character Creator](#) > [Modules](#) > [History](#) > [Undo-Redo](#)

These prefabs contain both **Undo and Redo buttons** pre-setup.

Drag and drop them anywhere inside the **contents of your Character Creation Menu**.

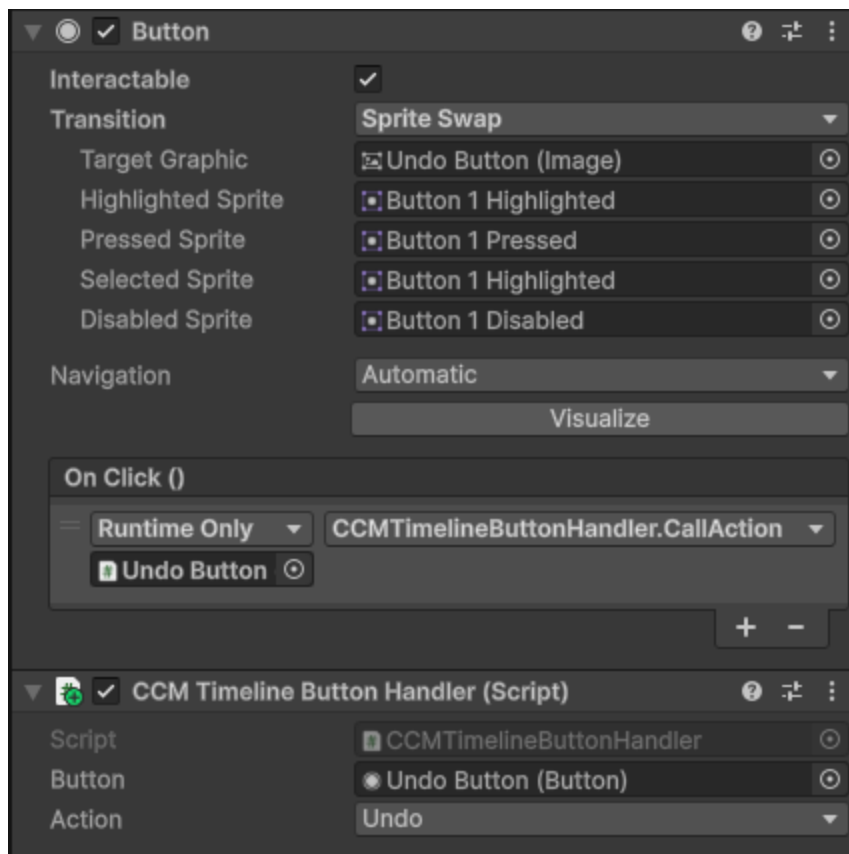
There are four prefabs in total. All are identical except for the sprites used.



## Manual Setup

Setup is extremely simple:

1. Add the [Button](#) and [CCMTimelineButtonHandler](#) component to any **GameObject** inside the your menu.
2. Assign the [Button](#) reference to the [CCMTimelineButtonHandler](#) component.
3. Set the [Action](#) to either **Undo** or **Redo**.
4. Set the buttons **On Click** event to call [CCMTimelineButtonHandler.CallAction](#).



If a button has nothing to undo/redo, the button will be automatically disabled until valid again.

# Character Randomization

The **Character Creation Menu** supports multiple ways to randomize a character's appearance. You can randomize:

- The **entire character**
  - **Selected layers only**
  - A **single layer**
- 

## Complete Character Randomization

This is the simplest form of randomization.

A single button randomizes all layers of the character at once.



---

## Prefabs

**location:** Prefabs > Character Creator > Modules > Randomization Controls.

Within are variation folders, each folder contains identical prefabs with different visual styles.

### Setup Steps:

1. Choose a variation folder.
2. Locate the **Randomize Character Button** prefab.
3. Drag and drop the prefab into the contents of your Character Creation Menu.

When the button is pressed in **Play Mode**, all layers of the character are randomized.

---

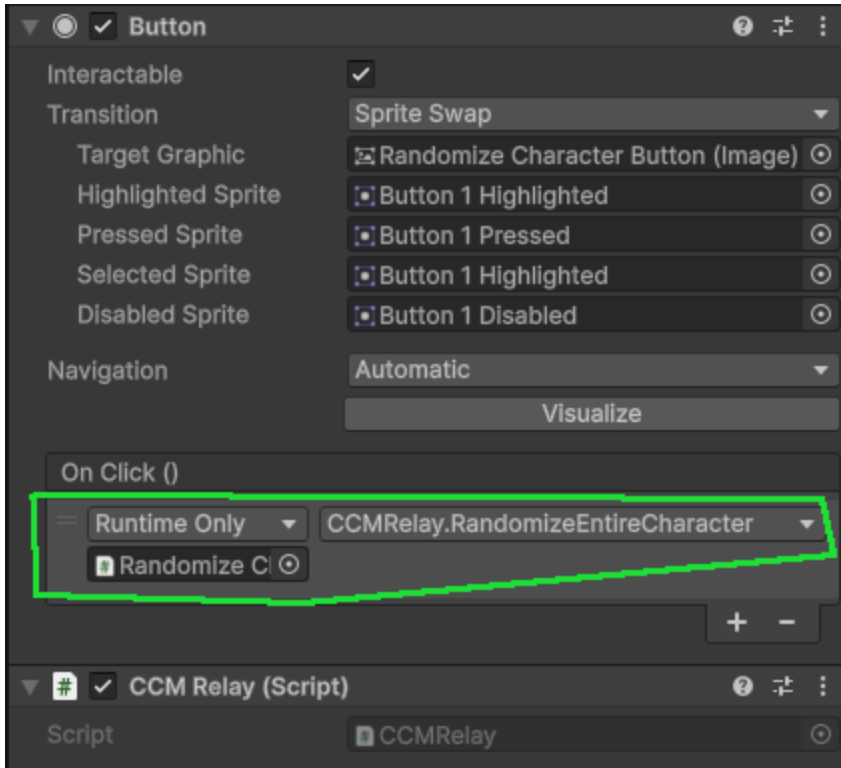
## Manual Setup

To create a complete randomization button from scratch:

1. Create a new **GameObject**.
2. Add the **Button** component.

3. Add the **CCM Relay** component.
  - See: [CCM Relay component](#).
4. Add an **On Click** event to the **Button**
5. Assign the **CCM Relay** component.
6. Select the **RandomizeEntireCharacter()** method.
7. Enter **Play Mode** and test your button.

When your button is pressed, all layers of your character should be randomized.



## Controlled Randomization

Controlled Randomization allows selective randomization of layers.

It consists of:

- A **Randomize** button.
- An **Options** button.

The **Options** button opens a popup listing all layers with toggles.

Only enabled layers are randomized when the **Randomize** button is pressed.



---

## Prefabs

location: Prefabs > Character Creator > Modules > Randomization Controls > (Variation folder) > Controlled Randomization.

Within The Randomization Controls folder are variation folders, each folder contains identical prefabs with different visual styles.

### Setup Steps:

1. Choose a variation folder.
2. Open the **Controlled Randomization** Subfolder.
3. Drag and drop the **Controlled Randomization** prefab into the contents of your Character Creation Menu.
4. Enter **Play Mode** and test your new randomization controls.

---

## Layer Toggle Prefab

Located inside the same folder.

The **Layer Toggle** prefab is used inside the options popup.

This prefab uses the CCM Layer Randomize Toggle component. It lives on the same GameObject as the Toggle component.

The CCM Layer Randomize Toggle component updates the Controlled Randomization Handler when its state is changed.

Additionally the component requires a reference to a text component which is used to display the name of the layer.



---

## Manual Setup

A **step-by-step** walkthrough to create your own **Controlled Randomization** prefab.

---

### GameObject Structure

Create three GameObjects:

- **Controlled Randomization** (Root GameObject)
  - **Buttons** (Container for controls, place inside root)
  - **Randomization Options Popup** (Toggle list UI, place inside root)
- 

### Handler Component Setup

1. Add the **CCM Controlled Randomization Handler** component to the **root** (Controlled Randomization) GameObject.
  2. Assign the **Randomization Options Popup** to the field of the same name.
  3. Assign the **Layer Randomize Toggle Prefab**. You can use the [pre-created prefab](#) or create your own.
- 

### Buttons Setup

1. Create two child GameObjects under the **Buttons** GameObject:
    - Options Button
    - Randomize Layers Button
  2. Add **Button** components to both.
  3. Configure events:
    - **Options Button**
      - Calls `ToggleRandomizationOptionsUI()`
    - **Randomize Layers Button**
      - Calls `RandomizeCharacter()`
- 

### Randomization Options Popup

The **Randomization Options Popup** GameObject should have a grouping component to space out the toggles.

This will usually be a **Horizontal**, **Vertical** or **Grid Layout Group**.

A **Content Size Fitter** component can also be used to automatically expand the parent GameObject to the size the toggles take up. This is useful for a background to automatically adjust its size as toggles are added or removed.

Toggles can be added directly in the editor.

**At runtime:**

- If too few toggles exist - More are created.
- If too many exist - Extras are disabled.

The **Randomization Options Popup** GameObject can be enabled or disabled in the editor. Its state will be automatically managed during runtime.

---

## Single Layer Randomization

Some **Layer Selectors** include a variant with a built-in randomize button that affects only that layer.

Check each Layer Selectors **prefab folder** to see if it includes a variant with randomization controls.



## Adding a Randomize Button to a Layer Selector

If a Layer Selector does **not** include a variant with a randomize button, one can be added:

1. Create a new **GameObject**.
2. Add the **Button** component.
3. Add an **On Click** event.

4. Assign the **Layer Selector** Component to the event and select the [CharacterLayerSelector.RandomizeLayer\(\)](#) method.

# Loading Screens

When a **Character Creation Menu** is first opened it can take time before the menu is ready as it needs to load all layer options first.

A **loading screen** can be used to hide the **Character Creation Menu** while it loads and give the player something to look at until the menu is ready.



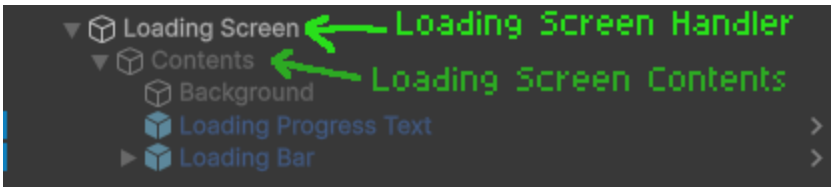
## How It Works

Just like the Character Creation Menu itself, loading screens are also modular. Pieces can be added or removed to create your own loading screen.

**The Hierarchy is as follows:**

1. **First GameObject** manages the loading screen and holds the [CCMLoadingScreenHandler](#) component. This GameObject should be kept enabled.
2. **Second GameObject** holds the actual contents of the loading screen. This is the GameObject the **Loading Screen Handler** enables and disables. This GameObject can be kept enabled or disabled, its state in the editor doesn't matter since it will be controlled by the **Loading Screen Handler**.

The **Contents** GameObject will hold the loading screen background and any additional pieces of the loading screen such as a progress bar/text or loading icon.



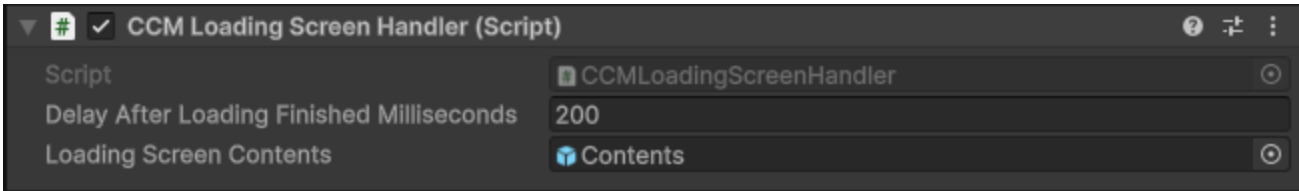
## Loading Screen Handler Component

The [CCMLoadingScreenHandler](#) component is responsible for toggling the state of a loading screen.

When the Character Creation Menu is enabled, the Loading Screen Handler automatically enables the loading screen and listens for when the Character Creation Menu finishes loading. Once finished, it disables the loading screen.

The component has two fields:

| Field                               | Description                                                                           |
|-------------------------------------|---------------------------------------------------------------------------------------|
| <b>Delay After Loading Finished</b> | The time in milliseconds to wait before disabling the menu after loading has finished |
| <b>Loading Screen Contents</b>      | The parent GameObject containing the actual contents of the loading screen            |



## Prefabs

Loading Screen prefabs are available to drop into your own project without the need of setting them up yourself.

**Location:** [Prefabs](#) > [Character Creator](#) > [Modules](#) > [Loading Screen](#).

## Core Prefab

Contained within the **Core** Subfolder.

A loading screen in its most basic state. Only a background to display during the loading process.

This prefab can be added to using the [Additions prefabs](#).

## Pre-Setup Prefabs

Contained within the **Pre-Setup** subfolder.

These are Loading Screens with additional features included. They're drag and drop solutions, meaning once added, no other setup is required.

Four **Pre-Setup** loading screens are included. Each one uses a different background and different elements such as loading bars, progress text, loading icons, etc.



---

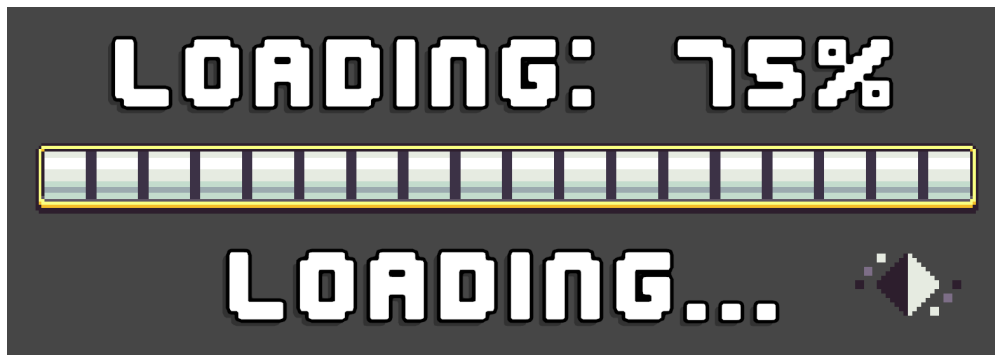
## Additions Prefabs

Contained within the **Additions** Subfolder.

Additional prefabs that can be added to the [Core loading screen prefab](#).

**These prefabs include the following:**

- **Loading Bar** - A progress bar that fills as the Character Creation Menu loads.
- **Loading Icon** - An animated icon that spins repeatedly.
- **Loading Progress Text** - Text which displays the percentage the Character Creation Menu has loaded from 0%-100%
- **Loading Repeating Text** - Text which cycles through a list of preset sentences at a set interval.



---

## Loading Screen Components

### NOTE

Most loading screen components require a reference to the **Loading Screen Handler**.

These components can be added to a loading screen to add additional features.

They're also used in the [Additions Prefabs](#). If you're looking for a simple drag and drop implementation, use those instead.

---

### LoadingScreenProgressText Component

- Adds progress text indicating how far loading has progressed.
- Requires a **TMP Text** component reference.
- **Loading String** can be modified to change the loading bar text. **{0}** will be replaced with the loading percentage.
- **Smoothing Speed** value can be changed to modify lerp speed through loading progress.

[Component Link](#)

---

### LoadingScreenProgressBar Component

- Adds a progress bar which fills up as the loading progresses.
- Requires an **image** component reference
- The **image** components **Image Type** should be set to filled. The **Fill Amount** can then be modified automatically to fill up the progress bar.
- **Smoothing Speed** value can be changed to modify loading bar lerp speed through loading progress.

[Component Link](#)

---

### LoadingScreenRepeatingText Component

- Adds text which cycles through a list of pre-set strings.
- Requires a **TMP Text** component reference.
- A list of strings containing the repeating text must be set.
- Repeating text cycle duration can be modified (Default = 0.5 seconds).

[Component Link](#)

# Character Display Name Field

The **Character Display Name Field** allows the player to give their character a display name. This name can later be used anywhere in your game, such as display it above the character's head.



---

## Prefabs

**Location:** Prefabs: Character Creator > Modules > Character Display Name Field

Two prefabs are included:

- **Character Name Field** - Contains the text field only.
- **Character Name Field [+Title]** - Contains the text field with a title displayed above it.

Add either of these prefabs as a child of the Character Creation Menu Contents.

---

## Manual Setup

The Character Display Name Field uses a **TextMeshPro Input Field** to allow the player to enter a name.

To setup one up manually:

1. Add a **TMP Input Field** to the Character Creation Menu Contents.
2. Add the **CCM Display Name Input Field Handler** component to the same GameObject.
3. Assign the **TMP Input Field** to the handler's input field reference.

When the Character Creation Menu is opened, the input field will automatically be populated with the character's current display name, if one has been set.

When the character is saved, the value entered in the input field is saved as the character's display name.

---

## Related

- [Character Display Name Renderer](#)

# Character Grouping System

Character groups are a convenient way to store/sort characters. They are for [Layered Characters](#) only. Characters in a group are automatically saved to disk and loaded again next session.

---

## The Primary Character Slot

Every Character Type contains a **Primary Character Slot** which can hold a singular character. This is useful for simple setups where only one character is used.

The main use for the **Primary Character Slot** is for use with the [Character Creation Menu](#).

The menu can be opened using the character contained within the **Primary Character Slot**.

---

## Group Types

**There are two types of groups:**

### Flexible Group Type

A dynamic list. Characters can be added, removed or edited at any time.

Flexible Groups are great when you want the player to have full control of the list. For example a list of playable characters with the ability for the player to create new characters.

[API](#)

---

### Fixed Group Type

A list of a pre-determined amount of characters. The number is set when the list is created and all characters are created immediately. Characters within the list can be edited at anytime but cannot be removed and new characters cannot be added.

Best for situations where an immutable list is needed. Such as a set of pre-created characters the player can edit the designs of but not change the count. Or a set of playable characters the player can choose from.

[API](#)

---

## Character Group Registries

The [CharacterGroupRegistry](#) is a wrapper class. An instance of this class exists for each **Layered Character Type**. This wrapper class holds the [Primary Character Slot](#) as well as all flexible and fixed

groups created for the same **Layered Character Type**.

---

## Character Group Manager

The [LayeredCharacterGroupManager](#) is a static class which handles all groups.

The Layered Character Group Manager contains a dictionary of type `CharacterGroupRegistry`. One entry for each **Layered Character Type** used.

The `LayeredCharacterGroupManager.GetGroupRegistryForType(LayeredCharacterTypeSO type)` method can be used to retrieve the registry for a specific **Character Type**.

---

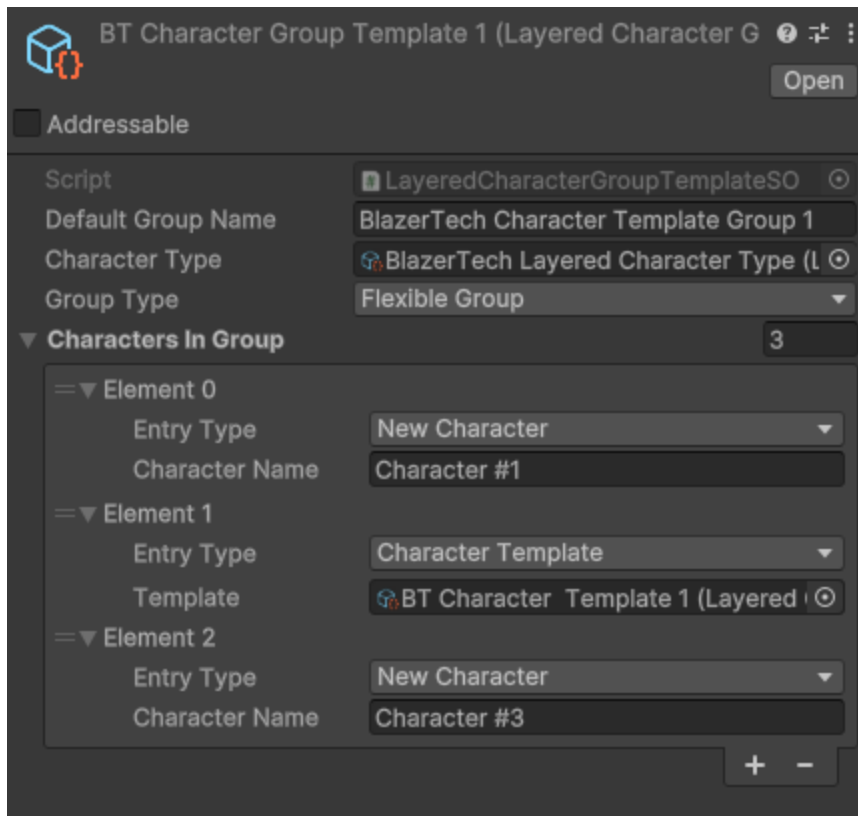
## Character Group Templates

A **Layered Character Group Template** is a scriptable object. It is a blueprint for creating a flexible or fixed group during runtime.

To create a Layered Character Group Template **right click** the project window and navigate to:  
**Create > BlazerTech > Character Management System > Character Templates > Layered Character Group Template**

### TIP

If a **Layered Character Group Template** is placed inside a folder titled `Resources`, the group will be automatically created at runtime.



## Fields

- **Default Group Name** - The name of the group when it's created. Can be overridden.
- **Character Type** - The Character Type the group will be for when created.
- **Group Type** - The type of group that will be created. Flexible or Fixed.
- **Characters In Group**
  - A list of all characters that will be created and put in the group when the group is created. Each entry has two options:
  - 1. **New Character**
    - Creates a new character with the supplied name.
  - 2. **Character Template**
    - Creates a new character using the supplied Layered Character Template.

# Character Group Editor

The Character Group Editor is a menu for players to view and manage characters stored in a [Character Group](#).



It provides an interface for displaying characters in a group and, when configured as a Flexible group, lets players create and remove characters from the group.

Both Fixed and Flexible character groups are supported. Which one is used is determined by the **Group Type** field on the `CharacterGroupEditor` component.

## TIP

The fastest way to get started is to drop one of the [premade menu prefabs](#) into your scene and change its **Group Name**, **Group Type**, and **Layered Character Type**. Everything else is already set up.

## How It Works

When a Character Group Editor is enabled, it looks for a group matching the configured **Group Name** and **Group Type**.

- If a matching group already exists, that group is used.
- If no matching group exists, a new group is created automatically.

This means a character group does not need to be created manually beforehand.

## Group Identification

A group is identified by three things: its **Layered Character Type**, its **Group Name**, and its **Group Type**.

Because the type is part of the identity, a Fixed group and a Flexible group can share the same name without conflicting.

## Fixed Groups

When the editor is configured to use a **Fixed** group, the **Fixed Group Size** must be set in the inspector.

All characters required by the group are created at once when the group is created, and characters cannot be added or removed afterwards.

The editor then creates one list entry per character in the group.

## Flexible Groups

When the editor is configured to use a **Flexible** group, the group starts out empty.

- If **Create New Characters Permission** is enabled, players can add characters through the New Character entry.
- If **Remove Characters Permission** is enabled, players can remove characters using the remove button on each list entry.

---

## Setup

Add the `CharacterGroupEditor` component to the parent `GameObject` of your Character Group Editor menu. If you are using one of the premade prefabs, the component is already present.

The sections below cover every setting on the component.

## Group Settings

| Property                               | Description                                                                                           |
|----------------------------------------|-------------------------------------------------------------------------------------------------------|
| <a href="#">Layered Character Type</a> | The <a href="#">Layered Character Type</a> used by the Character Group.                               |
| <a href="#">Group Type</a>             | Determines whether the editor uses a Fixed or Flexible Character Group.                               |
| <a href="#">Group Name</a>             | The name used to find an existing Character Group or create a new one.                                |
| <a href="#">Fixed Group Size</a>       | The number of characters in a Fixed group. Only shown when <b>Group Type</b> is set to <b>Fixed</b> . |

## Layered Character Type

Assign the Layered Character Type that the characters in the group should use.

**Character Groups** are separated **by Character Type**, meaning a character group with the **same name** and **type** won't be found if it was created with a different Character Type.

## Group Type

Select the type of group the editor should use:

- **Flexible** - Characters can be added and removed from the group after creation.
- **Fixed** - The group contains a fixed number of characters. Characters cannot be added or removed after creation.

## Group Name

The **Group Name** is used to find or create the **Character Group**.

If a group with the same named and type already exists, the existing group is loaded. Otherwise, a new group is created.

A **Fixed** and **Flexible** group can have the same Group Name because groups are also separated by type.

## Fixed Group Size

Only shown when **Group Type** is set to **Fixed**.

Specifies the number of characters the group should contain.

---

## Permissions

Flexible Character Groups provide two permissions on the `CharacterGroupEditor` component.

| Property                                | Description                                                                                                 |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Create New Characters Permission</b> | Determines whether new characters can be added to the list, and whether the New Character entry is created. |
| <b>Remove Characters Permission</b>     | Determines whether existing characters can be removed from the list.                                        |

Both settings apply only to **Flexible Character Groups**.

Disabling **Create New Characters Permission** prevents the New Character entry from being created.

Disabling **Remove Characters Permission** hides the remove button on character list entries.

---

## Entry Click Action

The **Entry Click Action** determines what happens when a character in the list is selected. **Edit** is the default.

| Action               | Description                                                       |
|----------------------|-------------------------------------------------------------------|
| <b>None</b>          | Disables interaction with the character entry.                    |
| <b>Edit</b>          | Opens the Character Creation Menu to edit the selected character. |
| <b>Custom Action</b> | Invokes a UnityEvent assigned by the developer.                   |

### None

Entries remain visible but interaction is disabled, selecting a character entry does not perform an action.

### Edit

Clicking a character calls the Character Creation Menu Manager to open the Character Creation Menu for that character.

An instance of a Character Creation Menu must be enabled in the scene for this to work.

[Read More → Character Creator Setup](#)

### Custom Action

Custom Action lets you assign your own behaviour through a UnityEvent.

The selected LayeredCharacter is passed to the event, so the character is available to your own methods. Use this to open a custom character information screen, select a character for gameplay, or run any other logic.

To run extra logic around the standard edit behaviour, add CharacterGroupEditor.EditCharacter to the event list. That call opens the Character Creation Menu, and any events placed before or after it run in order.

---

## References

The References section contains the prefabs and GameObject references required to build the character list.

| Reference                              | Description                                                       |
|----------------------------------------|-------------------------------------------------------------------|
| <b>List Entry Prefab</b>               | Prefab used for each character in the list.                       |
| <b>New Character List Entry Prefab</b> | Prefab used for the New Character entry in Flexible groups.       |
| <b>List Parent</b>                     | Transform that acts as the parent for all character list entries. |

A **New Character List Entry Prefab** is only required when using a Flexible Character Group.

### ⊗ IMPORTANT

The List Parent should contain only the list entries used by the Character Group Editor. Any existing children that do not contain a `LayeredCharacterListEntry` component are removed when the editor initializes at runtime.

## Entry Components

### CharacterGroupEditorEntry

The [CharacterGroupEditorEntry](#) component is used by character list entries.

It supports three display modes:

| Display Type           | Contents                             |
|------------------------|--------------------------------------|
| <b>Text</b>            | Character name                       |
| <b>Sprite</b>          | Character preview                    |
| <b>Text And Sprite</b> | Character preview and character name |

The component handles the remove button automatically. The button is shown only when both of the following are true:

- The Character Group Editor allows character removal.
- The Group Type is set to Flexible.

### CharacterGroupEditorNewEntry

The [CharacterGroupEditorNewEntry](#) component is used by the New Character entry, the entry that appears at the start of a Flexible Character Group list.

When selected, it opens the Character Creation Menu to create a new character in the group.

---

## Prefabs

Ready-to-use prefabs, including complete premade menus and the individual modules needed to build your own.

**Location:** Prefabs > Character Creator > Character Group Editor

## Premade Character Group Editors

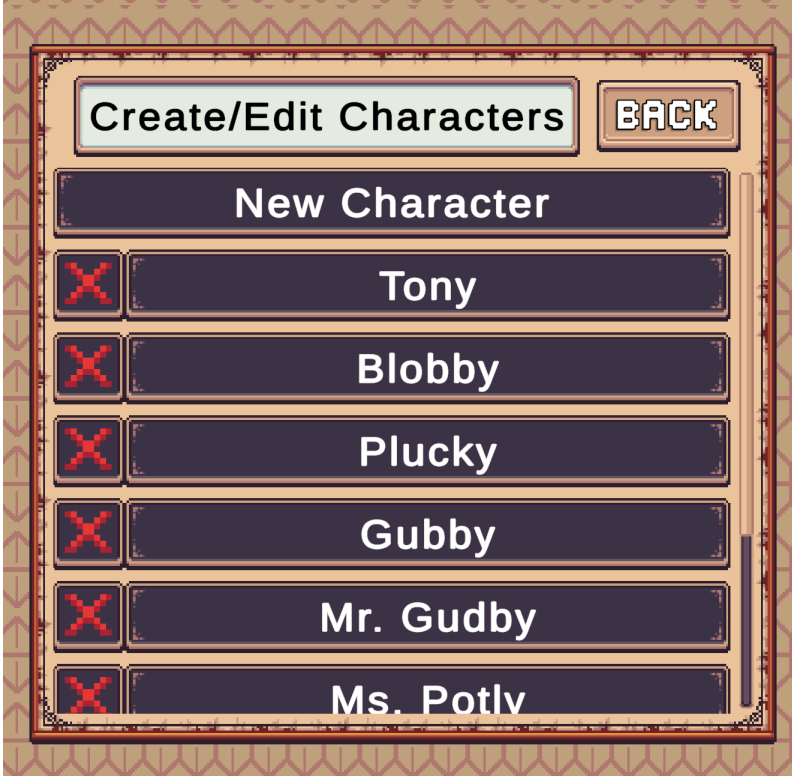
Complete menus with everything wired up. These are the easiest way to get started.

**Location:** Prefabs > Character Creator > Character Group Editor > Premade Character Group Editors

Two folders are included, **Fixed** and **Flexible**, each containing two menu prefabs:

- **Text Vertical List** — uses a `VerticalLayoutGroup`, intended for traditional vertical character lists.
- **Sprite Grid** — uses a `GridLayoutGroup` to arrange character entries into a grid.

These prefabs can be customized freely. Backgrounds, layout, and element arrangement can all be changed; the `CharacterGroupEditor` only needs a reference to a `GameObject` to instantiate character entries into.



## Character List Entries

Prefabs used as entries in the character list.

**Location:** Prefabs > Character Creator > Character Group Editor > Character List Entries

Four folders are provided.

## Sprite Based Entries

Displays a preview of the character only. Does not display the character's name.



## Text Based Entries

Displays the character's name only, with no visual representation of the character.

Uses the character's Display Name if one is set, otherwise the character's name.

Useful for compact lists.



## Sprite and Text Based Entries

Displays both a preview of the character and the character's name.



## New Character List Entry

The prefabs used for the first entry in a Flexible Character Group. Two variants are included:

- **New Character Entry [Sprite]** — displays a character skeleton sprite (changeable) and the text **New Character**.
- **New Character Entry [Text]** — displays only the text **New Character**, intended for compact lists.



The New Character entry is instantiated as the first item in the list. Selecting it opens the Character Creation Menu to create a new character, and the character is added to the Flexible Character Group when it is saved.

## Loading Screens

**Location:** Prefabs > Character Creator > Character Group Editor > Loading Screens



Four loading screen prefabs are included, each using a different background.

These work much like the loading screens included with the Character Creation Menu, with one difference: a Character Group Editor loading screen needs an explicit reference to the `CharacterGroupEditor` component.

Once the component has been added to your menu, assign it to the loading screen prefab so the loading screen can receive progress and setup information from the editor.

---

## Manual Setup

A Character Group Editor can be assembled manually. Building it yourself gives you full control over how the menu looks and behaves.

A Character Group Editor menu follows this general GameObject structure:

```
Character Group Editor
├── Contents
└── Loading Screen
```

### 1 Create the Character Group Editor

Create a GameObject named `Character Group Editor`.

Add the `CharacterGroupEditor` component to it.

Configure its [Group Settings](#), [Permissions](#), [Entry Click Action](#), and [References](#).

### 2 Create the Contents

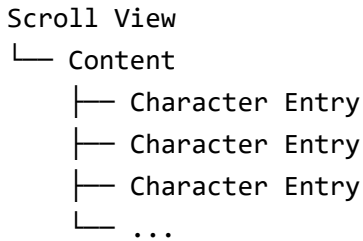
Create a child GameObject named `Contents`. This acts as the container for the visible menu and can hold any UI structure your game requires.

A typical Character Group Editor might look like this:

```
Character Group Editor
├── Contents
│   ├── Background
│   ├── Title
│   ├── Back Button
│   └── Scroll View
│       └── Content
└── Loading Screen
```

### 3 Create the List Parent

Create a GameObject to act as the parent for the character entries. For a typical ScrollView this is the ScrollView's Content object:



Assign this Transform to the List Parent field on the `CharacterGroupEditor`. Every character in the Character Group is represented by a list entry under this Transform.

#### TIP

A `ContentSizeFitter` with **Vertical Fit** set to **Min Size** is recommended when using a vertically scrolling list. This allows the Content object to automatically expand to accommodate the number of character entries created by the Character Group Editor. Without an appropriately sized Content object, the ScrollView may not correctly determine the size of its scrollable area.

The same concept can be applied to other layouts when the list is arranged horizontally or as a grid.

## **4** Add the List Entry Prefab

You can either use a premade prefab or build your own.

### Option 1: Use a premade list entry prefab

Assign any of the [Character List Entry prefabs](#) to the **List Entry Prefab** field on the `CharacterGroupEditor`. These are the same prefabs used by the [premade menus](#).

### Option 2: Create your own prefab

To create your own Character List Entry follow these steps:

1. Create a new GameObject.
2. Add the `CharacterGroupEditorEntry` component.
3. Set **Display Type** (Text, Sprite, or Text And Sprite).
4. Set the **Character Name Text** reference (if Display Type is Text or Text And Sprite).
5. Set the **Character Image** reference (if Display Type is Sprite or Text And Sprite).
6. Set the **Select Character Button** reference.
7. Set the **Remove Character Button** reference.
8. Drag the parent GameObject into the Project window to turn it into a prefab.
9. Assign the prefab to the **List Entry Prefab** field on the `CharacterGroupEditor`.

A typical GameObject structure will look something like this:

Character Entry

- ├ Character
  - ├ Background
  - ├ Character Sprite
- ├ Character Name Text
- └ Remove Character Button

## 5 Configure Flexible Groups

When using a Flexible Character Group, assign a **New Character List Entry prefab**.

Use one of the premade prefabs located at:

Prefabs > Character Creator > Character Group Editor > Character List Entries > New Character Entry

Or create your own:

1. Create a new GameObject.
2. Add the `CharacterGroupEditorNewEntry` component.
3. Drag the parent GameObject into the Project window to turn it into a prefab.
4. Assign the prefab to the **New Character List Entry Prefab** field on the `CharacterGroupEditor`.

The Character Group Editor creates this entry before the character entries and places it at the beginning of the list.

If **Create New Characters Permission** is disabled, the **New Character Entry** is not created.

## 6 Add a Loading Screen

You can either use a premade loading screen or build your own.

### Option 1: Use a premade loading screen

Add one of the provided [loading screen prefabs](#) as a child of the **Character Group Editor** GameObject.

Assign the parent `CharacterGroupEditor` component to the loading screen's required reference. The loading screen then enables itself automatically while the editor is loading.

### Option 2: Create your own loading screen

1. Create another child GameObject under **Character Group Editor**.
2. Add the `CharacterGroupEditorLoadingScreenHandler` component.
3. Assign the parent `CharacterGroupEditor` component to it.
4. Create a new child GameObject named `Contents` and assign it to the `Contents` field.

5. Add a new GameObject named **Background** with an **Image** component that covers the whole screen, as a child of **Contents**.

The **Contents** GameObject is automatically enabled while the Character Group Editor is initializing.

Additional functionality can be added to the loading screen the same way as Character Creation Menu loading screens. Refer to [Loading Screen Components](#) for more components you can use to add features to your loading screen.

---

## Related

- [Character Grouping System](#)